

Academic Thesis for the Attainment of the Degree of
Master of Science
at the Technical University of Munich

An Algorithmic Framework for Very Large-Scale Capacitated Vehicle Routing Problems

Examiner: Prof. Dr. Stefan Minner
Chair of Logistics and Supply Chain Management
Technical University of Munich

Supervisor: Till Krieger, M.Sc.

Degree Program: Master of Management and Technology

Submitted by:
Lucca Altendeitering
Stolzestr. 21, 44789 Bochum
Tel.: +49 162 7630077
Matriculation Number: 03782307

Date of Submission: 30.09.2026

An Abstract should not include more than 150 words.

Table of Contents

List of Figures	iv
List of Tables	v
Abbreviations	vi
1 Introduction	1
2 Literature Review	5
2.1 Problem Statement and Benchmark Instances	5
2.2 Monolithic Solvers for Large-Scale CVRP	6
2.3 Scaling and Configuring Routing Heuristics	8
2.3.1 Parallel Search	8
2.3.2 Route Pools and Set Partitioning	9
2.3.3 Decomposition Strategies	10
2.3.4 Repairing Decomposition Boundaries	11
2.3.5 Adaptive Operator Selection and Algorithm Configuration	12
2.3.6 Learned Decomposition	13
2.4 Algorithmic Frameworks	14
2.5 Research Gap	16
3 Framework	18
3.1 Hierarchical Adaptive Operator Selection	19
3.2 Decomposition	23
3.3 Routing	25
3.4 Boundary-Guided Adaptive Iterated Local Search	26
3.5 Route Pool and SC/SP Recombination	32
3.6 Post-Recombination Improvement	35
3.7 Parallelized Pipeline Orchestration	36
4 Computational Study	39
4.1 Experimental Design	39
4.1.1 Dataset and Reference Solutions	39
4.1.2 Implementation and Hardware Specifications	40
4.1.3 Hyperparameters	41
4.1.4 Replication Design	41
4.2 Results	44
4.2.1 Comparison with Monolithic Solvers	44

4.2.2	Performance by Instance Characteristics	49
4.2.3	Learned HAOS Weights	54
4.2.4	Convergence Behavior	61
4.2.5	Contribution by Stage	63
5	Conclusion	66
5.1	Findings	66
5.2	Limitations	68
5.2.1	Validity of the Comparison	69
5.2.2	Scope of the Study	70
5.3	Future Research	71
5.3.1	Repairing the Controller	72
5.3.2	Untested Comparisons	73
5.3.3	The Long-Route Regime	73
5.3.4	Transfer to Other Problem Classes	74
	Bibliography	75
	Appendix	84
A	Instance and Solution Maps	84
B	Set Covering Formulation	86
C	DRISPI's Instance-Level Results for the XL Set	87

List of Figures

3.1	Sequential Flowchart of Decompose-Route-Improve-Set-Partition-Improve (DRISPI).	19
3.2	Illustrative BG-AILS Call on Instance XL-n1281-k29. Colorbar Applies to Panel (d) Only.	29
3.3	Per-Instance Mean Cost Improvement (a) and Paired BG-AILS Differences (b).	31
3.4	Two Consecutive DRISPI Iterations in One 8-Core Container (6 for DRI, 1 for BG-AILS, and 1 for SP). Block Lengths Are Illustrative.	36
4.1	Empirical CDF of the Mean-of- N Gap to the Current Best Known Solution (BKS).	47
4.2	Mean-of-3 Gap to the Current BKS Against Customer Count, Average Route Length, and Log Average Demand per Customer.	51
4.3	Per-Instance Seed Standard Deviation Against Customer Count and Average Route Length.	52
4.4	Mean-of-3 Gap to the Current BKS by Instance Characteristic.	53
4.5	Normalized Shannon Entropy of the Six HAOS Wheels Against Decompose-Route-Improve (DRI) Iteration.	55
4.6	Mean Immediate Reward of Each Arm Against Its Final Wheel Probability. Each point represents one arm in one run with at least five draws.	57
4.7	Immediate (a) and Deferred (b) HAOS Rewards, Excluding the Ten Warm-Up Iterations.	58
4.8	Mean Pairwise Total-Variation Distance Between the Three Seeds' Final Wheel Probabilities, Against the Instance's Mean Number of Completed DRI Iterations. The distance is $\frac{1}{2} \sum_a w_a - w_b $, averaged over the three seed pairs. The line is a LOWESS smooth.	60
4.9	Convergence Over the 7,200-Second Budget. (a) Number of Incumbent Updates By Wall-Clock Mark, (b) Mean-of-3 Gap By Route Length to the Current BKS at Each Wall-Clock Mark	62
A.1	Unsolved Instances XL-n3888-k1010 and XL-n9571-k55 for Illustrative Purposes.	84
A.2	Solved Instances of XL-n3888-k1010 and XL-n9571-k55 (Seed 11 Solutions).	85

List of Tables

2.1	Research Gap Matrix for Large-Scale Routing Methods.	17
3.1	Default Immediate and Deferred HAOS Rewards.	22
3.2	Inputs Used By Each Decomposition Method.	24
4.1	Hyperparameters of the Benchmark Configuration.	42
4.2	Forecast Standard Error and 95% Half-Width of $\bar{G}$ by Seed Count S . . .	43
4.3	DRISPI Mean Gap to Current BKS Comparison Against Benchmark Solvers, Sorted by Ascending Mean-of- N Gap Over All Instances. . . .	44
4.4	Paired Comparison of DRISPI's Mean-of-3 Gap Against Each Published Solver's Mean-of-60 Gap, Over the 100 XL Instances.	47
4.5	DRISPI's Best-of-3 Cost Matching or Beating a Published Reference. . .	48
4.6	DRISPI's Largest Mean-of-3 Gap to Current BKS.	49
4.7	Share of Incumbent Updates, Cost Reduction, and Recorded Stage Time.	64
4.8	The Same Shares Split by Route Length and by the $n = 3,400$ Cut. . . .	64
C.1	Instance-Level Best-of-3 Results for DRISPI on the 100 XL Instances. .	87

Abbreviations

ALNS	Adaptive Large Neighborhood Search.
BG-AILS	Boundary-Guided Adaptive Iterated Local Search.
BKS	Best Known Solution.
CVRP	Capacitated Vehicle Routing Problem.
DRI	Decompose-Route-Improve.
DRISPI	Decompose-Route-Improve-Set-Partition-Improve.
DRSCI	Decompose-Route-Set-Covering-Improve.
HAOS	Hierarchical Adaptive Operator Selection.
HGS	Hybrid Genetic Search.
HVRP	Heterogeneous Vehicle Routing Problem.
ILS	Iterated Local Search.
SC	Set Covering.
SP	Set Partitioning.
TSP	Traveling Salesman Problem.
VRPTW	Vehicle Routing Problem with Time Windows.

1 Introduction

Moving and storing goods is expensive. In 2025, business logistics costs in the United States reached \$2.4 trillion, or 7.8% of GDP (CSCMP, 2026). Transportation alone accounts for 23% of global energy-related CO₂ emissions (ITF, 2023, p. 46), and the International Transport Forum projects urban freight activity to grow by a factor of 2.6 between 2019 and 2050 (ITF, 2023, p. 91). Parcel delivery shows the scale most directly. Carriers in Germany handled 4.37 billion courier, express, and parcel shipments in 2025 (BPEX and KE-CONSULT, 2026, p. 8), and carriers in the United States 23.1 billion (Pitney Bowes, 2026, p. 3). Where route planning is already automated, further savings have to come from better solutions. At full deployment, the ORION route optimization system at UPS was estimated to cut about 100 million miles a year from roughly 55,000 daily routes (Holland et al., 2017, pp. 8, 19). Those are 100 million miles saved that do not cause any more costs, emissions, and delays.

Modern parcel carriers plan their delivery tours per depot. A single depot can serve several thousand customers a day. Instances that Arnold et al. (2019) derive from Belgian parcel operations range from a few thousand to 30,000 customers. Each such plan can be modeled as an instance of the Capacitated Vehicle Routing Problem (CVRP). A homogeneous fleet of vehicles with limited capacity leaves a depot, visits every customer exactly once, and returns at minimum total travel cost (Toth and Vigo, 2014). Once customers and fleet are fixed, the route plan decides the travel cost. A small relative improvement that repeats on every tour of every day can add up to a significant cost reduction in absolute terms. But the plan also has to be ready within hours and at this scale, a method is only useful if it reaches a good solution inside a fixed wall-clock budget.

This is why benchmark research has recently moved to the same scale. Queiroga et al. (2026) released an XL problem set of CVRPLib. The set contains 100 instances ranging from 1,000 to 10,000 customers, generated under a similar scheme of the X instances (Uchoa et al., 2017). The release also reports initial Best Known Solution (BKS) values from two-hour runs of eight published solvers. With the release they opened a CVRPLib BKS Challenge on January 12, 2026, allowing any team to submit an improved solution for automatic verification. Within a month, the challenge had improved 99 of the 100 initial values. The XL set therefore combines a deliberately varied CVRP instance set under a common time budget with a reference that moves as the community improves it.

The CVRP is NP-hard (Lenstra and Rinnooy Kan, 1981). And while branch-cut-

and-price algorithms prove optimality for instances with a few hundred customers (Pecin et al., 2017; Pessoa et al., 2020), that size is an order of magnitude or more below the XL range. Solutions in that range come from metaheuristics. These methods trade the optimality guarantee for a feasible solution within a practical budget. On the XL set the strongest of these methods are monolithic. A monolithic solver keeps one complete solution, or a population of complete solutions, under a single search process. Of the 100 initial BKS values, AILS-II (Máximo et al., 2024) found 93, FILO2 (Accorsi and Vigo, 2024) six, and FILO (Accorsi and Vigo, 2021) one.

These solvers scale through engineering inside one solution. Granular neighborhoods and selective vertex caching restrict move evaluation to promising arcs and recently changed vertices (Toth and Vigo, 2003; Accorsi and Vigo, 2021). The cost of neighborhood evaluation, the memory for move descriptors, and the number of iterations needed to reach a given quality still grow fast with the customer count (Arnold et al., 2019; Accorsi and Vigo, 2021). Among the eight solvers that Queiroga et al. (2026) compare, the hybrid genetic search HGS-CVRP (Vidal, 2022) has the best mean quality on instances with up to about 330 customers. AILS-II leads on the upper half of the X set and on XL. No single solver leads across the size range. Additionally, they also run with a single configuration on every instance. But X and XL instances vary depot position, customer distribution, demand, and route length intentionally. A configuration ideally tailored to each instance would require long and computationally expensive tuning campaigns for every single instance.

Decomposition addresses the first of these limits and makes the second explicit. Splitting the customer set into subproblems of a few hundred customers moves each subproblem into the size range where HGS-CVRP leads. The resulting independent subproblems could also be solved in parallel. The Decompose-Route-Improve (DRI) framework of Kerscher and Minner (2025) and its set-covering successor Decompose-Route-Set-Covering-Improve (DRSCI) (Kerscher et al., 2026) wrap existing routing software in this way. Both note that their subproblems could be solved in parallel, but the published experiments run on a single thread only so far. Santini et al. (2023) show that route-based decomposition improves both Adaptive Large Neighborhood Search (ALNS) and Hybrid Genetic Search (HGS) on the X instances. The benefit, however, depends on the strategy and the instance, and a poor decomposition can fail to help just as easily.

Selecting which decomposition to apply and which subsolver to run on the subproblems is therefore a configuration choice of its own. State-of-the-art methods settle it by a fixed rule or calibrated default (Queiroga et al., 2021; Santini et al., 2023), by a clustering-quality criterion evaluated once per instance (Kerscher and Minner, 2025), or by a policy trained before the instance is seen (Li et al., 2021; Ye et al., 2024). DRSCI comes closest to adapting within a run. It narrows the range of cluster counts to the interval spanned by the counts that contributed routes to the current

best solution. But it draws uniformly inside that range and credits neither the clustering paradigm nor the routing strategy with the quality of what they produced. No published decomposition framework weights its decomposition and subsolver choices by the solution costs observed on the instance it is solving (Chapter 2.5).

This thesis builds on an unpublished project study that adapted DRSCI to the CVRP (Altendeitering and Bachmann, 2026) for the above-mentioned CVRPLib BKS Challenge. That study drew its decomposition and routing configuration uniformly at random in every iteration and reached a mean gap of 0.53% to the initial XL BKS. It left open whether a framework that learns this choice during the run can close the remaining distance to the monolithic solvers.

This thesis develops DRISPI, a decomposition framework for very large CVRP instances. Its controller, Hierarchical Adaptive Operator Selection (HAOS), is designed to learn that choice within a single run. The central goal of this thesis is to assess the frameworks performance, and therefore the primary research question is formulated as follows.

RQ1: Can an adaptive decomposition framework that learns its decomposition and routing strategy within a single run match or outperform state-of-the-art monolithic solvers on very large CVRP instances within the same two-hour wall-clock limit per run?

To answer the question, this thesis presents a benchmark campaign across all 100 XL instances. DRISPI ran three times per instance under one shared configuration, with a two-hour wall-clock limit per run. Its mean gap to the current BKS is then compared against the published mean-of-60 results of the eight solvers benchmarked by Queiroga et al. (2026). The paired differences with Wilcoxon signed-rank tests under a Holm correction, are then tested once for an advantage and once for a disadvantage against each solver within the performance range of DRISPI.

The secondary research question is concerned with what HAOS learns.

RQ2: Do the weight distributions HAOS learns differ systematically with instance characteristics, and do these differences give interpretable insight into instance structure?

To answer that the logged weight trajectories of the same 300 runs were analyzed. Resulting weights can however only reflect instance structure if they reflect the underlying operators' performance. After testing for that precondition first, the three runs of an instance would have to agree on the choices made by HAOS to proof a pattern. The campaign contains no control arm with a frozen configuration. The answer to RQ2 is therefore descriptive. It shows what HAOS learns, not how much the learning itself adds.

This thesis makes four contributions. First, it develops the Decompose-Route-Improve-Set-Partition-Improve (DRISPI) framework, an algorithm-agnostic framework that repeats a DRI loop. The framework stores every route it generates in a

persistent pool, which it periodically recombines that pool by running a Set Covering (SC) or Set Partitioning (SP) problem over it. Any CVRP solver that takes an instance and returns routes is a valid subsolver. The deployed configuration chooses among AILS-II, FILO, FILO2, and PyVRP (Wouda et al., 2024). Second, it introduces HAOS, a hierarchical roulette-wheel controller in the tradition of ALNS (Ropke and Pisinger, 2006). In every iteration, HAOS selects the number of subclusters, the degree to which it considers demand dissimilarity, the clustering paradigm, the clustering method, and the subsolver. It credits each configuration in two different ways. The first credit is immediate, for the solutions its own iteration produces. The second is deferred, for every route that SC/SP selects from the pool, traced back through a configuration tag stored on each route. Third, it proposes Boundary-Guided Adaptive Iterated Local Search (BG-AILS). This improvement stage perturbs the concatenated subproblem solutions with a focus on subcluster boundaries with a chain of cross-reconnect moves. It then hands the result to AILS-II, which it guides towards the now perturbed boundary regions of the intermediate solution. None of the repair strategies reviewed in Chapter 2.3.4 uses the partition to place a perturbation. Section 3.4 tests BG-AILS in isolation against a standard AILS-II pass and against the same perturbation without boundary guidance. For the fourth and final contribution, it runs these stages as a parallel pipeline on eight cores. BG-AILS and the SC/SP step run asynchronously next to the DRI loop. A fitted wall-time model sizes the BG-AILS budget to the stage it overlaps with.

The remainder of this thesis is structured as follows. Chapter 2 states the CVRP. It reviews the monolithic solvers that lead on the XL set and the strategies used to scale routing search. It then locates the research gap this thesis aims to address. Chapter 3 presents DRISPI stage by stage and closes with the parallel pipeline and its main loop. Chapter 4 describes the experimental design. It reports the comparison with the published solvers, analyzes the results by instance characteristic, the resulting HAOS weights and their implications, the convergence behavior, and the improvement contributions of each stage. Chapter 5 answers both research questions, discusses the limitations of the study, and outlines directions for future research. Detailed instance-level results to the benchmark campaign can be found in Appendix C.

2 Literature Review

The field of vehicle routing originated from the truck dispatching model of Dantzig and Ramser (1959). Subsequent research has treated the CVRP as its canonical member, with a homogeneous fleet, capacity constraints, and one visit per customer (Toth and Vigo, 2014). Vidal et al. (2020) and Archetti et al. (2026) review how that core model has been extended since.

This chapter states the CVRP problem and the benchmark instances used in the computational study (Section 2.1). It then moves from the state-of-the-art solvers that (Section 2.2) to the techniques that scale and configure routing heuristics (Section 2.3). Section 2.4 covers the frameworks that wrap existing solvers, and Section 2.5 outlines the remaining research gap.

2.1 Problem Statement and Benchmark Instances

The CVRP is defined on a complete undirected graph $G = (V, E)$, where $V = \{0\} \cup C$ is the set of locations. Node 0 represents the depot, and $C = \{1, \dots, n\}$ is the set of customers, so that $n = |C|$ is the customer count excluding the depot. Each edge $\{i, j\} \in E$ carries a non-negative travel cost $c_{ij} \geq 0$, and each customer $i \in C$ has a demand $q_i > 0$. A homogeneous fleet of vehicles with capacity Q is dispatched from and returns to the depot. A solution is a set of routes that start and end at the depot, visit every customer exactly once, and respect the capacity Q on every route. The objective is to minimize the total travel cost.

As a generalization of the Traveling Salesman Problem (TSP), the CVRP is NP-hard (Lenstra and Rinnooy Kan, 1981), and the number of feasible solutions grows factorially with n . This makes solving larger CVRP instances with exact methods impractical. Nevertheless, branch-cut-and-price algorithms prove optimality for instances with a few hundred customers (Pecin et al., 2017; Costa et al., 2019; Pessoa et al., 2020). But that still leaves them an order of magnitude short of the scale considered here. Heuristic and metaheuristic methods therefore trade the optimality guarantee for feasible solutions within a practical wall-clock time budget.

Existing heuristic research on the CVRP has focused on a small number of benchmark sets. The X set of Uchoa et al. (2017) contains 100 instances with 100 to 1,000 customers, and is generated by varying four characteristic attributes across the set: depot position, customer distribution, demand distribution, and average route length. Queiroga et al. (2026) report that it became the primary benchmark for both

exact and heuristic methods. It also formed the basis of the CVRP track of the 12th DIMACS Implementation Challenge.¹ Queiroga et al. (2022) also added the XML set of 10,000 instances with 100 customers each, providing optimal solutions for almost all of them. This allowed benchmarks of machine-learning heuristics to be evaluated against absolute error rather than a moving reference, as is the case for other instance sets.

Two additional collections reach far beyond 1,000 customers. The AGS set of Arnold et al. (2019) contains ten instances with 3,000 to 30,000 customers, which were derived from parcel delivery in Belgium. The AV set of Accorsi and Vigo (2024) contains 20 instances with 20,000 to 1,000,000 customers, and were generated from sampled addresses in Italian regions. Queiroga et al. (2026) observe that all 30 of these instances draw their demands uniformly from one to three units, and that most of them have long routes. The existing sets also left a gap in the range from 1,001 to 2,999 customers.

The XL set of Queiroga et al. (2026) closes that range with 100 instances of 1,000 to 10,000 customers. They are generated very similarly to the X set, varying the same four attributes. Queiroga et al. (2026) released the set with initial BKS values from a selection of state-of-the-art solvers for large-scale instances. With their release, they opened the CVRPLib BKS Challenge,² a community competition that started on January 12, 2026, with the goal to find new BKS values for the XL set without any further constraints. Chapter 4 uses this XL set and lists its attribute levels. Figures A.1 and A.2 in the appendix visualize two of these instances, unsolved and solved, for illustrative purposes.

2.2 Monolithic Solvers for Large-Scale CVRP

On the XL set, the methods that produced the initial BKS values are metaheuristics that search over the complete solution space. This thesis uses the term *monolithic* for a method that maintains one complete feasible solution, or a population of them, under a single search process that can in principle modify any route. Neighborhoods may be localized or sparsified without affecting this classification. What falls outside it are independent customer partitions, solved separately and merged by an outer recombination layer.

Queiroga et al. (2026) ran AILS-II, FILO, FILO2, KGLS-XXL, HGS-CVRP, SISR, LKH-3, and OR-Tools on every XL instance, with 60 seeds of two hours per code on a single thread. Of the 100 initial BKS values, AILS-II found 93, FILO2 six, and FILO one. The authors report HGS-CVRP, SISR, LKH-3, and OR-Tools as an indication only, because their code implementations were not calibrated for instances

¹<http://wwwdimacs.dimacs.rutgers.edu/programs/challenge/vrp/>, accessed September 1, 2026.

²https://galgos.inf.puc-rio.br/cvrplib/index.php/en/bks_challenge/overview, accessed September 1, 2026.

with up to 10,000 customers. The two-hour time limit is the budget previously, that was also used on X instances of at most 1,000 customers.

A key concept that several of the above-mentioned solvers employ is Iterated Local Search (ILS). ILS alternates a perturbation of the current solution with a local search that returns it to a local optimum. An acceptance criterion then decides which local optimum the next perturbation starts from (Lourenço et al., 2003). Lourenço et al. (2003) argue that the perturbation strength governs the behavior of the method. A perturbation that is too weak lets the local search fall back into the same local optimum, while one that is too strong turns the method into a random restart. The classical example calibrated to that tension is the double-bridge move of Martin et al. (1991), which a simple local search built from small moves cannot undo in one step.

AILS-II, developed by Máximo et al. (2024), utilizes an adaptive ILS for large-scale CVRP instances. It extends a predecessor, AILS-PR (Máximo and Nascimento, 2021), which hybridizes adaptive ILS with path relinking and performs well up to 1,000 customers. AILS-II adapts the perturbation degree during the search, and later during convergence, it draws the solution to be perturbed from an elite set. Its original experiments reach 30,000 customers on the AGS set, where it reported lower average gaps than both AILS-PR and HGS on all ten instances.

Accorsi and Vigo (2021) build FILO as an ILS with a simulated-annealing acceptance criterion (Kirkpatrick et al., 1983). Granular neighborhoods (Toth and Vigo, 2003) restrict move evaluation to a subset of promising arcs, and selective vertex caching restricts it further to recently changed vertices. FILO2 (Accorsi and Vigo, 2024) carries this design to instances with up to one million customers, with pointed changes to improve scalability. But solvers still localize the search inside one full solution, rather than partitioning the customer set into independently solved sub-problems.

Hybrid genetic search evolves a population of complete solutions and improves offspring by local search (Vidal et al., 2012). Vidal (2022) published HGS-CVRP as an open implementation for the CVRP with the SWAP* neighborhood, and Wouda et al. (2024) provide PyVRP, a separate implementation of the same paradigm. The comparison of Queiroga et al. (2026) uses the HGS-CVRP code.

Those same experiments show that the leading method changes with scale. On the XML instances and on the smaller half of the X set, up to about 330 customers, HGS-CVRP has the best average solution quality of the eight codes, while AILS-II leads on the larger half of the X and dominates on XL. Queiroga et al. (2026) interpreted this as a transition from population-based search, which evolves several complete solutions at once, to single-trajectory search.

The remaining state-of-the-art solvers follow different designs. The knowledge-guided local search of Arnold and Sörensen (2019) applies CROSS-exchange, relocation chains, and Lin-Kernighan moves to a complete solution. When a local op-

timum is reached, it penalizes long and wide edges so that later moves leave those edges out. Arnold et al. (2019) scale that design to instances with up to 30,000 customers as KGLS-XXL by pruning neighborhoods and storing only nearest-neighbor distances. SISR (Christiaens and Vanden Berghe, 2020) ruins a solution by removing geographically adjacent strings of customers. That removal concentrates unused capacity on a few nearby routes. It then reinserts the customers by greedy insertion with blinks, skipping some candidate positions at random. A simulated-annealing criterion accepts or rejects the rebuilt solution. LKH-3 transforms constrained routing problems into a TSP with penalties, then applies the Lin-Kernighan-Helsgaun local search to the result (Helsgaun, 2017). An earlier variable neighborhood search had already addressed instances with up to 20,000 customers (Kytöjoki et al., 2007). POPMUSIC (Queiroga et al., 2021) is the exception among the competitive large-scale methods. It reoptimizes subproblems extracted from a complete solution, and is therefore not a monolithic solver. Section 2.3.3 discusses it with the other decomposition methods.

2.3 Scaling and Configuring Routing Heuristics

Exact methods fail at scale because the solution space grows factorially, but local-search heuristics face a different cost. What grows with n is the effort of neighborhood evaluation, the memory for move descriptors and caches, and the number of iterations needed to reach a given quality (Arnold et al., 2019; Accorsi and Vigo, 2021).

Literature offers several ideas to tackle the scalability problem. Parallel search spreads the effort over processors (Section 2.3.1), and route pools recombine routes found earlier by set covering or set partitioning (Section 2.3.2). Decomposition splits the customer set into smaller subproblems (Section 2.3.3), which leaves seams that later steps must repair (Section 2.3.4). Adaptive control and learned policies then decide which operator, configuration, or partition to apply (Sections 2.3.5 and 2.3.6).

2.3.1 Parallel Search

Crainic (2008) and Crainic (2019) classify parallel metaheuristics by how they split the search. Low-level parallelism accelerates neighborhood evaluation inside one trajectory. Multi-search strategies run several trajectories, either independently or as cooperating searches that exchange knowledge such as complete solutions. Crainic (2019) concentrates on cooperative strategies, and describes those built on asynchronous exchanges as the most effective of them.

Applied to vehicle routing, these designs multiply processor effort on the full instance. Groër et al. (2011) combine heuristic local search with integer programming

across as many as 129 processors. Jin et al. (2014) run cooperative tabu-search threads that share a common solution pool, where entering solutions are clustered by similarity so that the clusters can balance between intensification and diversification more effectively.

In both methods, every worker still searches the full customer set. Parallelism changes how many neighborhoods or incumbents are examined per wall-clock second, without reducing the size of the problem each worker faces. Taillard (1993) instead partitions the customers geographically and solves the resulting subproblems with parallel iterative search. But solving customer subsets concurrently requires a dedicated decomposition layer, discussed later in Section 2.3.3.

2.3.2 Route Pools and Set Partitioning

Using set covering or set partitioning over routes has been an approach that is older than local search metaheuristics. Balinski and Quandt (1964) formulate delivery as an integer program over routes, and Garfinkel and Nemhauser (1969) treat set partitioning as set covering with equality constraints. Foster and Ryan (1976) solve the set partitioning model of vehicle scheduling with the revised simplex method, restricting the columns to routes through radially contiguous locations. Their test problems range from 21 to 100 locations. Once a collection of feasible routes exists, selecting a subset that covers each customer once is an integer program whose size tracks the number of routes rather than n .

In more modern approaches, Rochat and Taillard (1995) store elite routes in an adaptive memory, draw new starting solutions from it probabilistically, and recombine the stored routes by set covering. The local search generating the routes and the model recombining them are already separate components here. Taillard et al. (2001) describe this scheme as adaptive memory programming and argue that several metaheuristics share it. The memory of solution components builds new starting solutions, a local search improves them, and the results update the memory. Kelly and Xu (1999) combine route generation by heuristics with a set partitioning model that selects among the generated routes.

Jourdan et al. (2009) classify such combinations of exact methods and metaheuristics in a taxonomy of hybrid approaches. Several CVRP heuristics apply the pattern repeatedly during the search rather than once at the end. Subramanian et al. (2013) hybridize ILS with a set partitioning model over the routes it generates. Cavaliere et al. (2022) alternate LKH-3 local search with a restricted set partitioning program. It was allowed to run 24 hours past its last improvement, and reported new best known solutions for large-scale CVRP instances. FSP4D, a finalist of the 12th DIMACS Implementation Challenge, follows the same alternation with a FILO-based fast ILS feeding a route pool and then running a set partitioning heuristic to identify high-quality combinations (Accorsi et al., 2022a).

Arenas-Vasco et al. (2025) quantify the benefit of this route and set partitioning pattern. Their analysis covers 30 set partitioning or set covering matheuristics for vehicle routing problems, selected from 54 eligible studies. The recombination step improved solution quality by 0.51% on average, with a 95% confidence interval of 0.41 to 0.61%. The authors describe the gain as modest but consistent across implementations.

A central problem of recombination is that set covering or set partitioning can only choose among routes that some other component has already generated. But this also means a pool dominated by the routes of one incumbent reproduces that incumbent, effectively rendering the set partitioning pointless. The diversity of the result is bounded by the diversity of whatever feeds the pool new routes. The recombination step itself decides nothing about how the instance is split, which solver generates the routes, or how either choice changes during a run.

2.3.3 Decomposition Strategies

A decomposition method splits the instance into smaller subproblems, solves each one, and then reassembles a complete solution. How good this divide-and-conquer solution is depends on where the cuts fall, how often they are redrawn, and how the pieces are joined back together. Two central questions in decomposition literature are whether the customers are grouped before or after a tour is built, and whether the groups are formed from customer coordinates or from the routes of an incumbent solution.

The Cluster-first route-second approach answers the first question by partitioning the customers and then routing each cluster. There are various clustering methods following that approach. The sweep algorithm of Gillett and Miller (1974) forms clusters by polar angle around the depot and solves a TSP on each. Fisher and Jaikumar (1981) assign customers to vehicle seeds through a generalized assignment problem before routing each assignment. Kerschler and Minner (2025) and Karaahmetoglu and Kim (2026) follow the same order with modern components, building a customer partition and routing its clusters with existing solvers.

Route-first cluster-second reverses that order. Beasley (1983) constructs a single tour through all customers, then partitions it into capacity-feasible routes, optimally for the given order. Prins (2004) embeds this split procedure in a genetic algorithm that encodes solutions as giant tours. The clustering step then operates on a customer sequence rather than on an unstructured point set.

The second question only arises once a partition budget is fixed, and it separates vertex-based from route-based clustering. Vertex-based clustering partitions the customer set directly, which is what the clustering step of cluster-first route-second does. Route-based clustering instead requires a complete incumbent, grouping cus-

tomers that already share a route or clustering the routes themselves by their barycenters.

Bent and Van Hentenryck (2010) study adaptive schemes that repeatedly decompose a Vehicle Routing Problem with Time Windows (VRPTW) instance from the current solution. In their reading, vehicle-based decompositions, which partition the vehicles of the incumbent across subproblems, cannot express temporal splits and may produce spatial splits that are not focused enough. They propose customer-based decompositions instead. On class R2 of the extended Solomon benchmarks, the customer-based variants improved on plain large neighborhood search where the vehicle-based ones did not.

Santini et al. (2023) compare decomposition strategies inside ALNS (Pisinger and Ropke, 2007) and hybrid genetic search (Vidal et al., 2012) on the X instances. Route-based decompositions, which group customers that already share a route, outperformed path-based coarsening, which merges customers into supernodes. Barycenter clustering was the strongest strategy of those tested, improving both host algorithms relative to runs without decomposition. The benefit depended on the strategy and on the instance, and a poor decomposition failed to help at all. Subproblem size and decomposition frequency both required calibration. Their ALNS learns destroy and repair weights within the run, but the decomposition method itself stays a fixed, calibrated choice.

The two studies favor different paradigms. Bent and Van Hentenryck (2010) find customer-based decompositions stronger on a class of VRPTW instances, while Santini et al. (2023) obtain their best results with a route-based strategy on the CVRP. Because the studies differ in problem, host heuristic, and benchmark, the comparison does not isolate the cause.

Another decomposition approach, POPMUSIC (Taillard and Voß, 2002), starts from a complete solution and optimizes sub-parts of it until a local optimum is reached. Queiroga et al. (2021) apply the idea to the CVRP, using a branch-cut-and-price algorithm as a heuristic on subproblems of typically 25 to 200 customers. The main experiments cover instances of 302 to 1,000 customers. Long runs started from strong incumbents or CVRPLib BKS values improved several best known solutions, including some on very large instances. The method needs a complete solution before it can cut, and it uses a fixed subproblem policy. Its exact subsolver is also not an interchangeable off-the-shelf heuristic in the sense of Section 2.4.

2.3.4 Repairing Decomposition Boundaries

Each subproblem of a decomposition can be solved well in isolation while the assembled solution still carries defects along the cuts. A partition drawn from coordinates or route centroids uses no information about which customers a vehicle will end up serving together. Routes on either side of a cut may therefore be cheaper

when reconnected across it. Decomposition and recombination therefore introduce a structural suboptimality of their own. The literature suggests four approaches on how to deal with the seams in between two clusters.

The first is to redraw the cut. Iterative schemes decompose again from the current solution at regular intervals (Bent and Van Hentenryck, 2010; Santini et al., 2023), so that a seam of one iteration can fall inside a subproblem of a later one. Whether any given seam is repaired then depends on where the next partition happens to cut.

The second is to take the subproblems from the incumbent in the first place. POP-MUSIC follows this pattern (Taillard and Voß, 2002; Queiroga et al., 2021). Because every subproblem is carved out of the current solution, the method does not introduce the structural suboptimality in the first place, which would need to be repaired.

The third is an explicit improvement step after the subproblem solutions are concatenated. Kerschler and Minner (2025) apply local search to inter-route moves between subproblems, and prune it with relational data from the clustering step. A move is evaluated only when the two subproblems lie within a vicinity of each other in a dissimilarity matrix already used for the decomposition. Where fuzzy clustering was used, a customer additionally qualifies only when its highest membership degree falls below a certain threshold, which marks it as boundary-adjacent. The pruning therefore concentrates the improvement budget on the customers the decomposition placed at a seam.

The fourth is recombination through a set covering step (Kerschler et al., 2026). They select routes from a pool holding the output of many decompositions. A route that one partition cut badly can then be replaced by a route from another, more favorable partition.

In all four, the partition information decides where the next subproblem falls or which moves the local search evaluates. We found no method in the reviewed literature that uses the partition boundaries to place a perturbation before a local search starts.

2.3.5 Adaptive Operator Selection and Algorithm Configuration

Every heuristic in this review exposes a number of parameters, and many offer alternative operators. Eiben et al. (1999) distinguish parameter tuning, which fixes values before a run, from parameter control, which changes them during one. Offline configurators like the irace package of López-Ibáñez et al. (2016) automate the tuning side by racing candidate configurations on a set of training instances and eliminating those that a statistical test finds worse. The result is one configuration for the instance class that the training set represents.

Automated algorithm selection asks the same question at the level of whole algorithms. Kerschke et al. (2019) observe that for practically every intensely studied

computational problem, different instances are best solved by different algorithms. Per-instance selectors exploit this complementarity by predicting from instance features which algorithm of a portfolio to run. The scale transition that Queiroga et al. (2026) observe on the CVRP is one instance of such complementarity. But both tuning and per-instance selection are trained before the test instance is solved.

Online operator selection adapts during the run instead. The ALNS of Ropke and Pisinger (2006) updates roulette-wheel weights of destroy and repair heuristics from scores collected during the search, and Pisinger and Ropke (2007) apply the same adaptive layer across five vehicle routing variants. Windras Mara et al. (2022) survey 252 publications on ALNS from 2006 to 2021, and Voigt (2025) reviews and ranks the operators used in ALNS for vehicle routing problems.

The selection problem can be read as a multi-armed bandit. The search observes the reward of the operator it applied, but not what the other operators would have produced (Auer et al., 2002). Fialho et al. (2010) analyze bandit-based adaptive operator selection mechanisms in this setting. Surveys of hyper-heuristics (Burke et al., 2013) and of machine learning in metaheuristics (Karimi-Mamaghan et al., 2022) place the work in a broader literature on choosing heuristics during a search.

The evidence on what adaptive selection buys is mixed. Turkeš et al. (2021) run a meta-analysis of 25 ALNS implementations against their non-adaptive variants. On average, the adaptive layer improved the objective function value by 0.14%, with a 95% confidence interval of 0.06 to 0.21%. Its value depends on the set of heuristics it selects from, and because the layer adds complexity, the authors recommend it only in specific situations.

Reinforcement learning replaces the roulette update with a trained policy. Kallestad et al. (2023) train a deep reinforcement learning agent that selects the next low-level heuristic from features of the search state. It outperformed ALNS and uniform random selection on four combinatorial optimization problems. But unlike ALNS, it was not hurt by a larger set of heuristics. Reijnen et al. (2024) use deep reinforcement learning for the online control and configuration of ALNS. Both policies are trained on sets of instances before they are applied.

What this literature adapts are operators inside one search process. In the work reviewed here, neither the decomposition of the instance nor the choice of subsolver is among them. Online control also moves part of the design problem into the update rule. The adaptive layer of ALNS carries its own parameters, among them a reaction factor, a segment length, and the score values (Ropke and Pisinger, 2006).

2.3.6 Learned Decomposition

Machine learning for combinatorial optimization has grown into a research line of its own (Bengio et al., 2021). For routing, Kool et al. (2019) train attention-based construction policies with reinforcement learning on instances of up to 100

customers. Queiroga et al. (2026) observe that machine-learning methods for the CVRP were rarely tested on standard benchmark sets, and often relied on ad hoc instance generators instead.

Learning-based work on large instances has turned to decomposition. Li et al. (2021) train a Transformer to choose which spatially local subproblem to delegate to a black-box subsolver, applying the policy iteratively on instances of roughly 500 to 3,000 customers. Hou et al. (2023) train a two-stage divide method that partitions a CVRP instance into traveling salesman subproblems, and report real-time solutions beyond 5,000 nodes. Ye et al. (2024) combine a learned global partition policy with local construction in GLOP.

These policies are trained before the test instance is solved. They can be applied iteratively at test time, but the mapping from instance features to a partition or a delegation is not updated from the costs observed on that instance.

Karahmetoglu and Kim (2026) use a pretrained large language model in place of a trained policy. The model selects clustering, balancing, and refinement actions over a hierarchical cluster tree from partition diagnostics alone, without feedback from solution costs. The method builds one partition per run, then routes the clusters with the guided local search of OR-Tools. But it addresses the Heterogeneous Vehicle Routing Problem (HVRP), and the preprint reports no gaps on either the X or the XL set for CVRP.

2.4 Algorithmic Frameworks

Sörensen (2015) distinguishes a metaheuristic as a problem-independent framework from a problem-specific implementation of its guidelines, and Sörensen et al. (2018) keep that vocabulary. This thesis uses three terms. A framework is a reusable control structure that can wrap existing solvers. A solver is a concrete implementation that searches a routing instance, such as AILS-II, FILO, FILO2, or PyVRP. A heuristic is any procedure that returns a feasible solution without an optimality guarantee. Every solver named above is a heuristic, though not every heuristic is a framework.

A framework for the VRPTW is proposed by Kerscher and Minner (2025). Their DRI framework is built on three phases. The decomposition phase clusters customers under a spatial-temporal-demand dissimilarity, which scales the pairwise spatial distance by the scheduling flexibility and the capacity consumption of the customer pair. The partition therefore reflects the objective and the constraints rather than geography alone. The routing phase solves each resulting sub-VRPTW independently with an existing solver, PyVRP in their experiments. The improvement phase then applies the pruned local search of Section 2.3.4 to the concatenated result.

Two properties of the DRI matter for the comparison drawn in Section 2.5. First, the number of subproblems is not a handcrafted constant. Kerscher and Minner (2025) evaluate the silhouette coefficient of the clustering over a range of candidate counts and take the maximizing value, so that the granularity of the partition adapts to each instance. They report that this matches a perfect-information baseline closely on clustered instances, at a fraction of its runtime. Second, that criterion measures cluster compactness and separation rather than routing cost, and it is evaluated once before the routing phase begins. The clustering algorithm, the dissimilarity metric, the vicinity size, and the fuzzy threshold are all compared offline in a sensitivity analysis and then held fixed. Their computational study covers the Gehring and Homberger instances together with a new set of 20 very large-scale VRPTW instances of 3,000 to 30,000 customers. On that new set, the DRI reports lower costs than PyVRP on every instance where PyVRP found a usable solution. PyVRP failed to produce one on most instances with tight time windows.

Kerscher et al. (2026) extend the pattern to the HVRP in DRSCI, whose four steps give it its name. Each iteration decomposes the instance with k -means clustering, either over customer vertices or over the routes of the current best solution. Each cluster becomes a sub-instance by duplicating the depot and assigning a sub-fleet to it. The routing step then generates routes for each sub-instance through one of four strategies. These solve it as several independent homogeneous VRPs, as connected prize-collecting VRPs, through a giant-tour partitioning procedure, or directly with an HVRP solver. All generated routes enter a route pool, and a set covering model selects a subset from it. A savings-based heuristic then removes the customers that appear in more than one selected route, before local search refines the result.

How DRSCI chooses its configuration is the part closest to this thesis, so it is worth stating precisely. The clustering paradigm is a coin flip in every iteration, with vertex-based and route-based clustering equally likely. The number of clusters and the routing strategy follow a scheme that their hyperparameter tuning selected over a fully random alternative. Under that scheme, every routing strategy is applied exactly once at the current cluster count before the count advances. The candidate counts are drawn from powers of two bounded by the route count of a feasible solution. Once a solution exists, the candidate range narrows to the interval spanned by the smallest and the largest cluster count that contributed a route to the current best solution. DRSCI therefore does use within-run feedback on one dimension of its configuration, by restricting a range rather than by weighting options. Their study covers heterogeneous-fleet instances of 101 to 1,001 vertices. DRSCI improved the previous best known solution on 14 of the 60 FSMVRP instances in the XH dataset, and on 561 of the 600 FSMVRPTW instances. The published experiments run single-threaded, though the authors note that route generation is parallelizable by design.

An unpublished project study adapted DRSCI to the CVRP and reported a mean

gap of 0.53% to the initial XL BKS values (Altendeitering and Bachmann, 2026). That study drew its decomposition and routing configuration uniformly at random in every iteration.

2.5 Research Gap

No published decomposition framework maintains weights over its decomposition and subsolver choices and updates them from the solution costs observed on the instance it is solving. The reviewed methods settle that configuration in one of five ways:

- By a fixed rule or a calibrated default (Taillard, 1993; Queiroga et al., 2021; Santini et al., 2023),
- By a clustering-quality criterion evaluated once per instance, before routing begins (Kerschler and Minner, 2025),
- By a schedule that cycles the routing strategies and narrows the cluster-count range to the interval spanned by the counts that contributed routes to the incumbent (Kerschler et al., 2026),
- By a policy trained before the test instance is solved (Li et al., 2021; Hou et al., 2023; Ye et al., 2024), or
- By a pretrained model that reads partition diagnostics without cost feedback (Karaahmetoglu and Kim, 2026).

DRSCI (Kerschler et al., 2026) is the closest of these to this thesis. Its narrowing rule does react to the instance at hand, but it acts on the cluster count alone. It selects uniformly inside the narrowed range, and it credits neither the clustering paradigm nor the routing strategy with the quality of what they produced. The DRI (Kerschler and Minner, 2025) adapts its granularity per instance as well, from a criterion that never observes a routing cost.

The evidence on whether closing that distance pays off points in two directions. Turkeš et al. (2021) find a small average benefit from the adaptive layer in ALNS, whereas trained controllers outperformed roulette-wheel selection on the problems where they were tested (Kallestad et al., 2023). Neither body of work adapts a decomposition or a subsolver, so whether online control helps at that level has not been tested.

Algorithm-agnostic solver plug-in, parallel subproblem solving, route-pool recombination by set covering or set partitioning, and evaluation on the XL set all fall within the scope of this thesis. Taken separately, none of them is the gap. Table 2.1 records which published methods combine which of them.

Table 2.1 can be read in two directions. Down the four capability columns, the entries fill in over time. Decomposition, route-pool recombination, and an algorithm-

agnostic solver interface grew as separate lines of work, and Kerscher et al. (2026) is the first published row to combine all three. Down the Configuration column, no such progression appears. Every row above the last fixes its configuration before the instance is seen, or derives it from a rule that never observes a routing cost. The gap is therefore not a capability that the literature lacks. It is a control layer over capabilities that already coexist.

Table 2.1: Research Gap Matrix for Large-Scale Routing Methods.

Method	Problem	P	D	SC	Ag	Configuration	Largest n
Máximo et al. (2024)	CVRP	no	no ^a	no	–	fixed	30,000
Accorsi and Vigo (2024)	CVRP	no	no ^a	no	–	fixed	1,000,000
Rochat and Taillard (1995)	VRP	no	no	yes	–	fixed	385
Cavaliere et al. (2022)	CVRP	no	no	yes	–	fixed	30,000
Queiroga et al. (2021)	CVRP	~ ^b	yes	no	~	fixed	1,000
Santini et al. (2023)	CVRP	~	yes	no	–	calibrated	1,000
Li et al. (2021)	CVRP	~	yes	no	yes	trained offline	3,000
Karaahmetoglu and Kim (2026)	HVRP	yes	yes	no	~	pretrained	500,000
Kerscher and Minner (2025)	VRPTW	no ^c	yes	no	yes	silhouette per instance	30,000
Kerscher et al. (2026)	HVRP	no ^c	yes	yes	yes	scheduled, range narrowed	1,001
Altendeitering and Bachmann (2026)	CVRP	~	yes	yes	yes	random	10,000
This thesis	CVRP	yes	yes	yes	yes	learned online	10,000

P = parallel subproblem solving, D = customer-set decomposition, SC = route-pool set covering or set partitioning, and Ag = algorithm-agnostic solver plug-in. ~ marks a partial capability. Largest n is the largest customer count in the original computational study.

^a Localized neighborhoods through granular arcs and selective vertex caching, not customer-set decomposition (Toth and Vigo, 2003; Accorsi and Vigo, 2021).

^b Subproblems are reoptimized within an intensification loop rather than solved as a standing parallel layer.

^c The published experiments run on a single thread. Both papers note that the subproblems are independent and that route generation is parallelizable by design.

Chapter 3 introduces DRISPI against that reading. It keeps the decompose-route-improve DRI loop and the persistent route pool of the frameworks in Section 2.4. What it replaces is their configuration layer. A controller draws the decomposition and the subsolver from weighted alternatives in every iteration, and updates those weights from a reward scored on each iteration’s outcome. Two further elements answer points raised earlier in this chapter. The improvement stage places its perturbation on the subcluster boundaries, which Section 2.3.4 found missing from the reviewed repair strategies. Finally DRISPI is a highly parallelized framework, which both frameworks in Section 2.4 note as possible extension but left unimplemented.

3 Framework

This chapter presents DRISPI, the framework this thesis develops for the CVRP at very-large scale (Section 2.1). DRISPI repeatedly executes a decompose-route-improve (DRI) loop. Each iteration first partitions the customer set into smaller subclusters and solves one routing subproblem per subcluster with an existing monolithic solver, then concatenates the resulting routes and revises them near the subcluster boundaries with a dedicated improvement stage, boundary-guided adaptive iterated local search (BG-AILS). Hierarchical adaptive operator selection (HAOS) draws the decomposition and routing configuration for every iteration from a set of weighted alternatives and updates those weights from the outcomes it observes, rather than fixing one configuration for the whole run. Every route the DRI loop produces enters a persistent route pool; at scheduled intervals, DRISPI steps back from the current incumbent and recombines the pool through a set covering or set partitioning (SC/SP) model, followed by one further post-SC/SP improvement pass. The loop continues until a wall-clock time limit or a run of iterations without improvement is reached.

Figure 3.1 shows the resulting control flow for one instance. HAOS (Section 3.1) draws one configuration per iteration; decomposition (Section 3.2) partitions the customer set into k subclusters under that configuration; routing (Section 3.3) solves each subcluster independently and concatenates the routes; BG-AILS (Section 3.4) revises the routes near the subcluster boundaries. Both the routing stage and BG-AILS add their routes to the route pool, and either can become the new incumbent if it improves on the current one. At scheduled intervals, the same iteration also runs set covering or set partitioning (Section 3.5) over the pool accumulated so far, followed by one further post-recombination improvement pass (Section 3.6) whose output feeds the pool again and can likewise become the new incumbent; on every other iteration, this pair of stages is skipped. The loop then either returns to a fresh HAOS draw or stops. Figure 3.1 shows this control flow as one sequential loop; Section 3.7 shows how DRISPI executes it with several stages running in parallel across iterations instead.

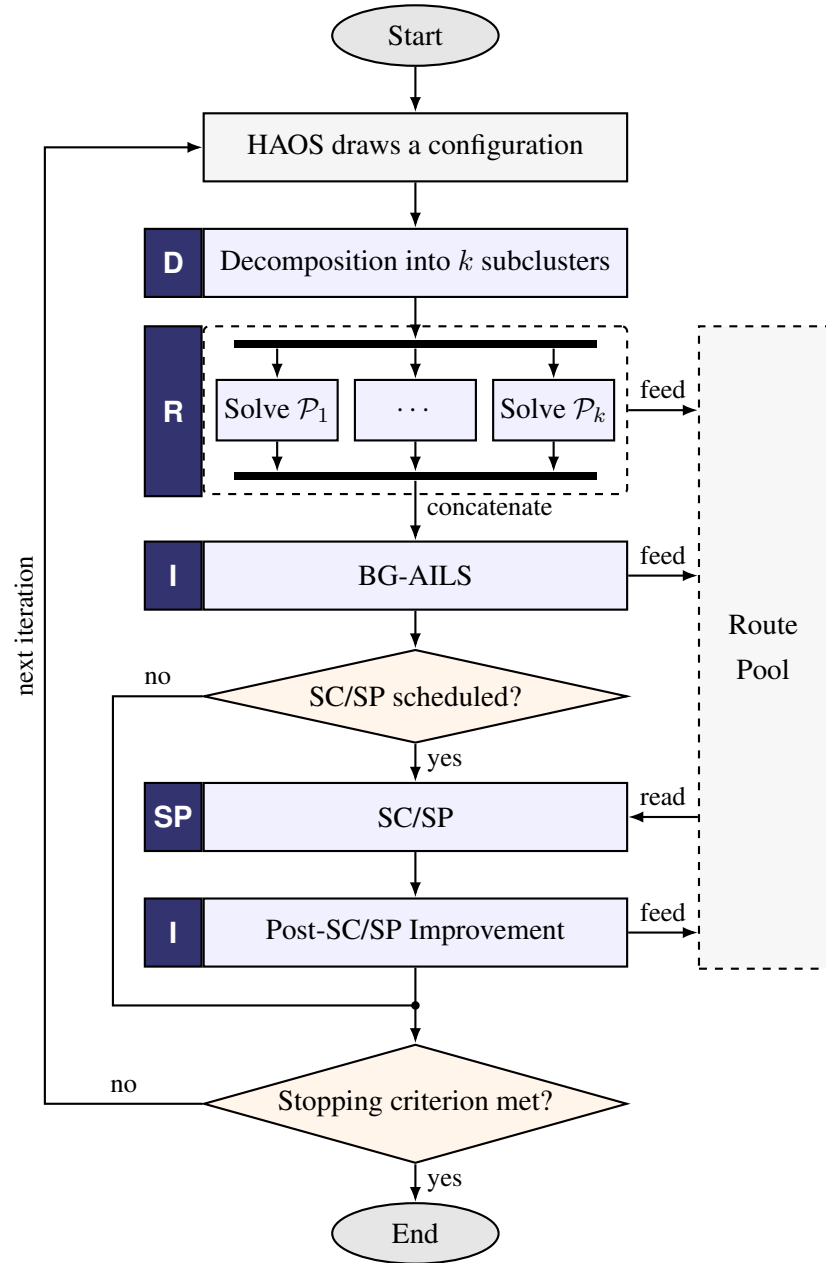


Figure 3.1: Sequential Flowchart of DRISPI.

The remaining sections address one stage each, in the order Figure 3.1 shows: Section 3.1 on HAOS, Section 3.2 on decomposition, Section 3.3 on routing, Section 3.4 on BG-AILS, Section 3.5 on the route pool and SC/SP, and Section 3.6 on the post-SC/SP improvement pass. Section 3.7 closes the chapter by assembling these stages into the parallelized pipeline DRISPI actually runs, together with the pseudocode for its control loop.

3.1 Hierarchical Adaptive Operator Selection

Ideally, a heuristic would always be tuned to no more than one specific problem instance. That would promise both, an effective and efficient approach configured to exploit that instance's size, spatial and demand distributions, and fleet capac-

ity. Recent benchmark sets, however, have deliberately combined a wide range of these characteristics to test whether performance transfers across instance structures (Uchoa et al., 2017; Queiroga et al., 2026). That is why finding and selecting a tailored configuration for every instance requires a costly tuning campaign. That is why most benchmarks are compromising on one fixed configuration for the entire problem set. This framework aims to forego this compromise through Hierarchical Adaptive Operator Selection (HAOS), an approach to adapt DRISPI’s configuration while solving the problem instead.

The principal methodological inspiration is adaptive large neighborhood search. Competing destroy and repair operators receive weights based on observed performance and are selected by roulette wheel (Ropke and Pisinger, 2006). The same adaptive layer has been applied across five vehicle routing variants (Pisinger and Ropke, 2007). HAOS adopts this principle but moves the adaptive choice outside a single local search. It instead controls the decomposition granularity, the representation and method used for clustering, the demand contribution cluster formation, as well as the subsolver. The adaptation is online because rewards update the weights while the current instance is being solved. The weights of each operator summarize observed performance on that instance. The Convergence to a globally optimal configuration is neither required nor guaranteed.

The operator choices can also be interpreted as arms of a multi-armed bandit. After an arm is drawn, the search observes a reward but does not know what the unselected arms would have produced. This analogy connects HAOS to the exploration–exploitation problem (Auer et al., 2002) and to bandit-based adaptive operator selection (Fialho et al., 2010). However, HAOS does not implement a confidence-bound policy, an explicit exploration bonus, or a separate exploratory action. Its roulette probabilities follow the current weights directly. Stochastic sampling leaves every positive-weight option selectable, while the probability-floor transformation described below raises normalized shares that become sufficiently small.

At the beginning of every DRI iteration HAOS draws one configuration across five levels. Level one selects the number of subclusters k from a scale-adaptive domain. Its base values are clipped to the instance’s minimum feasible fleet size and extended geometrically for larger instances. The number of arms is capped at ten to allow for convergence to occur within a reasonable wallclock time. Level two selects the demand weight λ_q from six evenly spaced values between zero and one; this value controls how strongly differences in customer demand influence the decomposition. Level three selects the clustering paradigm π : whether customers are clustered directly (the vertex-based paradigm) or through the routes of the current incumbent solution (the route-based paradigm). Level four selects a clustering method m from the wheel associated with π , with five vertex-based and three route-based methods in the deployed configuration. This conditional fourth level makes the selection hierarchical since the selection of m depends on π . Level five selects

Algorithm 1 Hierarchical Adaptive Operator Selection (HAOS)

input: wheels $W_1, W_2, W_3, W_4^v, W_4^r, W_5$; floors $p_{\min}^{(\ell)}$; decay γ ; base weight w_0 ; warm-up $i_0 = 10$

- 1: **function** DRAW(S^*)
- 2: draw k, λ_q, π, s from W_1, W_2, W_3, W_5 with probabilities (3.3)
- 3: **if** $\pi = \text{route}$ **and** S^* is undefined **then** $\pi \leftarrow \text{vertex}$
- 4: draw method m from W_4^π
- 5: $\theta \leftarrow (k, \lambda_q, \pi, m, s)$ $\triangleright$ credited
- 6: **if** $\pi = \text{route}$ **then** $k \leftarrow \min(k, |S^*|)$
- 7: **return** credited θ and applied $(k, \lambda_q, \pi, m, s)$
- 8: **end function**
- 9: **function** CREDIT(θ, r)
- 10: **if** $i \geq i_0$ **then** $w_a \leftarrow \max(w_a + r, w_0)$ for every arm $a \in \theta$ by (3.1)
- 11: **end function**
- 12: **function** DECAY()
- 13: **if** $i \geq i_0$ **then** $w_a \leftarrow \max(\gamma w_a, w_0)$ for every arm of every wheel by (3.2)
- 14: **end function**

one of four routing solvers s and applies it to every subcluster generated during the iteration. The first four levels therefore configure decomposition, and the fifth configures subproblem routing. Write $\theta = (k, \lambda_q, \pi, m, s)$ for this five-level configuration.

Two feasibility adjustments apply before the configuration is used; Algorithm 1 gives Draw, Credit, and Decay. Route-based clustering requires an incumbent solution. Without one, HAOS uses the vertex-based paradigm and redraws the method from the vertex wheel; these applied choices receive the subsequent credit. If k exceeds the number of incumbent routes under route-based clustering, the applied k is clipped to the route count. The immediate update retains the index of the k arm originally drawn, while route tags record the configuration that was applied. Each iteration is logged with the complete operator selection and the product of the five selected probabilities, which reports the joint probability of the realized configuration under the current wheels.

HAOS maintains six roulette wheels for the five levels because of the conditional clustering method on the fourth level. Each wheel tracks one weight per option. A reward r adds to the weight of every arm selected for the credited configuration. Once per iteration, every weight is also multiplied by a common decay factor. This uniform decay is the only temporal mechanism in the weights. Both operations retain the starting weight as a lower bound:

$$w_i \leftarrow \max(w_i + r, w_0), \quad (3.1)$$

$$w_i \leftarrow \max(\gamma w_i, w_0), \quad \gamma = 0.95 \text{ by default}, \quad (3.2)$$

with every wheel starting from the same weight $w_0 = 10.0$. Before an arm is drawn, HAOS first computes its normalized weight $\hat{p}_i = w_i / \sum_l w_l$. A level-specific floor is applied to these values, followed by renormalization:

$$p_i = \frac{\max(\hat{p}_i, p_{\min})}{\sum_j \max(\hat{p}_j, p_{\min})}. \quad (3.3)$$

The floor parameter is the minimum probability any weight of any wheel can decay to. For the five levels, the floors are 0.025 for k , 0.05 for λ_q , the clustering methods, and the solver, and 0.10 for the paradigm respectively. Because Equation 3.3 renormalizes after flooring, these values are nominal pre-normalization floors rather than strict lower bounds on the final probabilities. HAOS retains three elements of the adaptive layer proposed by Ropke and Pisinger (2006) and used for general vehicle routing by Pisinger and Ropke (2007): initially equal weights, quality-dependent scores, and roulette selection proportional to the resulting weights. Its additive reward, global decay, and six-wheel hierarchy are novelties introduced for DRISPI. Section 4.2.3 finds that the additive reward is also what keeps those weights from ranking the arms.

HAOS adapts its weights by receiving rewards from later stages of the current or following iterations. First, there is an immediate channel for results produced directly by the current DRI iteration and a deferred channel for routes that prove useful during later pool recombination. Table 3.1 gives the deployed reward values. Both channels compare a candidate with the incumbent best cost and the previous iteration’s cost, but the deferred scores are smaller because several earlier configurations can contribute routes to the same recombined solution.

Table 3.1: Default Immediate and Deferred HAOS Rewards.

Outcome	Immediate	Deferred
New incumbent best	10.0	7.0
Improvement over previous iteration only	4.0	3.0
No improvement	1.0	0.0
No solution	0.0	0.0

The immediate channel scores two candidates: the solution obtained after decomposition and subproblem routing, and the solution obtained after BG-AILS. Both rewards are credited to the same five-level configuration drawn for that iteration and therefore accumulate. Each selected arm receives the full reward. By doing so, HAOS learns from the performance of the complete configuration but does not isolate the contribution of one level or model interactions between levels. In the asynchronous pipeline of Section 3.7 the BG-AILS reward of iteration i is credited only after decompose-route of iteration $i+1$, so the next HAOS draw does not yet see it; both rewards still land on the θ of iteration i .

When route-based clustering draws a k larger than the incumbent route count, only the applied k changes; the credited configuration θ keeps the originally drawn k -arm.

The deferred channel is evaluated after SC/SP has selected routes from the pool and the resulting solution has received its post-SC/SP improvement pass. Every stored route carries a tag containing the complete configuration under which it was generated. The deferred reward is credited once to every distinct tag among the routes selected by SC/SP, even if those tags originate from much earlier iterations. Routes created only during the post-SC/SP improvement pass are excluded because no HAOS selection produced them. Deferred attribution is necessary because the usefulness of one stored route may become visible only when it is combined with routes generated later.

During the first 10 iterations, HAOS performs the same five-level draw but suppresses both reward updates and decay. All weights therefore remain equal, making every option uniform within the wheel from which it is eligible to be drawn. This warm-up period lets the first incumbent and an initial route pool form before early outcomes influence later probabilities. Learning begins with iteration 10, after which the immediate and deferred updates follow the same weight system.

3.2 Decomposition

The decomposition layer runs at the beginning of every iteration in the DRI loop, adapting the decompose-route-improve terminology of Kerscher and Minner (2025), who propose the framework for large-scale vehicle routing problems with time windows that this work extends to the CVRP at very-large scale in a fully parallelized, self-tuning setting. HAOS (Section 3.1) controls this stage through four levels, redrawn at every iteration: first the number of subclusters k ; second the clustering paradigm, either vertex-based, which partitions the customers directly, or route-based, which partitions the routes of the current incumbent solution and propagates the resulting labels to their customers; third the clustering method used within that paradigm; and fourth the demand weight λ_q of the dissimilarity metric utilized a subset of clustering approaches. Additionally, a fifth quantity that also feeds the metric, the angular offset ϕ , is drawn uniformly from $[0, 2\pi)$ at every iteration independently of HAOS. ϕ ensures that the wraparound point described below does not stay fixed across iterations.

For the decomposition, we form a dissimilarity metric that combines three terms for every pair of customers i and j with coordinates (x_i, y_i) and (x_j, y_j) and demands q_i and q_j : their Euclidean distance, an angular term, and a demand term,

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + \lambda_\theta \Delta\theta_{ij}^2} \left(1 + \lambda_q \frac{|q_i - q_j|}{Q} \right). \quad (3.4)$$

$\Delta\theta_{ij}$ is the shortest angular distance between customer i 's and customer j 's polar angle around the depot, computed with the offset ϕ introduced above. λ_θ scales the angular term to the average coordinate magnitude of the instance, so that neither term dominates across instances of different spatial extent. ϕ shifts where the $\Delta\theta_{ij}$ wraparound at 2π falls. The demand factor applies the multi-feature dissimilarity metric of Kerscher and Minner (2025), who combine spatial, temporal, and demand features, here to a setting without time windows, utilizing only the demand ratio $\lambda_q|q_i - q_j|/Q$ remains alongside spatial features.

Table 3.2: Inputs Used By Each Decomposition Method.

Method	Paradigm	Input	Angular	Demand
K-Means	vertex	raw coordinates	—	—
Fuzzy C-Means	vertex	raw coordinates	—	—
K-Medoids	vertex	dissimilarity matrix	✓	✓
Agglomerative (Average)	vertex	dissimilarity matrix	✓	✓
Agglomerative (Complete)	vertex	dissimilarity matrix	✓	✓
K-Means	route	route centroids	—	—
Agglomerative (Average)	route	route centroids	—	—
Agglomerative (Complete)	route	route centroids	—	—

Both clustering paradigms trace back to the classic cluster-first, route-second scheme for capacitated routing (Gillett and Miller, 1974; Fisher and Jaikumar, 1981), reviewed with route-first cluster-second and with vertex-based versus route-based clustering in Section 2.3.3. Vertex-based clustering partitions the customers directly and offers five methods (Table 3.2). K-Means (MacQueen, 1967) and Fuzzy C-Means (Bezdek et al., 1984) cluster on raw two-dimensional coordinates, Fuzzy C-Means hardened to a partition by assigning each customer to its highest-membership cluster. The remaining methods K-Medoids, partitioning around medoids (Kaufman and Rousseeuw, 1990), and Agglomerative Hierarchical Clustering under average or complete linkage (Kaufman and Rousseeuw, 1990) instead cluster directly on the dissimilarity matrix of Equation 3.4. They are the only three method arms that are influenced by λ_q and ϕ .

Route-based clustering represents each route by the centroid of its customers' coordinates and clusters the centroids with K-Means or with the same average or complete linkage, applied to the plain Euclidean distance between centroids rather than to the dissimilarity matrix (Table 3.2). This follows the barycenter-clustering decomposition strategy studied by Santini et al. (2023).

The clusters are then handed over in form of a set of independent subproblems to the following routing layer (Section 3.3) to be solved. The dissimilarity matrix, rebuilt from (λ_q, ϕ) , still feeds the boundary detection used at the end of the DRI loop

in the BG-AILS (Section 3.4) stage regardless of which paradigm decomposition uses. Labels then propagate from routes to customers directly, since every customer belongs to exactly one route in the incumbent solution.

3.3 Routing

Once decomposition produces the k subclusters (Section 3.2), the routing stage builds one sub-instance per subcluster, each with its own internal node IDs and its own copy of the depot. It then solves all k of them in parallel, independently of each other. The fifth and last HAOS level draws the solver once per iteration and applies it for every subcluster that iteration, so all k sub-instances are solved by the same one of four solvers. Each sub-instance receives a time budget of $\max(\text{floor}, \text{size} \times \text{rate})$ seconds, with a default rate of 0.06 seconds per customer and a floor of 5 seconds, so small subclusters are not starved of search time. The k sub-instances are dispatched to a pool of worker processes pinned to the cores reserved for this stage; if k exceeds the worker count, the remainder queue in further waves. Section 3.7 details how this pool fits into the wider core split. A wall-clock timeout derived from a fitted wave-time model guards against a hung worker.

The four solvers are monolithic, full-instance CVRP heuristics that DRISPI treats as black boxes: AILS-II (Máximo et al., 2024)³, an adaptive iterated local search with an elite-solution pool for large instances that extends an earlier adaptive ILS hybridized with path-relinking (Máximo and Nascimento, 2021), runs as a Java subprocess; FILO (Accorsi and Vigo, 2021)⁴, a ruin-and-recreate local search built on the COBRA granular-neighborhood library, runs as a compiled C++ subprocess; FILO2 (Accorsi and Vigo, 2024)⁵, an evolution of FILO for instances up to one million customers, also a C++ subprocess; and PyVRP (Wouda et al., 2024)⁶, a Python package implementing hybrid genetic search (Vidal et al., 2012; Vidal, 2022) that DRISPI calls in-process rather than as a subprocess. The four solvers selected are not an arbitrary shortlist. Queiroga et al. (2026), who introduce the XL instance set this thesis focuses on (Section 2.1), benchmark them against several other state-of-the-art heuristics and report that AILS-II and FILO2 dominate the best-known solutions on the XL range itself, while HGS-CVRP remains the strongest performer on the smaller, medium-scale instances that a typical DRISPI subcluster is closer in size to. The selection of solver level spans exactly this cluster size spectrum.

Once the routes returned by the k individually solved subclusters are concatenated, they form one combined solution for the full instance. Because the subclusters partition the customers, no route returned at this stage spans two subclusters, and the concatenation needs no reconciliation. Each route is tagged with the subcluster of

³<https://github.com/INFORMSJoC/2023.0106>

⁴<https://github.com/acco93/filo>

⁵<https://github.com/acco93/filo2>

⁶<https://pyvrp.org>

its customers, information that BG-AILS (Section 3.4) reuses once its perturbations start moving customers across subcluster boundaries. This combined, subcluster-tagged solution is the input BG-AILS perturbs and re-optimizes next.

3.4 Boundary-Guided Adaptive Iterated Local Search

The decomposition introduces structural local optima by drawing partitions that are, from the routing problem’s point of view, potentially arbitrary: a cluster boundary sits wherever the drawn clustering method happened to cut, not necessarily wherever the customer geometry would suggest a natural route boundary. Each subcluster can be routed to a good local optimum in isolation (Section 3.3), yet the concatenated solution can still carry structural defects that only exist across the concatenation seams: routes on either side of a boundary that would be cheaper reconnected across it, or customers left on the wrong side of an arbitrary cut. A plain, unrestricted improvement pass over the whole solution could in principle fix these defects, but under a short iteration time budget it spends most of that budget on customers that already sit comfortably inside one subcluster’s interior, where the decomposition did nothing that needs correcting. Boundary-Guided Adaptive Iterated Local Search (BG-AILS) is DRISPI’s answer to this asymmetry: an improvement stage that reuses AILS-II’s own local search but spends its comparatively short time budget disproportionately on the boundary regions the decomposition itself created.

At its core, BG-AILS is AILS-II (Máximo et al., 2024), the same monolithic solver DRISPI can also draw as a HAOS routing arm (Section 3.3), run here purely as an improvement operator rather than a cold-start solver. Two additions to stock AILS-II, both introduced for this thesis, make that role possible: a `-initialSolution` flag that seeds the search with the combined post-decompose-route solution instead of building one from scratch, and a `-initialOmega` flag that overrides the starting value of ω , AILS-II’s own diversification-control parameter governing how strongly its internal perturbation cycle disturbs the incumbent before every local-search pass (Máximo and Nascimento, 2021). Seeding AILS-II with the concatenated solution is not sufficient on its own, however: a local search that starts from that solution and only ever considers its immediate neighborhood tends to repair it straight back toward the same seam defects the decomposition introduced, because nothing has told it that the seams, rather than a subcluster’s interior, are where a profitable move is more likely to be waiting. BG-AILS therefore precedes the AILS-II call with a purpose-built perturbation step, implemented in Python, that displaces customers specifically across the boundaries between subclusters, using a single, large-neighborhood move rather than the small single-customer or single-edge moves a subsequent local search pass could trivially undo in one or two steps. This mirrors the classical iterated-local-search argument, introduced for the double-

bridge move on the traveling salesman problem (Martin et al., 1991) and generalized since (Lourenço et al., 2003), that perturbation strength must be calibrated so that the following local search neither collapses straight back into the same local optimum nor is reduced to an effective random restart. No formal proof establishes this property for an arbitrary local search and perturbation pair; the argument, for the double bridge as for cross-reconnect, is structural rather than a theorem: both disconnect the current solution through a simultaneous multi-edge change that a local search built from smaller, incremental moves does not consider as a single step, and so is unlikely to reverse in one move of its own.

The perturbation step reuses the dissimilarity matrix built for the decomposition stage (Section 3.2) rather than defining a separate notion of distance, so that “boundary” remains consistent between BG-AILS and whichever clustering method drew the partition. The deployed worker rebuilds that matrix from the same (λ_q, ϕ) rather than receiving d_{ij} . Algorithm 2 gives the full perturbation procedure, from boundary scoring through the chain of cross-reconnects, ending at the AILS-II call itself. For every customer i with cluster label $\kappa(i)$ under the current partition, BG-AILS first scores how close i sits to a customer outside its own cluster,

$$b_i = 1 - \frac{\min_{j: \kappa(j) \neq \kappa(i)} d_{ij} - d_{\min}}{d_{\max} - d_{\min}}, \quad (3.5)$$

$$\tilde{b}_i = \frac{b_i}{|\kappa(i)|} \left(1 + \alpha \max\left(0, \frac{\sigma - |\kappa(i)|}{\sigma}\right) \right), \quad (3.6)$$

where $d_{\min}$ and $d_{\max}$ are the smallest and largest nearest-other-cluster dissimilarity across all customers, so a customer with a nearby neighbor in a different cluster scores close to one and a customer deep inside its cluster scores close to zero. Equation 3.6 divides by the size $|\kappa(i)|$ of i ’s own cluster, so boundary attention is not monopolized by whichever cluster happens to contribute the most customers near a seam, and multiplies by a boost that grows as the cluster shrinks below a small-cluster cap σ (20 customers by default) up to a factor of $1 + \alpha$ ($\alpha = 0.5$ by default), so the very small clusters route-based decomposition can produce are not perturbed less just because they contribute few customers in absolute terms. After renormalizing $\tilde{b}_i$ to $[0, 1]$ via min–max to obtain $\hat{b}_i$, a boundary threshold τ gates the result,

$$\rho_i = \frac{\max(0, \hat{b}_i - \tau)}{1 - \tau}, \quad \tau = 0.5 \text{ by default}, \quad (3.7)$$

setting ranks at or below τ to zero and rescaling the remainder linearly back onto $[0, 1]$, structurally the same floor-then-renormalize pattern HAOS applies to its wheel probabilities (Equation 3.3), just against a fixed threshold rather than a per-level minimum share. This is the sense in which BG-AILS is more efficient than an unrestricted improvement pass under the same time budget: with $\tau = 0.5$ it never spends perturbation effort on the lower half of customers by boundary rank, in particular never on a customer sitting deep inside a cluster’s interior, and instead concentrates

entirely on the customers the decomposition itself flags as most likely to be sitting on a defect it created.

Boundary ranks are defined per customer, but the only perturbation BG-AILS applies acts on whole routes at once: a cross-reconnect move that cuts two routes at a random position each and exchanges their tails, so routes r_i and r_j cut after position a and b respectively become $r_i[:a] \frown r_j[b:]$ and $r_j[:b] \frown r_i[a:]$. This is the only perturbation operator BG-AILS implements. It is a large-neighborhood move by construction, since it can reassign an arbitrarily long tail of customers between two routes in a single step, which is precisely what keeps it from being trivially undone by the small single-customer and single-edge moves of a subsequent local search pass. A route’s own weight for perturbation is the maximum boundary rank among its customers, and the first route of a cross-reconnect pair is drawn from these weights, either by roulette-wheel sampling or, under DRISPI’s default greedy pair-selection setting, by taking the arg-max outright. Its partner is not drawn from the same weights again but from a per-route affinity toward each other cluster – the inverse of the mean dissimilarity between that route’s customers and a given cluster’s customers, row-normalized to sum to one – used first to pick which neighboring cluster to reach into from the first route’s own affinity row, then to pick, among the routes majority-assigned to that cluster, the one with the highest affinity back toward the first route’s own cluster. This two-step draw is what keeps a cross-reconnect pair spatially adjacent across one real cluster boundary, rather than pairing two routes that both happen to contain a boundary customer but face away from each other. BG-AILS repeats this draw-and-reconnect step for a chain of k iterations by default, one per subcluster (a $k - 1$ variant is also implemented but is not the deployed default), with each step in the chain acting on the routes as already modified by the previous one. When decomposition realizes only a single cluster, there is no boundary to guide perturbation toward, and BG-AILS skips the perturbation step entirely, falling back to a plain AILS-II improvement pass on the unperturbed combined solution.

The partner of the greedy first route is drawn in two steps from that route’s affinity row: first a neighboring cluster, then the route inside it with the highest affinity back. The chain compounds because the route weights are recomputed after every reconnect, so each of the k steps can act on a route an earlier step already changed.

Figure 3.2 walks through one such call on XL-n1281-k29. Panel (a) is the concatenated decompose-route input, customers colored by the six subclusters of the pinned K-Medoids partition. Panel (b) maps pre-threshold boundary rank $\hat{b}_i$ and highlights the six route pairs the chain selects. Panel (c) shows those routes after the cross-reconnect cuts. Panel (d) is the solution after the AILS-II pass, customers colored by evaluated-move count from a separate instrumented run under the same input solution and budget.

Algorithm 2 Boundary-Guided Adaptive Iterated Local Search (BG-AILS)**input:** solution S ; labels $\kappa(\cdot)$; λ_q ; ϕ ; subcluster count k ; budget T **parameters:** σ , α , τ , ω_0 (Table 4.1)

- 1: **if** $k > 1$ **then**
- 2: rebuild d_{ij} from (λ_q, ϕ)
- 3: compute $\tilde{b}_i$ by (3.5)–(3.6) and ρ_i by (3.7) for every customer i
- 4: $\rho(r) \leftarrow \max_{i \in r} \rho_i$ for every route $r \in S$
- 5: **for** $t \leftarrow 1$ **to** k **do**
- 6: $r_a \leftarrow \arg \max_{r \in S} \rho(r)$
- 7: draw a neighboring cluster c from the affinity row A_{r_a} ,
- 8: $r_b \leftarrow \arg \max \{A_{r, \kappa(r_a)} : r \in S \text{ assigned to } c\}$
- 9: cut r_a and r_b at random positions p_a, p_b and exchange their tails
- 10: update $\rho(r_a)$ and $\rho(r_b)$
- 11: **end for**
- 12: **end if**
- 13: **return** AILS-II(S, ω_0, T)

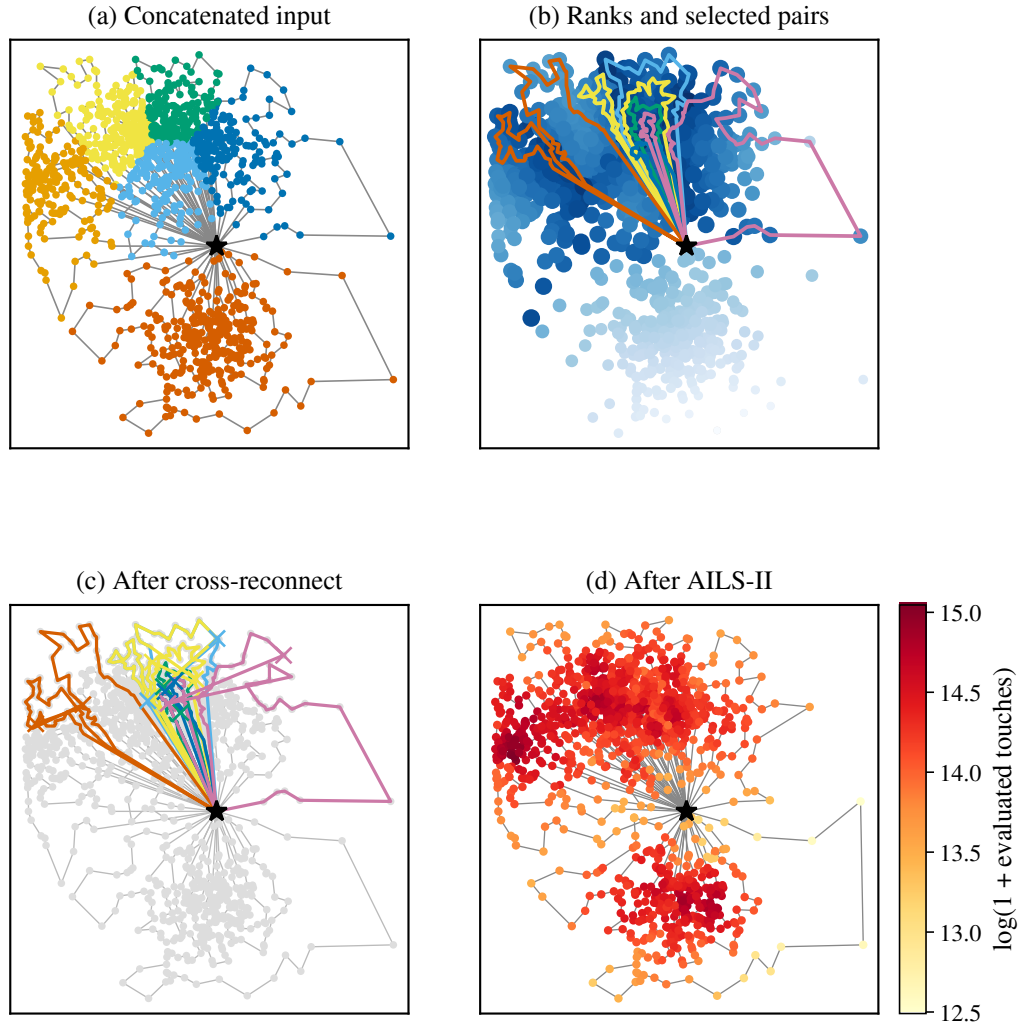


Figure 3.2: Illustrative BG-AILS Call on Instance XL-n1281-k29.

Colorbar Applies to Panel (d) Only.

Table 4.1 lists the values of the parameters this stage introduces on top of stock AILS-II. Of these, only the initial-omega override reaches into AILS-II itself; every other AILS-II hyperparameter governing its own local search and diversification control (the parameters $d_{\min}$, $d_{\max}$, γ , and φ of Máximo and Nascimento (2021)) is left at AILS-II’s own defaults. Overriding ω to 10 against AILS-II’s own starting value of roughly 30 lowers rather than raises the solver’s internal perturbation degree: BG-AILS has already displaced customers across subcluster boundaries with its own cross-reconnect chain before AILS-II starts, so the pass that follows is asked to intensify around that disruption rather than to add more of its own. The remaining parameters – the boundary threshold, the small-cluster cap and boost, the chain-count mode, and the pair-selection rule – are new and govern only the Python-side perturbation step described above.

The key difference between BG-AILS and a plain AILS-II improvement pass is therefore not the solver itself but this focus on boundary regions, imposed both by the perturbation step and by the short wall-clock budget BG-AILS is given. That budget follows neither the fixed per-customer rate used for subcluster solving (Section 3.3) nor a plain function of instance size, but a fitted model of the surrounding decompose-route stage’s own wall-clock time,

$$\text{budget} = \max(60\text{s}, 1.0 \times \hat{T}_{\text{DR}}), \quad (3.8)$$

$$\hat{T}_{\text{DR}} = 0.976 \times n_{\text{waves}}^{-0.180} \times \sum_{\text{waves}} \max(\text{per-cluster budgets in wave}), \quad (3.9)$$

refit on 835 post-strip pilot DRI iterations, where waves pack the current iteration’s per-subcluster solver budgets (Section 3.3) into the granted DRI worker count in partition-submit order. The campaign of Chapter 4 runs BG-AILS asynchronously under this budget: a dedicated worker perturbs and improves the current iteration’s solution while the main process decomposes and routes the next one. Section 3.7 returns to Equation 3.8 as the quantity that keeps that handoff tight.

Under this comparatively short wall-clock budget, BG-AILS should outperform an unguided AILS-II pass over the same combined solution, because the unguided pass spends an unknown and likely large share of that budget searching subcluster interiors the decomposition had already solved well. We tested this on all 100 XL instances (Section 4.1.1) at three checkpointed decompose-route solutions each, all arms forked from the same checkpoint: a stock AILS-II pass at the solver’s own defaults, a blind control that applies one cross-reconnect per subcluster to uniformly drawn route pairs, and BG-AILS in its deployed configuration (greedy first-route selection and no local-search mask), the control and BG-AILS at $\omega = 10$. Every arm receives the same AILS-II budget, the checkpoint’s own Equation 3.8 value; the Python-side perturbation of the two perturbing arms is a surplus on top of it, a median 0.28% of that budget for BG-AILS and 0.010% for the control. The checkpoints are generated under one pinned decomposition configuration, vertex

K-Medoids, with HAOS, the route pool, and SC/SP out of the loop, so what is compared is one improvement stage against another on identical input, not two full runs.

BG-AILS statistically significantly outperforms both baselines (paired Wilcoxon signed-rank tests, alternative greater, Holm-corrected in a family of two). It improves on the stock AILS-II pass by 0.056 pp of cost improvement over the input solution (Hodges–Lehmann paired difference, 95% CI [0.049, 0.063], Holm-corrected $p = 7.0 \times 10^{-11}$, ahead on 85 of 100 instances) and on the blind control by 0.058 pp (95% CI [0.052, 0.065], Holm-corrected $p = 2.7 \times 10^{-12}$, ahead on 89 of 100). The AILS-II comparison is the practical one and carries the perturbation together with the ω override as the package the framework deploys. The control comparison holds ω and chain length fixed and isolates the boundary guidance itself. The paired difference between the control and stock AILS-II is consistent with zero (Hodges–Lehmann -0.004 pp, 95% CI $[-0.011, 0.002]$), so the gain is not explained by perturbing the concatenated solution at all. Figure 3.3 shows the arm means and the paired differences behind those tests.

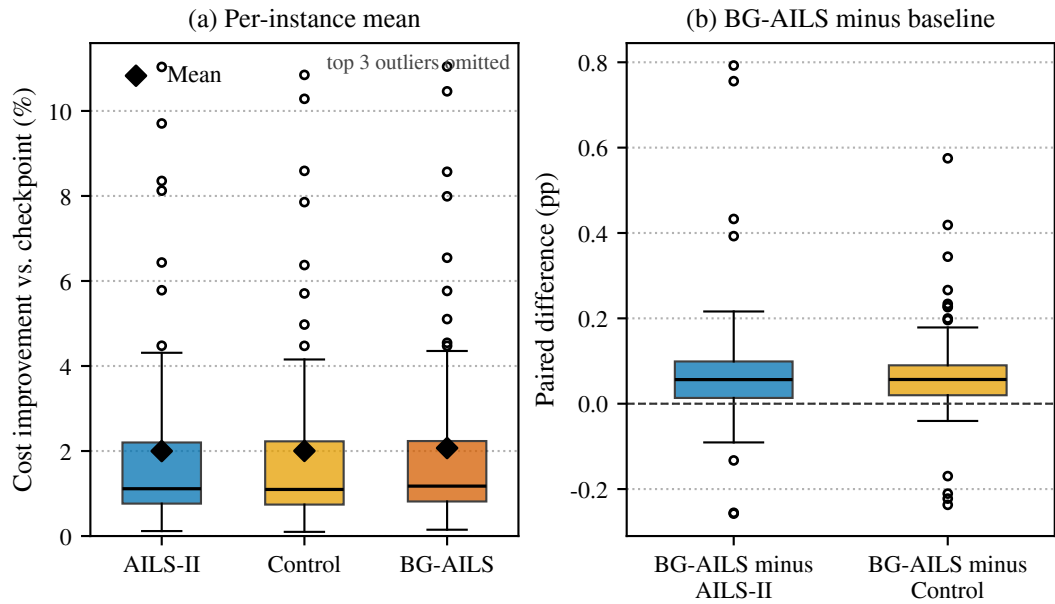


Figure 3.3: Per-Instance Mean Cost Improvement (a) and Paired BG-AILS Differences (b).

Three limits apply. The comparison is one DRI iteration, so it says nothing about what these routes are worth once the pool recombines them. It uses one pinned decomposition configuration, under which the clustering and the boundary metric read the same dissimilarity matrix, as they do for three of the eight deployed clustering methods but not for the five that cluster on raw coordinates or route centroids. The touch counts in Figure 3.2(d) come from a separate, identically configured instrumented run rather than from the runs the costs above are measured on.

BG-AILS closes the DRI loop: its output routes are contributed to the route pool like every other stage’s output (Section 3.5), and the next iteration begins with a fresh

HAOS draw over the decomposition and routing levels (Section 3.1). Periodically, rather than continuing the DRI loop for another iteration, DRISPI instead steps back and recombines the pool accumulated across many such iterations through set covering or set partitioning, the subject of the next section.

3.5 Route Pool and SC/SP Recombination

Periodically, DRISPI steps back from the current incumbent and recombines every route accumulated so far into one new candidate solution, using set covering or set partitioning (SC/SP) as the recombination model. Alternating a metaheuristic with an exact set-partitioning solve in this way is an established matheuristic pattern (Jourdan et al., 2009). Cavaliere et al. (2022) apply essentially this pairing to large-scale CVRP instances directly, by alternating LKH-3 local search with a restricted set-partitioning solve. Doing so only pays off if a shared memory of routes exists for it to recombine, so this section first describes that memory, the route pool, before turning to the SP and SC models themselves and the hyperparameter that decides between them.

The route pool stores one entry per distinct customer set $C_r \subseteq C$, where C is the customer set of the instance (Section 2.1), keyed by the frozen customer set rather than by the ordered sequence of customers a route visits them in: two routes serving the same customers in a different order, or at a different cost, are treated as the same column, and the cheaper of the two overwrites the other. This column-oriented memory, populated online during the DRI loop and periodically recombined through a partitioning model, follows the adaptive-memory principle of Rochat and Taillard (1995), who introduce the route pool and its set-partitioning recombination for vehicle routing.

Two pathways feed the pool every iteration, both through the same contribute-before-adopt rule: a candidate solution is added to the pool before the pipeline decides whether to adopt it as the new incumbent, so a losing candidate still leaves its routes behind for later reuse. The DRI loop feeds the pool at least twice per iteration, once from its decompose-route stage (Section 3.3) and once from the following improvement stage (BG-AILS (Section 3.4)). The SC/SP step covered in this section feeds the pool only indirectly. Solving SC or SP selects among the customer sets already in the pool and creates no new ones, so its own output adds nothing. The post-SC/SP improvement pass that polishes its result (Section 3.6) is what can move customers between routes and through that generate new columns. Every contributed route carries a tag recording the full HAOS selection that produced it, that is, k , the demand weight, the paradigm, the method, and the solver, with a separate flag marking routes from the post-SC/SP improvement pass; the tag survives even once HAOS has moved on to a different selection, which is what later lets reward attribution and pool inspection trace a route back to its origin. Whenever

a candidate is adopted as the new incumbent, its routes, already in the pool by the rule above, are marked elite; elite routes are exempt from eviction, so the pool never loses the one partition it is guaranteed to already have, namely the incumbent's own.

Every SC/SP solve also updates two running scores on each pool entry: A quality and a diversity score. The quality score represents a column's usefulness for the solver during the SC/SP solve. Therefore, an entry never touched by a solve carries no quality score yet. The diversity score is the mean Jaccard distance (Jaccard, 1901) between a route's customer set and every other pool entry's customer set,

$$\bar{J}(r) = \frac{1}{|R| - 1} \sum_{r' \in R \setminus \{r\}} \frac{|C_r \Delta C_{r'}|}{|C_r \cup C_{r'}|}, \quad (3.10)$$

where R is the pool at the time of the update, recomputed for every entry whenever any solve happens. New quality or diversity scores are appended to a three-value window, from which final scores are read as the mean of their respective window. This ensures that single unusually high or low values do not dominate a route's standing.

The route pool has a maximum size (10,000 columns by default) to keep the computational complexity from blowing up. We therefore establish an eviction policy that ranks the non-elite entries on both scores and combines the ranks into one fitness value. Among the entries that have been scored at least once, ascending order by mean quality score gives rank one to the weakest route and the highest rank to the strongest. Independently, ascending order by mean diversity score gives rank one to the most redundant route and the highest rank to the most unique. Writing $\rho_q(r)$ and $\rho_d(r)$ for these two ranks, fitness is $\Phi(r) = \rho_q(r) + w_d \cdot \rho_d(r)$, with the diversity weight w_d (0.5 by default) controlling how much redundancy counts against a route relative to raw quality. The entries with the lowest $\Phi(r)$ are evicted first. Because both terms grow toward the same end of their respective orderings, a route that the LP relies on heavily, but that is also nearly identical to several other pool entries, can rank as a stronger eviction candidate than a weak but singular one, because redundancy drives the diversity term up regardless of quality. The eviction policy only triggers once the pool exceeds its maximum size and then only removes the overflow amount. Unsourced entries are evicted only if the overflow cannot be covered by scored, non-elite entries alone. Elite entries are categorically exempt from eviction.

The recombination model itself treats the pool as a column set. Let R be the set of routes currently in the pool, c_r the cost of route $r \in R$, that is, the Euclidean depot-customers-depot round trip over the travel costs of Section 2.1, and $x_r \in \{0, 1\}$ a decision variable selecting route r . Set partitioning, in the formulation Balinski and

Quandt (1964) introduce for exactly this delivery-routing setting, reads

$$\min \sum_{r \in R} c_r x_r \quad (3.11)$$

$$\text{s.t.} \quad \sum_{r \in R: i \in C_r} x_r = 1, \quad \forall i \in C \quad (3.12)$$

$$x_r \in \{0, 1\}, \quad \forall r \in R. \quad (3.13)$$

Solving Equations 3.11–3.13 recombines route fragments that have never been jointly considered, for example because they may have entered the pool from different subclusters, different HAOS selections, or different iterations, into one coherent solution that still covers every customer exactly once. This is the sense in which SP looks across the boundaries any individual DRI iteration’s decomposition imposes, something no individual subcluster solve or BG-AILS perturbation can do on its own.

DRISPI builds this model once in Gurobi⁷ and solves it twice. It first relaxes Equation 3.13 to the continuous $x_r \in [0, 1]$ and solves the resulting linear program. This LP relaxation allows the solver to assign fractional weights rather than the binary ones to each route, making it significantly easier and faster to solve. The resulting fractional weights are then utilized as the quality scores fed back into the route pool above, so every SC/SP solve doubles as a scoring pass over the pool regardless of what it ultimately selects. This follows the same spirit in which column-generation approaches read pricing information out of LP duals (Lübbecke and Desrosiers, 2005), even though DRISPI’s route pool is populated by heuristic solves rather than a pricing subproblem. For the second solve, the variables are then switched to binary, warm-started from the LP solution, and re-optimized as a mixed-integer program under a wall-clock time limit and a relative optimality gap (720 seconds and 0.025% by default). A wallclock time limit is required to cut off the search with a good, if not provably optimal, solution rather than run to completion.

The equality constraint of Equation 3.12 is only a meaningful choice if the pool offers more than one real alternative for most customers. A pool still dominated by a single incumbent partition would just reproduce it again and again under SP. DRISPI therefore checks, before every SC/SP trigger, whether every customer in the instance is covered by at least a fixed minimum number of pool routes (five by default). Only when that holds does SP run. Otherwise DRISPI falls back to the less constrained set covering (SC), the inequality-constrained relaxation of the same formulation (Garfinkel and Nemhauser, 1969), whose formal problem statement and LP relaxation are given in Appendix B. SC only requires every customer to be covered at least once rather than exactly once, a condition the pool always satisfies, because the elite routes described above are never evicted and already form a feasible partition on their own. This is also why the fallback is safe and not merely

⁷<https://www.gurobi.com>

convenient: SP is never blocked by infeasibility, only by a pool that has not yet accumulated enough alternatives, and SC exists to give the SC/SP step something well defined and always solvable to run on every scheduled trigger, warmup aside, no matter how much the pool has matured. The price of that permissiveness is that SC's selected routes can overlap on a customer, meaning a customer can be visited by more than a single route and therefore more than exactly once. Since this would violate our problem statement formulated in Section 2.1, a savings-based deduplication heuristic then removes each duplicated customer from every selected route except the one where removing it would save the least cost, restoring a feasible partition.

A solved partitioning problem will pair routes into a new candidate solution which haven't been evaluated together before. And while search heuristics have been employed in identifying the routes used, the resulting new set of routes has not been polished by evaluating for example inter-route moves amongst each other. To do just that the SC/SP step is combined with a post-SC/SP improvement pass (Section 3.6) to improve the partitioning result even further if possible.

3.6 Post-Recombination Improvement

Once SC/SP (Section 3.5) recombines the route pool into one partition, DRISPI runs one further pass over it using AILS-II as solver, seeded only with that partition as an initial solution. Unlike in BG-AILS (Section 3.4), this step focuses less on perturbation but rather refinement by local search. This post-recombination improvement pass also allows the solver to touch all routes instead of biasing moves towards certain neighborhoods. The pass runs under its own wall-clock budget (360 seconds by default) on the same core reservation as the SC/SP solve that precedes it; Section 3.7 covers how exactly a parallel architecture of DRISPI looks.

The refined routes feed back into the route pool exactly like every other stage's output (see Section 3.5): a route whose customer set matches one SC/SP already selected keeps that route's existing tag, while a route AILS-II creates by moving customers between routes is added as new, tagged as an improvement route rather than credited to a HAOS selection of its own.

This is also where the deferred half of HAOS's reward system (Section 3.1) closes. The pass's outcome, a new best, an improvement over the previous iteration without beating the incumbent, or neither, is scored through the deferred branch of HAOS's reward function and credited not to the current iteration's own draw but to every distinct HAOS tag the routes SC/SP selected already carried. That is, to whichever earlier decompose-route or BG-AILS draws produced those specific routes, regardless of how many iterations ago that was. Improvement-route tags are excluded from this credit, since AILS-II created them without any HAOS draw of their own to reward.

3.7 Parallelized Pipeline Orchestration

The preceding sections describe HAOS, decomposition, routing, BG-AILS, the route pool and SC/SP, and the post-SC/SP improvement pass as if the DRI loop executed them one after another, the sequential picture Figure 3.1 shows. DRISPI instead splits eight cores into three disjoint groups: six for the main loop and the DRI routing pool of Section 3.3, one for the BG-AILS worker of Section 3.4, and one for the SC/SP and post-SC/SP improvement worker of Sections 3.5 and 3.6. The main process shares the six DRI cores rather than occupying a ninth.

Figure 3.4 shows this overlap across two consecutive iterations. After decompose-route of iteration i the loop waits for job $i-1$ and drains it ①, then launches job i on the new solution ②. After decompose-route of iteration $i+1$ the same pair of steps waits for job i , drains it, and launches job $i+1$. A finished SC/SP job is drained without blocking ③ at the start of the following iteration, before the next HAOS draw.

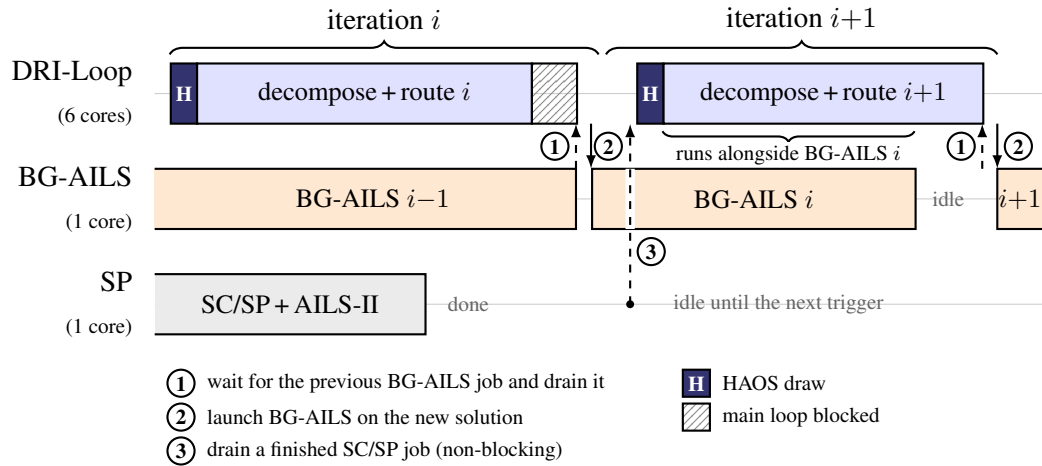


Figure 3.4: Two Consecutive DRISPI Iterations in One 8-Core Container (6 for DRI, 1 for BG-AILS, and 1 for SP). Block Lengths Are Illustrative.

Equation 3.8 sizes the BG-AILS budget to the decompose-route stage it overlaps with, which is what keeps the wait at ① short. A budget much shorter than the next decompose-route pass would leave the BG-AILS core idle before the handoff; a budget much longer would leave the DRI cores blocked on a job that already had the next iteration's input ready. In Figure 3.4 the wait after decompose-route of iteration $i+1$ vanishes because job i finished during that pass.

The SC/SP worker uses a skip policy rather than a blocking handoff. The campaign checks the trigger only at iteration boundaries, so a job starts at the first boundary after the 20-minute mark, not exactly at it. DRISPI then dispatches a snapshot of the route pool and the incumbent only if the worker is idle; if it is busy, the trigger is skipped rather than queued. The worker owns both the SC/SP solve and the post-SC/SP improvement pass of Section 3.6, running them one after another on the

same core. Those two ceilings, 720 s and 360 s, sum to 18 minutes and therefore leave two minutes of slack under the 20-minute interval; a skip is possible only if enqueue and overhead consume that slack.

Algorithm 3 is the deployed loop.

Algorithm 3 DRISPI Main Loop as Deployed (Async. BG-AILS and SP)

input: instance; time limit Γ ; stall limit Π
initialize: $R \leftarrow \emptyset$; S^* undefined; $i \leftarrow 0$

- 1: **while** elapsed time $< \Gamma$ **and** iterations without improvement $< \Pi$ **do**
- 2: **if** the SC/SP job has finished **then**
- 3: $\text{ABSORB}(S_{\text{SP}}, \Theta_{\text{SP}}, \text{deferred})$ $\triangleright \Theta_{\text{SP}}$: tags of selected routes
- 4: **end if**
- 5: $(\theta, \text{config}) \leftarrow \text{DRAW}(S^*)$ $\triangleright$ Algorithm 1
- 6: decompose into k subclusters under config and a fresh ϕ $\triangleright$ Section 3.2
- 7: **for** each subcluster **do in parallel**
- 8: route it with the drawn solver $\triangleright$ Section 3.3
- 9: **end for**
- 10: $S_{\text{DR}} \leftarrow$ concatenated subcluster routes; $\text{ABSORB}(S_{\text{DR}}, \{\theta\}, \text{immediate})$
- 11: **if** a BG-AILS job is in flight **then**
- 12: wait for its result (S_{BG}, θ') ; $\text{ABSORB}(S_{\text{BG}}, \{\theta'\}, \text{immediate})$
- 13: **end if**
- 14: dispatch BG-AILS on $(S_{\text{DR}}, \kappa, \lambda_q, \phi)$ $\triangleright$ Algorithm 2; θ for credit
- 15: **if** SC/SP is due **and** the sp worker is idle **then**
- 16: dispatch SC/SP-JOB(R, S^*) to the sp worker
- 17: **end if**
- 18: $\text{DECAY}()$; $i \leftarrow i + 1$
- 19: **end while**
- 20: **return** S^*
- 21: **function** $\text{ABSORB}(S, \Theta, \text{channel})$
- 22: $r \leftarrow$ reward of S in this channel (Table 3.1)
- 23: add the routes of S to R ; **if** $c(S) < c(S^*)$ **then** $S^* \leftarrow S$
- 24: **foreach** $\theta \in \Theta$ **do** $\text{CREDIT}(\theta, r)$
- 25: **end function**
- 26: **function** $\text{SC/SP-JOB}(R, S^*)$
- 27: **if** min. coverage of $R \geq 5$ **then** $S \leftarrow \text{SP}(R)$ **else** $S \leftarrow \text{dedup}(\text{SC}(R))$
- 28: **return** AILS-II(S) under the improvement budget $\triangleright$ Section 3.6
- 29: **end function**

After decompose-route the loop waits for the previous BG-AILS job ①, drains it through Absorb, and launches the next job on $(S_{\text{DR}}, \kappa, \lambda_q, \phi)$ ②, keeping θ only for later credit. A finished SC/SP job is absorbed without blocking ③ at the start of the next iteration. A synchronous mode that runs both stages inline is implemented as

a code option and recovers the sequential picture of Figure 3.1.

4 Computational Study

This chapter tests DRISPI on the 100 XL instances of Queiroga et al. (2026) against the eight monolithic solvers benchmarked there. The study aims to answer two questions: The first is whether an adaptive decomposition framework that learns its decomposition and routing choices during a run can match state-of-the-art monolithic solvers at this scale. The second is whether the weights HAOS learns along the way provide interpretable insights about the instances they were learned on.

Section 4.1 describes the dataset, specifies implementation and hardware specifications, the parameter values, and explains experimental design choices. Section 4.2 shows and analyzes the results. Here we answer the primary and secondary research questions through comparison with the published solvers, break the results down by instance characteristics, examine the learned HAOS weights and the convergence of individual runs, and attribute the improvement to the stages that produced it.

4.1 Experimental Design

All 100 instances are solved by the same DRISPI configuration. Nothing is tuned per instance by design without any exceptions. Section 4.1.1 describes the instances and the reference solutions used as baseline, section 4.1.2 the software and hardware the campaign ran on, section 4.1.3 the parameter values, and section 4.1.4 the number of seeds and the statistical precision it buys for our experiment.

4.1.1 Dataset and Reference Solutions

The XL set (section 2.1) extends the generation scheme of the X instances of Uchoa et al. (2017) to up to 10,000 customers. Every instance places its nodes on a $[0, 1000] \times [0, 1000]$ grid with Euclidean distances and combines one level of each of four attributes. The depot is Random (R), Central (C) at (500, 500), or Eccentric (E) at (0, 0) in the bottom left corner. Customers are Random (R), Clustered (C), or Random-Clustered (RC) around a small number of seed points. Demand follows one of seven distributions: unitary (U), four ranges of increasing width (1–10, 5–10, 1–100, 50–100), a quadrant-dependent range (Q), or many small and few large demands (SL). The target average route length r lies in one of seven bands, from ultra-short ($r \in [3, 5]$) to ultra-long ($r \in [50, 200]$), and capacity Q is set so that a solution with the minimum number of routes attains it. Figure A.1 draws one

instance from each end of that range. XL-n3888-k1010 combines a random depot, customers clustered around two seed points, many small demands and a few large ones, and $r = 4.2$. XL-n9571-k55 combines a random depot, random customers, quadrant-dependent demand, and $r = 174.7$. The instance name references n and that minimum route count, e.g. XL-n4535-k1134. This k equals $\lceil \sum_{i=1}^n q_i / Q \rceil$, making it the mathematical minimum number of routes, because fewer vehicles could not carry the total customer demand under capacity set to Q . This k is fixed by the benchmark and unrelated to the subcluster count k of Section 3.1. Table C.1 breaks down the above mentioned characteristics on a per-instance basis along with DRISPI's result and comparison to initial and current BKS values.

The initial BKS values of Queiroga et al. (2026) come from the solver runs described in section 2.2 and were released as baselines with the CVRPLib BKS Challenge, a community competition opened on January 12, 2026, in which any team can submit an improved solution for automatic verification. By the current record,⁸ the challenge has improved 99 of the 100 initial values, leaving only XL-n1094-k157 to still hold its initial value. We report the percentage gap to baselines such as the initial and the current BKS:

$$\text{Gap} = 100 \times \frac{c - c_{\text{BKS}}}{c_{\text{BKS}}}. \quad (4.1)$$

The quantity c is the reported solution cost. The current BKS represents the official best known solution. However, we primarily compare against initial BKS, because published solvers produced it in two-hour single-thread runs on specified hardware (Section 2.2), using us an experimental design we can reproduce to allow for any algorithmic comparability. The official BKS values, on the other hand, have been achieved without constraints regarding computational, with many of them coming from day-long runs that rely on raw compute, rendering any fair comparison infeasible.

4.1.2 Implementation and Hardware Specifications

All experiments in this chapter run on a single, shared department server under Ubuntu 22.04.5 LTS with two Intel Xeon Platinum 8280L CPUs at 2.70 GHz (28 cores each, 56 cores and 112 threads total) and 5,928 GiB of RAM, with no GPU. DRISPI runs inside Docker containers (Docker 28.1.1) pinned to a disjoint physical-core range with `-cpuset-cpus`. This pinned CPU count, not a library thread-pool default, sets the framework's parallelism budget.

The campaign presented in section 4.2.1 is comprised of all 100 instances, each run on three seeds with a wallclock runtime budget and termination criterion of $T=2\text{h}$ (300 runs, 7,200 seconds each). For that we pin 48 of the server's 56 physical cores

⁸https://galgos.inf.puc-rio.br/cvrplib/index.php/en/bks_challenge/score/instances, accessed September 7, 2026.

as six exclusive 8-core Docker containers, three per socket, parallelizing as outlined in section 3.7 with a split into six DRI workers, one BG-AILS worker, and one SP worker. SMT siblings are left idle. Runs are packed into waves ordered by instance size so that the three seeds of the largest instance, XL-n10001-k1570, never share a wave. The full campaign takes 50 waves and about 100 hours of wall-clock time.

DRISPI uses Python 3.11, with dependencies locked via `uv`: Libraries PyVRP 0.13.3 (HGS-CVRP), scikit-learn 1.8.0 (K-Means and Agglomerative Clustering), scikit-learn-extra 0.3.0 (K-Medoids), scikit-fuzzy 0.5.0 (Fuzzy C-Means), gurobipy 13.0.1 (SC/SP), and `vrplib` 2.1.0 for instance parsing. AILS-II, FILO, and FILO2 are vendored as git submodules pinned to a fixed commit each and compiled at image build time, rather than resolved from a floating branch, so a rebuild reproduces the exact binaries behind the results in Section 4.2.1. The whole framework runs inside a Docker image built from this locked environment, so the software stack behind every reported run is fixed independently of the host it executes on.

4.1.3 Hyperparameters

Table 4.1 lists the parameter values of the benchmark configuration, which all 100 instances and all three seeds share. Chapter 3 introduces each parameter with its mechanism, the table collects the values used to achieve the results presented in section 4.2. Most parameters were tuned during development in isolated pilot studies per category (HAOS, Decomposition and Routing, BG-AILS, Route Pool, SC/SP, and Improvement). These small studies were comprised of a few instances and seeds, usually running a limited parameter grid search, aimed at finding a reasonable default rather than an optimum, and these studies are not reported here. The only exception is the BG-AILS comparison of Section 3.4, which ran on all 100 XL instances and fixed the first-route rule and the boundary mask; these two settings are therefore chosen on the benchmark set itself. No sensitivity analysis over the full set was run for any of the other values.

Not included in the table are the option sets HAOS draws from: the number of subclusters k , the six values of λ_q , the two paradigms, the eight clustering methods of Table 3.2, and the four routing solvers of Section 3.3. They define the search space of HAOS rather than a value within it. Also excluded are settings regarding the campaign architecture and experimental design: the core split and asynchronous modes of Section 3.7, the fitted coefficients of the BG-AILS wall-time model (Equation 3.9), and the time limit and seeds that Section 4.1.4 covers.

4.1.4 Replication Design

The campaign runs DRISPI on every one of the 100 XL instances three times, at seeds 11, 22, and 33. The seed drives the weighted HAOS draws, the angular offset,

Table 4.1: Hyperparameters of the Benchmark Configuration.

Parameter	Value	Defined in
<i>HAOS</i>		
Warm-up iterations	10	Section 3.1
Decay γ	0.95	Equation 3.2
Starting weight w_0	10.0	Equation 3.1
Floor $p_{\min}$: k / λ_q , method, solver / paradigm	0.025 / 0.05 / 0.10	Equation 3.3
Immediate and deferred rewards	Table 3.1	Section 3.1
<i>Decomposition and Routing</i>		
Base k arms	{2, 3, 4, 6, 8, 10, 12}	Section 3.1
Maximum number of k arms	10	Section 3.1
Minimum routes per subcluster	8	Section 3.1
Minimum spacing between k arms	2	Section 3.1
Subcluster time budget	$\max(5 \text{ s}, 0.06 \text{ s} \cdot n_c)$	Section 3.3
<i>BG-AILS</i>		
Boundary threshold τ	0.5	Equation 3.7
Small-cluster cap σ / boost α	20 / 0.5	Equation 3.6
Initial ω_0	10	Section 3.4
First-route selection	greedy, unique first	Section 3.4
Chain length	k	Section 3.4
Boundary mask on first local search	on	Section 3.4
Budget floor / margin	60 s / 1.0	Equation 3.8
<i>Route Pool, SC/SP, and Improvement</i>		
Maximum pool size	10,000	Section 3.5
Diversity weight w_d	0.5	Section 3.5
SC/SP trigger interval	20 min	Section 3.7
Minimum coverage for SP	5	Section 3.5
MIP time limit / relative gap	720 s / 0.025 %	Section 3.5
Post-recombination improvement time limit	360 s	Section 3.6

and the random choices inside decomposition. It is also passed to PyVRP, FILO, and FILO2, but AILS-II has no seed flag. This makes the routing phase, BG-AILS and the post-SC/SP improvement pass independent of the seed. This makes the runs non-reproducible. Additionally, the plugged-in solvers run under wall-clock limits, so how far their search gets depends on machine load, and the worker pool schedules subcluster jobs in no fixed order. This affects again, the entire routing phase, BG-AILS, and the post-SC/SP improvement pass alike, so two runs at the same seed can differ in solution quality. Disclosing the seeds therefore does not suffice for exact reproducibility. The three runs per instance are exchangeable replications.

Because each run is stopped after 7,200 seconds (120 minutes) of wall-clock time rather than at convergence, each run resembles a terminating simulation in the sense of Law and Kelton (2000): a fixed initial condition, the first DRI iteration’s decomposition and routing pass (Section 3.2), and a fixed stopping rule rather than a steady state. The replication method of Law and Kelton (2000, ch. 9) therefore applies.

The precision target refers to the fixed set of 100 instances, which are a design factor (Queiroga et al., 2026) rather than a sample from a larger population, so the

seed is the only randomness the design controls. Using $\bar{G}$ the grand mean gap and $\bar{\sigma}^2$ the average per-instance seed variance, we get $\text{Var}(\bar{G}) = \bar{\sigma}^2 / (I \cdot S)$ with $I = 100$. The $I \cdot S$ observations are independent by construction. Every run is a separate process with its own random-number stream, and no pool, cache, or counter is shared across runs (Chapter 3). This interval does not extend to the published baselines of Section 4.2.1, which report one value per instance and no variance.

A pilot campaign of three diverse instances with four seeds each measured a root-mean-square per-instance seed standard deviation of 0.118 pp at the 7,200-second mark. The target half-width was fixed in advance at 0.03 pp, small enough to resolve the 0.02–0.06 pp effects seen in earlier component tests. Table 4.2 forecasts the standard error and the 95% half-width of $\bar{G}$ at the pilot value. Every S from one to four meets the target. This would in fact still hold true at a 99% interval, but precision alone does not decide S .

Table 4.2: Forecast Standard Error and 95% Half-Width of $\bar{G}$ by Seed Count S .

S	SE (pp)	Half-width (pp)
1	0.0118	0.023
2	0.0083	0.016
3	0.0068	0.013
4	0.0059	0.012

Three other considerations set $S = 3$. At $S = 1$ there are no within-instance degrees of freedom and no interval at all. At $S = 2$ a discrepancy between two runs is visible, but cannot be attributed to either of them. At $S = 3$, however, one anomalous run stands out against the other two. We also require $S > 1$ to present a best-of- S column, which is the convention on this instance set (Queiroga et al., 2026).

A fourth seed would add 100 runs and 1,600 core-hours to shrink the half-width from 0.013 to 0.012 pp. That gain is too small next to a second source of error. Two pilot campaigns that differed only in execution environment produced means 0.039 pp apart, three times the $S = 3$ half-width. Such a batch effect shifts every run of a campaign alike, so more seeds do not reduce it, and the budget of a fourth seed would not buy a meaningfully more precise $\bar{G}$ (Barr et al., 1995; Johnson, 2002). Neyman allocation, which assigns seeds in proportion to each instance’s standard deviation (Neyman, 1934; Cochran, 1977), would have further complicated the experimental design while only cutting the standard error of $\bar{G}$ by 8.8% at the same total budget. Therefore, every instance receives three seeds.

The pilot value was conservative. In the campaign, the mean per-instance seed standard deviation is 0.037 pp, about a third of the pilot value (Section 4.2.1). The root-mean-square per-instance seed standard deviation, which enters $\text{Var}(\bar{G})$, is 0.052 pp, which puts the realized 95% half-width of $\bar{G}$ near 0.006 pp. This interval

applies to the mean over seeds only. Best-of- S is an extreme-value statistic whose expectation depends on S , so it is not comparable across studies with different replication counts. Section 4.2.1 uses it descriptively and focuses its main comparison and the answer to our primary research questions on the mean-of- S gap.

4.2 Results

This Section presents the results of the campaign and analyzes the outcomes. In Section 4.2.1, we compare DRISPI’s 300 runs against the published solver benchmarks on the same 100 instances. Section 4.2.2 shows which instance characteristics effect the solution quality. Section 4.2.3 focuses on the secondary research question and analyzes the impact and resulting weight trajectories of HAOS. Section 4.2.4 explores the convergence behavior of DRISPI. Finally, Section 4.2.5 shows from which stages improvements made by DRISPI originate.

4.2.1 Comparison with Monolithic Solvers

This section compares DRISPI against the eight monolithic solvers Queiroga et al. (2026) benchmark on the 100 XL instances: AILS-II, FILO, FILO2, KGLSXXL, HGS-CVRP, SISRs, LKH-3, and OR-Tools. The mean-of- N and best-of- N cost from the original paper reports ($N = 60$ for the published solvers, $N = 3$ for DRISPI) represent the baseline, from which the gap to the current BKS is calculated. The per-instance gap is then averaged over all 100 instances for the main comparison. In a separate comparison, the mean-of- N and best-of- N gaps of each solver are compared on the fifty instances with fewer than 3,400 customers and the fifty with more, using the same split Queiroga et al. (2026) established for their own results table. Queiroga et al. (2026) report their own average gaps against the BKS at the time of publication, effectively AILS-II’s own result for most instances, so those published averages are not comparable to the ones reported here. A detailed instance-level result breakdown of DRISPI’s performance compared to the initial and current BKS columns can be found in Appendix C (Table C.1).

Table 4.3: DRISPI Mean Gap to Current BKS Comparison Against Benchmark Solvers, Sorted by Ascending Mean-of- N Gap Over All Instances.

Solver	Mean-of- N Gap (%)			Best-of- N Gap (%)		
	All	$n < 3,400$	$n > 3,400$	All	$n < 3,400$	$n > 3,400$
AILS-II	+0.159	+0.138	+0.181	+0.089	+0.060	+0.118
FILO2	+0.294	+0.291	+0.296	+0.203	+0.173	+0.233
DRISPI	+0.296	+0.235	+0.358	+0.264	+0.205	+0.323
FILO	+0.334	+0.312	+0.356	+0.237	+0.194	+0.280

Solver	Mean-of- N Gap (%)			Best-of- N Gap (%)		
	All	$n < 3,400$	$n > 3,400$	All	$n < 3,400$	$n > 3,400$
SISRs	+0.583	+0.486	+0.679	+0.425	+0.296	+0.554
KGLSXXL	+1.090	+0.866	+1.314	+0.879	+0.642	+1.116
HGS-CVRP	+1.448	+0.834	+2.062	+1.201	+0.598	+1.804
LKH-3 ($n = 90$)	+6.746	+5.009	+8.918	+5.560	+4.025	+7.479
OR-Tools	+6.902	+6.773	+7.031	+5.628	+5.428	+5.829

The mean-of- N is used for the primary comparison in Table 4.3, because DRISPI's mean-of-3 and each published solver's mean-of-60 estimate the same expected cost of a single run, and differ only in how many draws the average rests on. Best-of- N on the other hand is an extreme-value statistic whose expectation falls as N grows. DRISPI's minimum over three draws stands here against the minimum over sixty for every published solver, a sampling budget twenty times larger, and the comparison rewards the larger budget before it rewards the better algorithm.

The size of that mechanical advantage can be read off DRISPI's own runs. Sampling one of the three seed costs at a time gives an expected best-of-1 gap equal to the mean-of-3 gap, 0.296%. The average of the smaller cost in each of the three retained seed pairs gives a best-of-2 gap at 0.273%. The actual three-seed minimum then gives the best-of-3 gap at 0.264% as reported in Table 4.3. The first extra sample is worth 0.023 percentage points (pp) and the second 0.009 pp, and the sequence is still falling at $N = 3$. AILS-II, FILO2, FILO, SISRs, KGLSXXL, and HGS-CVRP gain 0.07, 0.09, 0.10, 0.16, 0.21, and 0.25 pp respectively between their own mean-of-60 and best-of-60 columns, which are even larger reductions drawn from many more repetitions. DRISPI's mean per-instance seed standard deviation is 0.037 pp, so additional seeds narrow its best column by less than they narrow a higher-variance baseline's. This is what produces the ranks in Table 4.3: DRISPI ranks third by mean-of- N and fourth by best-of- N without any change in the underlying solution quality, because the two columns reward different things. Best-of- N is reported here to match the published solvers' own headline metric, not as the primary comparison.

For the mean-of-three runs to carry that weight, the three runs have to be exchangeable replications. The campaign seeds 11, 22, and 33 produce mean gaps of 0.291%, 0.298%, and 0.300%, an ordering a Friedman test (Demšar, 2006) over the 100 matched triples does not separate from chance ($\chi^2 = 3.61$, $df = 2$, $p = 0.164$). No pairwise Wilcoxon signed-rank test between any two seeds reaches significance either, the smallest p being 0.158 for seeds 11 against 22. The performance order also runs against the iteration throughput: seed 33 completed the most DRI iterations of the three (8,208 against 7,730 for seed 22 and 6,865 for seed 11) and still finished last. The per-instance seed standard deviation has a mean of 0.037 pp and a median

of 0.026 pp. Its root mean square, the quantity that enters $\text{Var}(\bar{G})$, is 0.052 pp, and the realized 95% half-width of the grand mean is 0.006 pp (Section 4.1.4). The three runs are therefore exchangeable at set level: both tests block on instances and rule out a seed acting as a systematic factor across the 100, so the mean over them is a property of the configuration rather than of a particular seed. Neither establishes that the three runs are interchangeable on any individual instance, and three observations at two degrees of freedom could not settle that either way. Instance-level exchangeability follows from the design of Section 4.1.4, where every run is a separate process with its own random-number stream and no pool, cache, or counter is shared. The spread is also not uniform across the set. It runs from 0.0005 pp on XL-n4535-k1134 to 0.222 pp on XL-n6034-k61, and it tracks route length rather than instance size, a pattern Section 4.2.2 takes up with the gap itself.

The means presented in Table 4.3 do also not establish whether an observed difference is real rather than an artifact of the 100 sampled instances. DRISPI's mean-of-3 column is tested against the corresponding mean-of-60 column of five of the eight published solvers, AILS-II, FILO2, FILO, SISRs, and HGS-CVRP, using the paired Wilcoxon signed-rank test (Demšar, 2006), treating the 100 instances as matched pairs. KGLSXXL, LKH-3, and OR-Tools are omitted, because their gaps in Table 4.3 are large enough that the paired comparison would not be informative.

Each baseline is tested twice, in opposite directions, because the two claims the framework has to support are not the same claim. The first family asks whether DRISPI outperforms the other solvers, with H_0 stating that the distribution of paired differences is not shifted below zero. The second asks whether DRISPI is performing worse, with H_0 stating that it is not shifted above zero. Rejecting the first is evidence of an advantage, failing to reject the second is the weaker statement that no disadvantage is detectable. The second of the two is what being competitive with a state-of-the-art monolithic solver means on a fixed instance set. Each family is corrected separately for its five tests with the Holm step-down procedure (Demšar, 2006), and Table 4.4 reports both adjusted p -values side by side. The paired differences are the per-instance mean percentage gaps to the current BKS, and each test uses the normal approximation to the signed-rank statistic without a continuity correction.

Table 4.4 splits the five tested solvers into three groups. DRISPI is significantly better than SISR and HGS-CVRP. The margin is 0.286 pp against SISRs and 1.152 pp against HGS-CVRP, and DRISPI is ahead on 98 and 99 of the 100 instances. Against FILO the mean paired difference is marginal at -0.037 pp and the Hodges–Lehmann shift -0.026 pp, with an uncorrected 95% confidence interval of $[-0.060, -0.001]$, and the uncorrected signed-rank test rejects at $p = 0.043$. The Holm correction over five baselines leaves it at 0.064. On 100 instances with a paired-difference standard deviation of 0.203 pp, the corrected test resolves a shift of about 0.066 pp at 80% power, so a difference of the size observed here sits below what this instance set

Table 4.4: Paired Comparison of DRISPI’s Mean-of-3 Gap Against Each Published Solver’s Mean-of-60 Gap, Over the 100 XL Instances.

Solver	Δ (pp)	Ahead	Holm-corrected p		Verdict
			better	worse	
AILS-II	+0.137	4	1.00	5.2×10^{-17}	worse
FILO2	+0.003	47	1.00	1.00	indistinguishable
FILO	−0.037	55	0.064	1.00	indistinguishable
SISR	−0.286	98	1.0×10^{-17}	1.00	better
HGS-CVRP	−1.152	99	1.0×10^{-17}	1.00	better

Δ is the mean paired difference in percentage points; a negative value favours DRISPI. p -values are one-sided Wilcoxon signed-rank, Holm-corrected within each of the two families of five tests.

can separate. DRISPI is ahead of FILO on the point estimate and on 55 of the 100 instances, and the set is too small to call it. Against FILO2 neither test rejects: the mean paired difference is +0.003 pp, DRISPI is ahead on 47 of 100, and there is no detectable difference in either direction. Against AILS-II the second family rejects decisively. DRISPI’s mean-of-3 is 0.137 pp behind AILS-II’s mean-of-60 and ahead on only 4 of 100 instances. AILS-II found 93 of the 100 initial BKS values (Queiroga et al., 2026) and remains the strongest solver on this set by a clear margin. DRISPI, being level with FILO2, and not separated from FILO by the corrected test, places inside the leading group without instance-specific tuning, which is the claim Chapter 3 sets out to support.

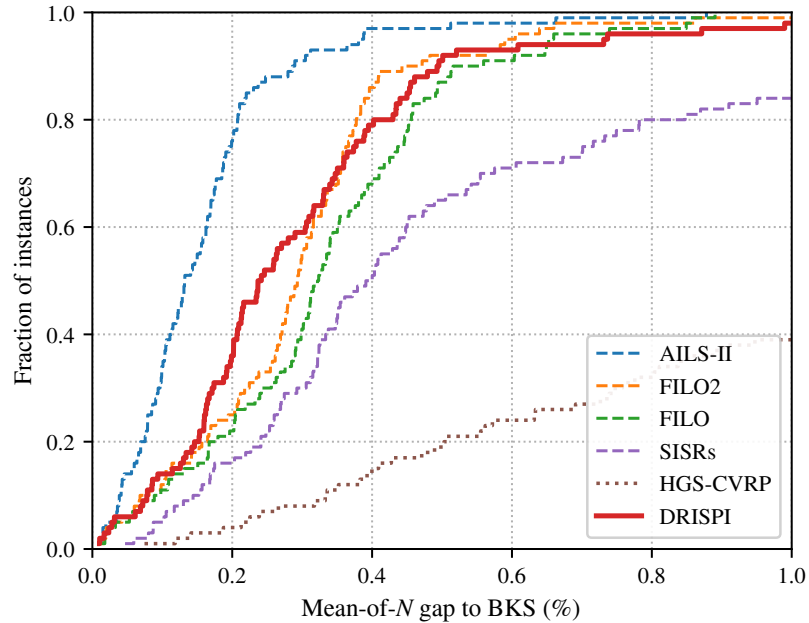
Figure 4.1: Empirical CDF of the Mean-of- N Gap to the Current BKS.

Figure 4.1 shows why the Table 4.3 means understate DRISPI’s typical result and its tail at once. DRISPI’s mean-of-3 gap has a mean of 0.296% against a median of

0.239% (interquartile range 0.162% to 0.374%, ninetieth percentile 0.493%, maximum 1.556%), following a very right-skewed distribution in which most instances sit below the mean, and a short tail sits well above it. DRISPI’s curve tracks above FILO2’s up to a gap of about 0.3% (59% of instances at or below that gap, against 55% for FILO2) and falls below it from 0.4% (79% against 86%), the crossover the non-significant paired test against FILO2 allows: DRISPI wins on the tighter half of the distribution and loses on the wider half. Against FILO, DRISPI’s curve lies above FILO’s through the body of the distribution (59% of instances at a gap of 0.3% or below, against 41% for FILO, and 79% against 69% at 0.4%), the same direction as the mean paired difference, which the Holm correction leaves just short of rejection. FILO moves ahead only past a gap of about 0.66%.

One individual result does not appear in any aggregate. On XL-n3888-k1010, DRISPI’s best-of-3 cost of 1,880,215 beats the initial BKS of 1,880,368 Queiroga et al. (2026) reported at publication. The seed 11 solution that attains this cost is drawn in Figure A.2(a). Its gap to the current BKS is +0.165%. No best-of-3 cost in this campaign matches the current BKS. Across all 100 instances, DRISPI’s best-of-3 cost matches or beats AILS-II’s mean-of-60 on 11 instances, listed in Table 4.5. For XL-n3888-k1010 the reference shown there is the initial BKS, which that run also beats.

Table 4.5: DRISPI’s Best-of-3 Cost Matching or Beating a Published Reference.

Instance	r	DRISPI Best-of-3	Reference Cost	Reference
XL-n1094-k157	7.8	112,441	112,452.6	AILS-II Mean-of-60
XL-n1281-k29	45.5	31,168	31,201.3	AILS-II Mean-of-60
XL-n1328-k19	72.6	38,288	38,317.4	AILS-II Mean-of-60
XL-n1514-k106	14.2	92,468	92,489.7	AILS-II Mean-of-60
XL-n1654-k11	155.4	36,420	36,429.1	AILS-II Mean-of-60
XL-n1794-k163	11.2	141,732	141,777.6	AILS-II Mean-of-60
XL-n1981-k13	153.5	32,638	32,639.1	AILS-II Mean-of-60
XL-n2307-k34	69.6	48,015	48,020.2	AILS-II Mean-of-60
XL-n2727-k546	5.3	431,171	431,173.0	AILS-II Mean-of-60
XL-n3804-k29	134.0	52,924	52,935.8	AILS-II Mean-of-60
XL-n3888-k1010	4.2	1,880,215	1,880,368.0	Initial BKS

Two further results come from earlier campaigns run under the same parameter configuration as Table 4.1 and are reported here because they bound what a single run of this configuration has been observed to reach. On XL-n1094-k157, the one instance whose initial value the challenge has not improved so far, a run tied the current BKS itself. This published campaign here only managed to get within ten units of it, a best-of-3 gap of 0.009%. On XL-n4535-k1134 a run finished below the published initial BKS, whereas this campaign’s best-of-3 sits 0.005% above

it. But neither of the two runs enters any number in this chapter. Given the per-instance seed spread of Section 4.1.4, outcomes of this kind are a property of the run distribution rather than of a particular campaign, and three seeds are too few to find them reliably.

In contrast, Table 4.6 lists the instances behind the tail of Figure 4.1, the largest mean-of-3 gaps in Table C.1, in descending order down to about +0.6 percent. XL-n9571-k55 leads that list. Figure A.2(b) draws its seed 11 solution, the worst run in the campaign, at a gap of +1.628% to the current BKS. The table's +1.56% is the mean of the three seeds. Five of those seven fall in the dataset's ultra-long route length, $r \geq 50$ (Section 4.1.1), which holds only 14 of the 100 instances, and the remaining two both carry the quadrant-dependent demand distribution. Neither concentration is a coincidence, and Section 4.2.2 takes both up directly.

Table 4.6: DRISPI's Largest Mean-of-3 Gap to Current BKS.

Instance	n	Depot	Customers	Demand	r	Mean-of-3 Gap (%)
XL-n9571-k55	9,571	R	R	Q	174.7	+1.56
XL-n6034-k61	6,034	R	C (5)	5-10	98.9	+1.07
XL-n2634-k17	2,634	R	C (4)	1-10	162.6	+0.99
XL-n5174-k55	5,174	C	C (6)	1-10	94.1	+0.87
XL-n8207-k108	8,207	C	C (2)	1-10	75.9	+0.74
XL-n5902-k122	5,902	E	RC (3)	Q	48.8	+0.73
XL-n8028-k294	8,028	C	R	Q	27.3	+0.61

Abbreviations are used as defined in Section 4.1.1.

4.2.2 Performance by Instance Characteristics

The XL instance set varies two candidate notions of scale at once: the customer count n and the target average route length r (Section 4.1.1). The two are not interchangeable stand-ins for the same underlying quantity. r is not derived from n ; it is an independent design factor of the benchmark's own generation scheme, which fixes vehicle capacity Q and fleet size jointly so that a minimum-route solution attains the target r for whatever n the instance happens to have, and the 100-instance design deliberately crosses a wide range of r against a wide range of n rather than letting one determine the other. Their sample correlation on this set is 0.054. For DRISPI specifically the two also act on the pipeline differently: n sets how many customers decomposition has to distribute across k subclusters, while r sets how many stops a single vehicle round trip covers on average and, by construction, compresses the number of routes relative to customers once it grows large.

A linear regression of the mean-of-3 gap on n over all 100 instances gives a slope of 3.12×10^{-5} percentage points per customer ($t = 3.48$, $p = 0.00076$, 95% confidence interval $[1.34 \times 10^{-5}, 4.90 \times 10^{-5}]$). A regression that treats n as the only scale

variable can pick up the long-route instances through it, so the same fit is repeated on the 86 instances with $r < 50$, including only the instances falling below the dataset's own ultra-long route threshold (Section 4.1.1). The slope falls to 1.65×10^{-5} percentage points per customer ($t = 2.62$, $p = 0.011$, 95% confidence interval $[3.96 \times 10^{-6}, 2.90 \times 10^{-5}]$). It stays positive and significant, so the association with n is not carried only by the 14 long-route instances, but it halves once they are removed.

Customer count and route length can be compared directly once each is measured in sample standard deviations from its own mean. One standard deviation is 2,456 customers for n and 40.4 stops for r . On that common scale the slope on n is 0.077 pp per standard deviation ($R^2 = 0.110$) and the slope on r is 0.133 pp per standard deviation ($R^2 = 0.332$). This shows that Route length accounts for about three times as much of the variation in the gap, and its slope per standard deviation is 1.7 times the slope on n . A joint regression on both standardized predictors leaves a partial slope of 0.070 pp per standard deviation for n and 0.130 pp for r , and the route-length partial slope exceeds the customer-count partial slope by 0.060 pp per standard deviation ($t = 2.31$, $p = 0.023$, 95% confidence interval $[0.008, 0.111]$).

Capacity Q and route length are set jointly by the generation scheme, so their ratio produces a quantity neither variable carries on its own. With $k \approx \sum_i q_i / Q$ the minimum fleet and $r \approx n/k$, the average demand per customer satisfies $\bar{q} = \sum_i q_i / n \approx Q/r$. On this set $\bar{q}$ ranges from 0.9 on the unitary-demand instances to 75.7 on the 50–100 instances, and its logarithm correlates with r at only -0.188 , which makes it a third axis rather than a restatement of the first two. On its own, standardized $\log \bar{q}$ has a slope of 0.062 pp per standard deviation ($R^2 = 0.073$).

Adding standardized $\log \bar{q}$ to the joint regression raises R^2 from 0.423 to 0.570 and leaves three significant slopes: 0.147 pp per standard deviation for r ($p = 5.1 \times 10^{-15}$), 0.090 pp for $\log \bar{q}$ ($p = 1.2 \times 10^{-7}$), and 0.069 pp for n ($p = 2.1 \times 10^{-5}$). Route length remains the largest of the three, and customer count the smallest, in every specification we fitted.

Figure 4.2 fits each predictor on its own. The slopes for r and $\log \bar{q}$ are smaller there than in the joint model because the two are mildly negatively correlated (-0.188) while both raise the gap, so each masks part of the other until both are in the model; the customer-count slope moves the other way, from 0.077 to 0.069.

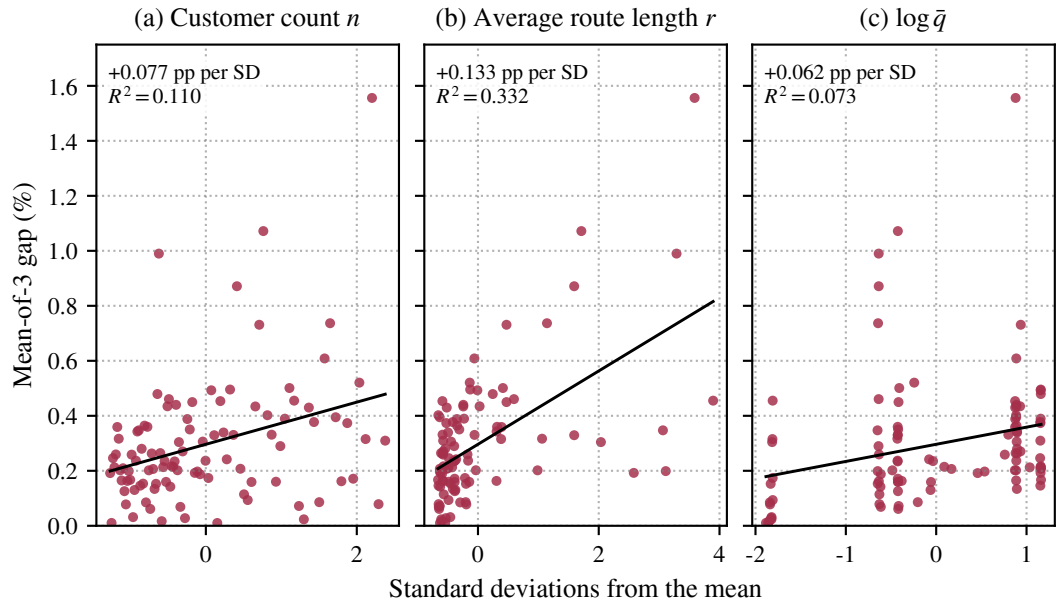


Figure 4.2: Mean-of-3 Gap to the Current BKS Against Customer Count, Average Route Length, and Log Average Demand per Customer.

The same asymmetry appears in a quantity the gap regressions do not measure. Regressing the per-instance seed standard deviation on each standardized predictor in turn gives 0.022 pp per standard deviation for r ($p = 7.2 \times 10^{-11}$, $R^2 = 0.353$), 0.002 pp for n ($p = 0.60$, $R^2 = 0.003$), and a slope indistinguishable from zero for $\log \bar{q}$ ($p = 1.00$). In the joint model only r survives, at 0.022 pp per standard deviation ($p = 4.6 \times 10^{-11}$). Instance size therefore moves the level of the gap without moving its stability, whereas route length moves both. Figure 4.3 shows the two panels side by side. Mean seed standard deviation is 0.093 pp on the 14 instances with $r \geq 50$ against 0.027 pp on the other 86, a factor of 3.4, and the same band carries a mean gap of 0.574% against 0.251% (Mann-Whitney $p = 0.00066$). The long-route instances behave as a separate regime rather than as the upper end of a single trend, which is the reason the scale claim above is stated conditional on r .

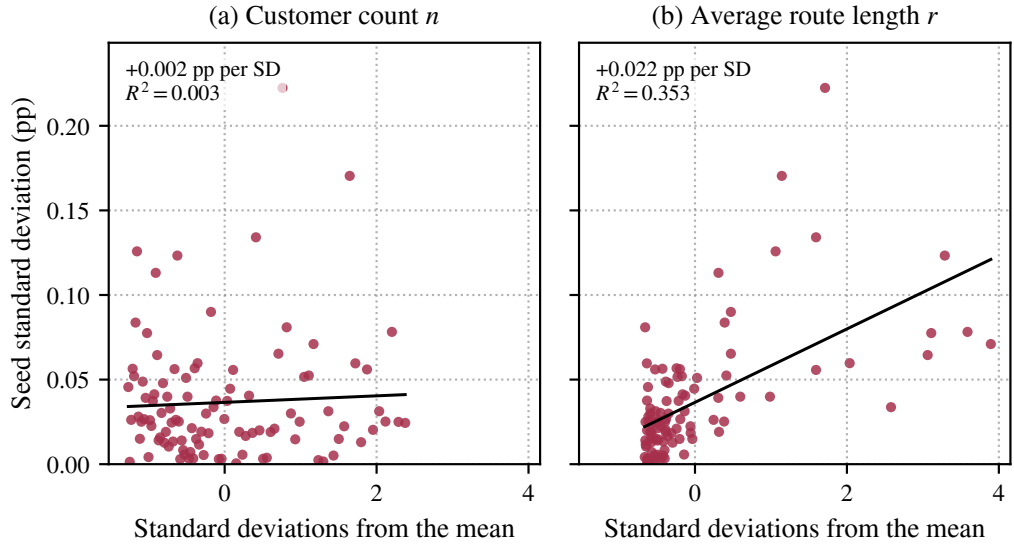


Figure 4.3: Per-Instance Seed Standard Deviation Against Customer Count and Average Route Length.

The concentration of the largest gaps on long routes is intuitive when considering what decomposition does to a long route. Every clustering method in Table 3.2 draws its partition from customer coordinates or route centroids and the dissimilarity metric of Equation 3.4, without reference to which vehicle will eventually need which customers. A long route spans a customer set spread further apart on average than a short route serving a comparable number of stops. And that same spatial spread makes it more likely that some clustering cut line passes through the route rather than around it. Once a long route straddles a cluster boundary this way, the routing stage (Section 3.3) still solves each side of the cut to a good local optimum in isolation, but the seam between them is exactly the kind of defect BG-AILS exists to repair (Section 3.4). BG-AILS repairs every seam in an instance under the same fixed chain length of k cross-reconnects and the same wall-clock budget of Equation 3.8 regardless of how many of those seams cut through a long route, so an instance whose long routes cross more boundaries has more repair work competing for a budget that does not grow to match it. The variance result fits this phenomenon: a repair budget that is sometimes sufficient and sometimes not produces run-to-run spread, which is what the long-route band shows.

These results point at a specific parameter rather than at the stage as a whole. Equation 3.8 adapts the BG-AILS budget according to the wall-clock time of the surrounding decompose-route stage. It tracks through the per-subcluster budgets of Section 3.3 subcluster customer counts, so it scales with the cost of producing the decomposition rather than with the amount of boundary or length of boundary-adjacent routes that the decomposition created. Seizing that from the boundary membership the framework already computes is the change Section 5.3 proposes to improve on this central weakness of DRISPI.

Figure 4.4 analyzes the remaining instance characteristics against the gap: depot position, customer distribution, and demand distribution. Of the three, only demand distribution separates the groups. Kruskal-Wallis tests give $p = 0.153$ for depot position (three levels), $p = 0.180$ for customer distribution (three levels), and $p = 0.00085$ for demand distribution (seven levels). Under Holm correction across the three tests the first two stay well above any conventional threshold (adjusted $p = 0.305$ for both) and the third remains significant (adjusted $p = 0.0025$). Depot position and customer distribution are also non-significant after controlling for n and r (partial F -tests, $p = 0.76$ and $p = 0.55$). Whether the depot sits at the centre, in the corner, or at a random point, and whether customers are scattered, clustered, or both, does therefore not measurably change what DRISPI achieves.

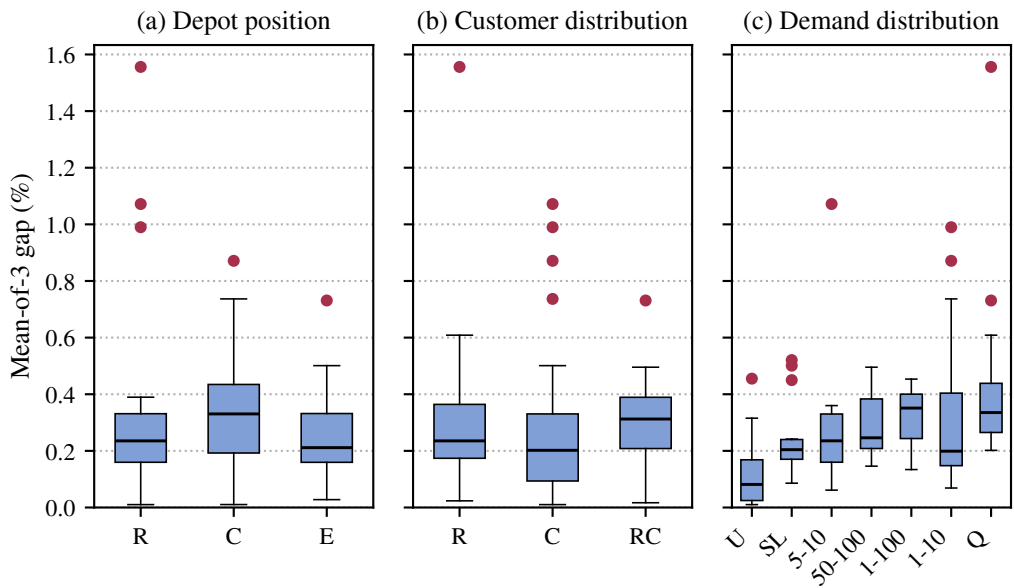


Figure 4.4: Mean-of-3 Gap to the Current BKS by Instance Characteristic.

The demand-distribution effect is real but it is not a seventh independent factor. The group means run from 0.127% on the unitary instances to 0.451% on the quadrant-dependent ones, and the five classes in between (0.250% to 0.337%) do not separate from each other at all (Kruskal-Wallis $p = 0.33$ on those 72 instances alone). What distinguishes the two extremes is $\bar{q}$: unitary demand fixes $\bar{q} = 1$, which means capacity is consumed one customer at a time, and a purely spatial partition is automatically a balanced-load partition. The quadrant-dependent distribution makes demand a function of position, where a partition drawn from coordinates alone cuts along a demand gradient and hands subclusters very unequal loads. Once standardized $\log \bar{q}$ enters the model alongside n and r , the demand-distribution grouping no longer adds anything ($F = 1.51$, $p = 0.18$), and neither does depot position ($p = 0.22$) or customer distribution ($p = 0.98$).

Additionally, each of the seven demand classes holds only 14 or 15 instances, leading to the individual group means in Figure 4.4 carrying the per-instance imprecisions.

sion discussed in Section 5.2 and being reported descriptively. The tests above are run on the full set.

In conclusion, DRISPI's gap to the BKS mixes two things: how hard the instance is for any solver, and what additional costs decomposition adds on top of that. The per-instance mean-of-60 gaps of AILS-II separate them. Fitting the same three standardized predictors to AILS-II's own gap gives 0.091 pp per standard deviation for r ($p = 6.8 \times 10^{-18}$), 0.046 pp for $\log \bar{q}$, and 0.024 pp for n ($p = 0.0064$). Longer routes are therefore hard for the monolithic solver too, and nearly as strongly as they are for DRISPI, while customer count moves AILS-II's gap by about a third of what it moves DRISPI's.

Fitting the same model to the paired difference then isolates what additional cost decomposition imposes. All three slopes are positive and significant, and they are close to one another: 0.055 pp per standard deviation for r ($p = 4.1 \times 10^{-8}$), 0.046 pp for n ($p = 2.3 \times 10^{-6}$), and 0.044 pp for $\log \bar{q}$ ($p = 5.9 \times 10^{-6}$). Route length dominates the raw gap because it is a factor expressing instance difficulty that happens to hit DRISPI slightly harder than it hits AILS-II. Customer count dominates neither, but what little it does to the raw gap is almost entirely a decomposition cost rather than general difficulty. Over the full observed range of the set, 3.6 standard deviations of n and 4.6 of r , the corresponding penalties relative to AILS-II are about 0.17 pp and 0.25 pp. The weak scale dependence claimed for the gap therefore holds in absolute terms and holds less comfortably in relative ones. The size split of Table 4.3 shows the same without a model: DRISPI trails AILS-II by 0.097 pp on the fifty instances below 3,400 customers and by 0.177 pp on the fifty above. Reaching the leading group is therefore better supported at the lower end of the XL range than at the upper.

4.2.3 Learned HAOS Weights

HAOS chooses the number of subclusters k , the demand weight λ_q , the decomposition paradigm, the clustering method, and the subsolver, one draw per DRI iteration, and updates its weights from the rewards of Table 3.1 (Section 3.1). The clustering method is split in two wheels, one per paradigm, so the five decisions are drawn from six decision wheels. Whether those weights carry interpretable information about an instance is the secondary research question of this thesis, and answering it takes two steps. A weight vector that has left the uniform distribution has learned something only if what it learned is the ranking of the arms. A wheel can also leave uniform distribution by reinforcing whichever arm its own early draws happened to favor, which produces the same shape and means nothing. The two are distinguishable, but not from the weight vector alone. To separate them, it is then tested whether the arms that ended up heaviest are the arms that actually paid off. This section establishes the movement first and then tests its direction.

DRISPI provides per-iteration weight snapshots its runs log for all six wheels. Re-computing Equation 3.3 from the logged raw weights reproduces the logged probabilities to a maximum absolute deviation of roughly 10^{-16} across 630,000 arm snapshots, so the weight trajectories below are read rather than reconstructed. The ten warm-up iterations are excluded throughout. Without them, runs complete between 22 and 118 iterations with a median of 78, and 149 of the 300 reach that median, so the later part of every trajectory describes the longer half of the campaign.

Figure 4.5 shows normalized entropy staying high on five of the six wheels. Normalized entropy goes from zero, at absolute singularity, to one, absolute uniformity. Figure 4.5 alone suggests the weights never leave a uniform distribution.

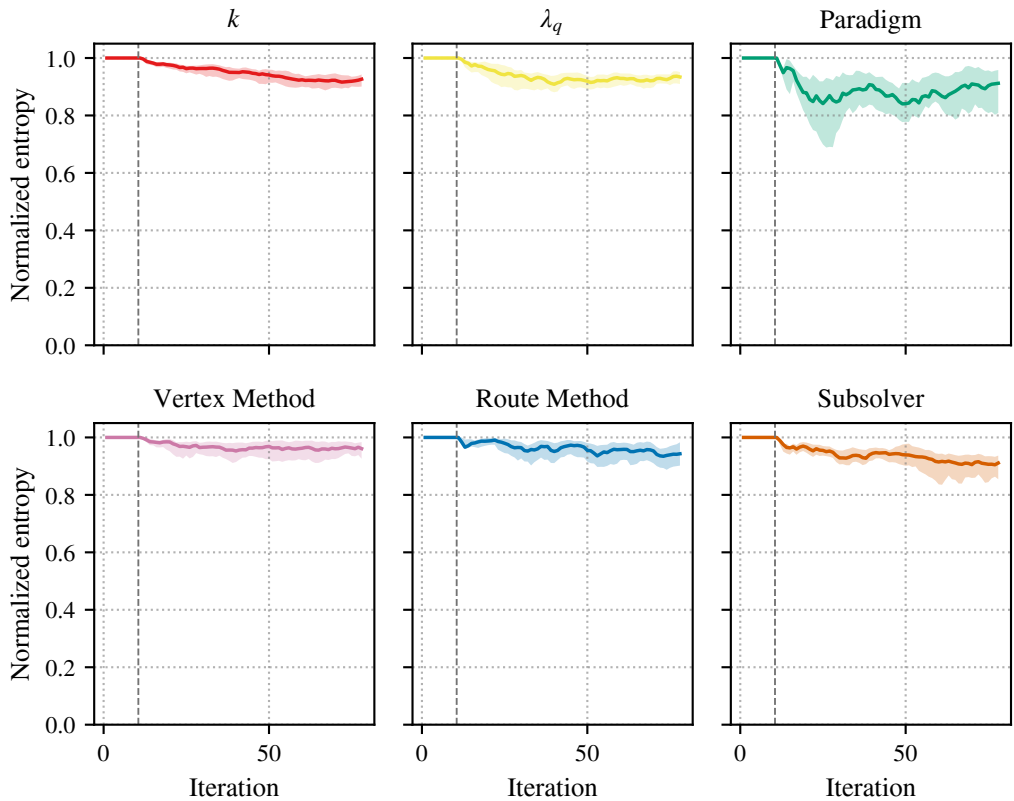


Figure 4.5: Normalized Shannon Entropy of the Six HAOS Wheels Against DRI Iteration.

But the weights do move, and the reward recurrence says how far they should. Credit adds the reward earned on the iterations an arm is drawn. Decay removes a fraction $1 - \gamma$ of its whole weight every iteration. With credit applied before decay (Equations 3.1 and 3.2), an arm drawn with a steady probability p and paid R per selection settles at

$$w^* = \frac{\gamma p R}{1 - \gamma} = 19 p R, \quad (4.2)$$

at the deployed decay factor $\gamma = 0.95$. Below it the arm is pinned to the floor. At a uniform draw on a wheel with A arms, $p = 1/A$, so the weight leaves the floor only

when $R > w_0(1 - \gamma)/(\gamma p) = 0.5263 A$. The arm count is relevant because an arm is paid only on the iterations it is drawn, and a wheel with more arms needs a larger reward per selection to sustain its weight.

The wheels sit well above the decay floor. At iteration 78 the median ratio of the largest weight on a wheel to w_0 is 4.89 for k , 4.65 for λ_q , 9.37 for the paradigm wheel, 2.85 for the vertex-method wheel, 3.36 for the route-method wheel, and 6.87 for the subsolver. Arms only rarely sit exactly on the decay floor, and that fraction does not rise with arm count. Nearly every arm leaves the floor at some point in a run, and once credited it stays above the floor for a mean of 31 to 40 snapshots against a re-selection gap of 1.9 to 7.8 iterations, so an arm is credited again long before it relaxes back.

And the probabilities follow the weights. The median top-arm probability at the last iteration is 0.257 on the k wheel against a uniform $1/7$ of 0.143, 0.323 on λ_q against 0.167, 0.694 on the paradigm wheel against 0.500, 0.295 on the vertex-method wheel against 0.200, 0.481 on the route-method wheel against 0.333, and 0.432 on the subsolver against 0.250. Every wheel ends between 1.4 and 1.9 times uniform on its leading arm.

Since we have established that the arms move, we need to investigate their direction now. Because what the wheels have not done is rank the arms. Within a run, the Spearman correlation between an arm's mean realized reward and that arm's final probability is 0.081 on the k wheel, 0.045 on λ_q , 0.043 on the vertex-method wheel, 0.030 on the route-method wheel, and 0.042 on the subsolver. The arms that finish heavy are not the arms that produces high-quality solutions. Figure 4.6 plots the two against each other.

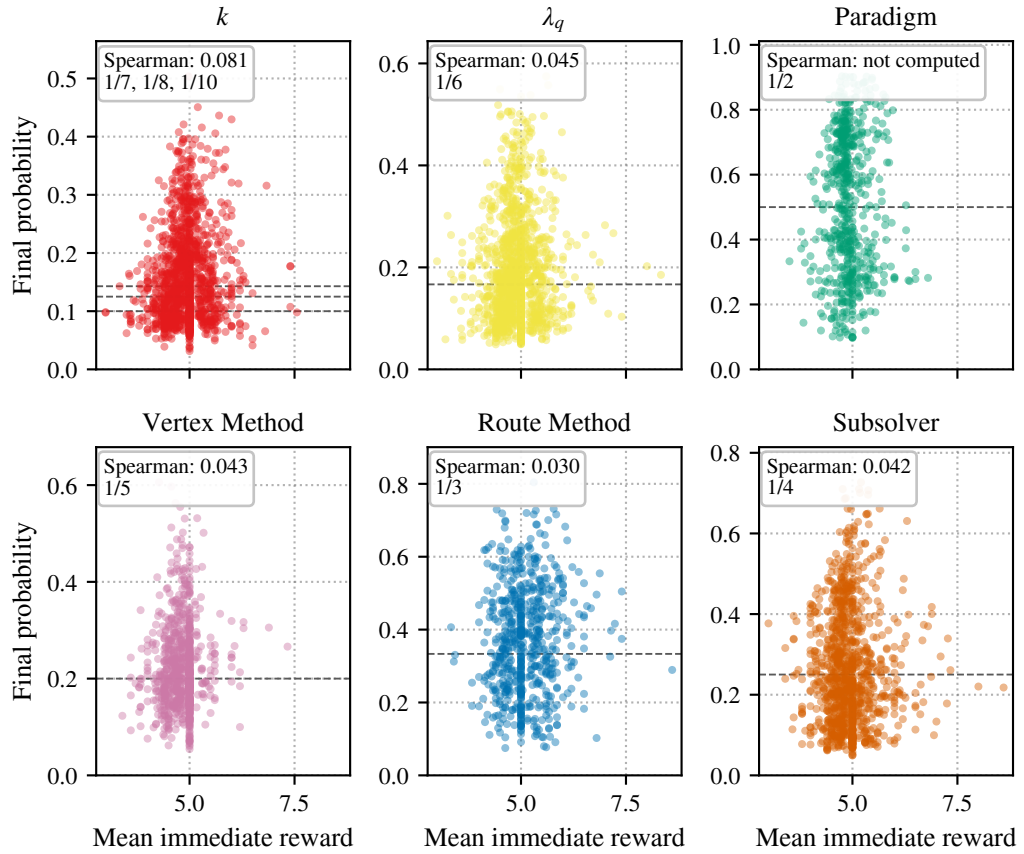


Figure 4.6: Mean Immediate Reward of Each Arm Against Its Final Wheel Probability. Each point represents one arm in one run with at least five draws.

Part of that is a property of the reward set rather than of HAOS. The mean within-run spread between the best and the worst arm is only 0.45 to 1.05 reward points against a median selection reward of 5. On the instances and budget of this campaign the arms are close enough that no wheel has a strong ranking available to learn.

That assumption can be checked on a measure the reward scale does not bound. Scoring every draw by the cost of the solution it produced relative to the incumbent it started from, $100(C_t - c_t^{\text{BG}})/C_t$ and adjusted for position in the run, the pooled best-to-worst arm spread clears the 0.037 pp seed-noise floor on five of the six wheels. That pooled ranking does not survive a change of seed. On the k wheel the best arm is 2, then 6, then 10 across seeds 11, 22 and 33; on λ_q it is 0.2, then 0.8, then 0.6; on the vertex-method wheel fuzzy c-means is the best arm on one seed and the worst on another; on the subsolver AILS-II is the worst arm on seed 11 at -0.305 pp and the best on seed 33 at $+0.028$ pp. Those pooled winners average three sequences that disagree. The route-method wheel is consistent, with agglomerative average best on every seed, but only marginally so, by 0.020 to 0.039 pp against a floor of 0.037. Only a single wheel carries a ranking that is both consistent and large enough to matter. Route-based clustering beats vertex-based on every seed, by 0.071, 0.110 and 0.065 pp, and by more as the run progresses, which is what the mechanism predicts, since route-based clustering partitions the incumbent's own

routes, which are worse earlier in the run.

The instances of one seed are not independent observations of an arm. Within a seed the early draw sequence is nearly identical across instances, so the arm drawn at a given iteration is the same on almost every instance of that seed, and paired tests over instances overstate the precision of the pooled comparison. Agreement across the seeds is the informative check, and it is what the paragraph above uses.

That one wheel is where this section's account can be tested, and it fails in the way the account predicts. The paradigm wheel is the wheel that concentrates, its leading arm reaching a median probability of 0.694; it has the fewest arms, the lowest lift-off threshold, and a real advantage to find. But it does not find it. At the last iteration the leading arm is the vertex-based paradigm on 154 runs and the route-based paradigm on 146. And the median final probability on the route-based paradigm, the better arm, is 0.489. What sets the leader is the seed and not the instance: seed 11 finishes on route in 99 of its 100 runs, seed 22 on vertex in 89, and seed 33 on vertex in 64, while the three seeds of one instance agree on the leader only in 8 instances out of 100. A wheel learning about an instance would agree with itself across seeds. This one commits at roughly seven to three to whichever arm its own early draws happened to favor.

That explains a mostly flat weight vector. It does not explain however why a weight vector that has moved to 1.4 to 1.9 times uniform on an arm chosen at random. Figure 4.7 gives the immediate reward R banked by one selection, summed over the decompose-route candidate and the BG-AILS candidate. Of 19,803 post-warm-up selections, 16,591 bank exactly $R = 5$: the decompose-route candidate scores the no-improvement value of 1 and BG-AILS scores the improvement-over-previous-iteration value of 4. A BG-AILS reward is present on 96.8% of selections. The median R is 5 and the mean 4.94, and with 84% of all selections, almost all receive the identical reward.

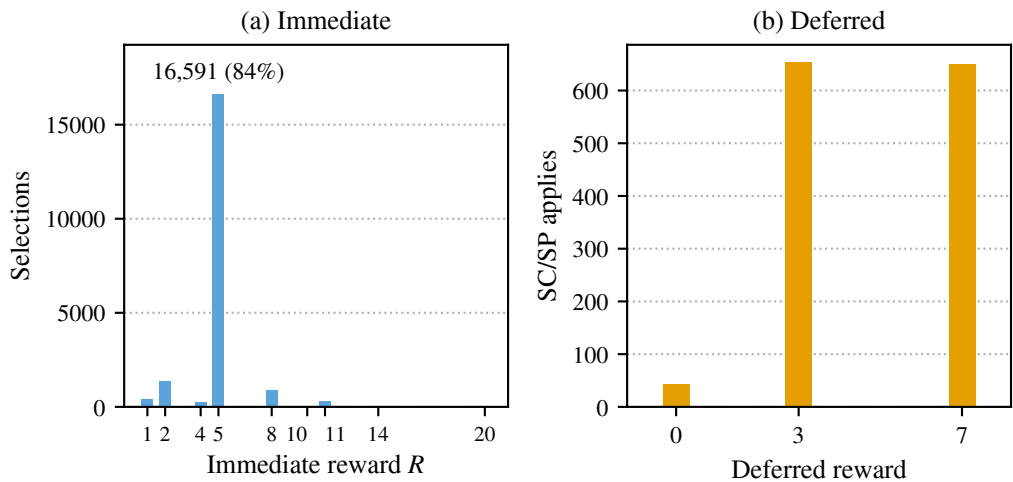


Figure 4.7: Immediate (a) and Deferred (b) HAOS Rewards, Excluding the Ten Warm-Up Iterations.

BG-AILS almost always improves on the previous candidate, and almost never produces a new incumbent best, so nearly every selection lands in the middle bucket. The factor that separates a good configuration from a mediocre one on this benchmark is the size of the improvement, and the reward scale ignores it completely. An improvement of 1% is rewarded the same as an improvement of 0.01%. With $R = 5$, the lift-off condition of Equation 4.2 is met comfortably: between 90% and 98% of selections clear $0.5263 A$ on every wheel except the ten-arm k domain, where the threshold is $0.5263 \times 10 = 5.26$ and only 5.4% clear it. Weights therefore rise on nearly every arm nearly every time it is drawn.

Because of that a missing ranking is turned into a confident-looking, but random weight vector. Furthermore, the smallest reward for any feasible solution in Table 3.1 is 1.0 rather than zero, so an arm accumulates weight just for having been drawn, independently of what followed. But weight sets selection probability through Equation 3.3, selection probability sets the draw rate, and the draw rate sets the rate of accumulation. An arm that is drawn a few extra times early therefore compounds that lead without any regard of its own performance. The result is reinforcement of sampling noise rather than of the outcomes, explaining every observation in this section at once: the entropy curves dip, the leading arm ends above uniform, the three seeds of one instance disagree at a total-variation distance of 0.2 to 0.3 (Figure 4.8), and that disagreement does not fall as runs get longer, because a reinforced random walk diverges rather than converges.

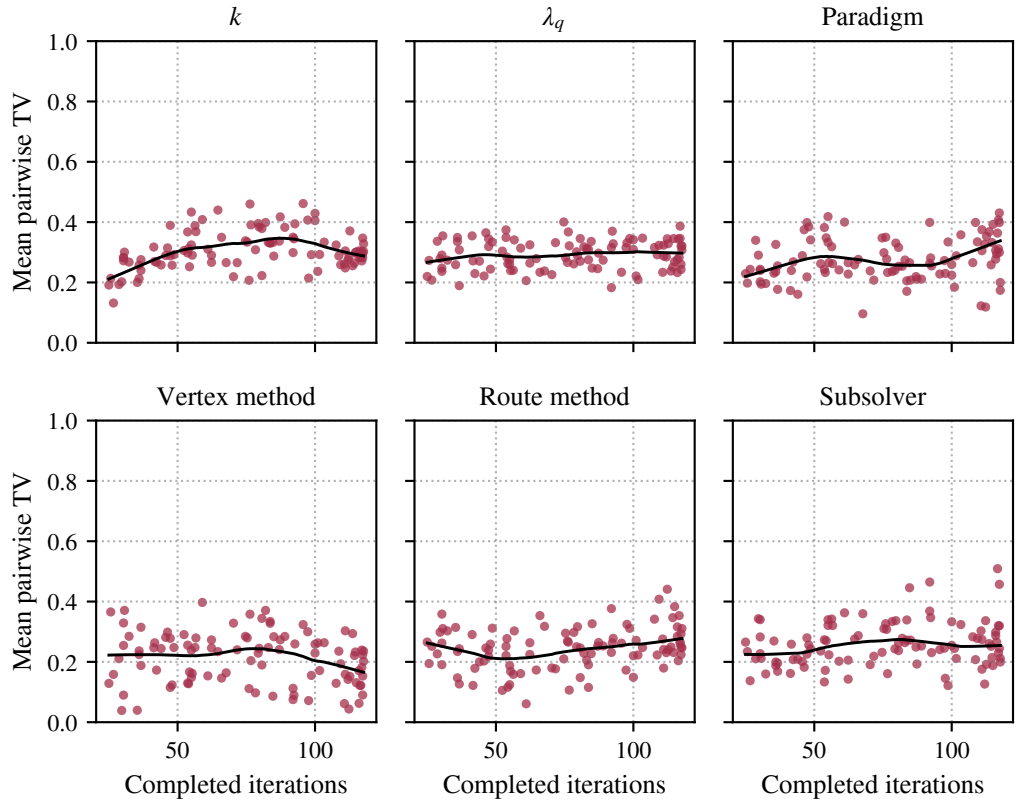


Figure 4.8: Mean Pairwise Total-Variation Distance Between the Three Seeds' Final Wheel Probabilities, Against the Instance's Mean Number of Completed DRI Iterations. The distance is $\frac{1}{2} \sum_a |w_a - w_b|$, averaged over the three seed pairs. The line is a LOWESS smooth.

But the weight level itself is not explained by the immediate reward alone. As shown, the interior fixed point of the recurrence in Equations 3.1 and 3.2, for a stationary selection probability p and a per-selection reward R under credit-then-decay, is $w^* = \gamma p R / (1 - \gamma) = 19 p R$. Evaluated at the run-average probability and the mean immediate reward, that prediction sits below the observed median maximum weight by a factor of 1.70 to 3.01 on every wheel. But two aspects are not considered here.

The first is that p is not stationary. Under the reinforcement above, the leading arm's probability rises over the run, so a run-average p actually understates the probability at which the largest weight was actually accumulated. The second is the deferred channel, which contributes 44.3% of all credit mass on every wheel across the entire run and does not appear in the immediate reward at all. Correcting for both, total credit per selection is $4.94 / (1 - 0.443) = 8.87$, and evaluating $19 p R$ at the median final top-arm probability gives 43.3 for k against an observed 48.9, 54.4 for λ_q against 46.5, 72.8 for the subsolver against 68.7, and 117.0 for the paradigm wheel against 93.7. The divergence between the assumed and observed probability level originates from the weight level once it is given the probability the weight was accumulated at and the full credit stream. The two conditional method wheels need

a further correction for how often their paradigm was drawn and are not reconciled here.

That deferred channel as a mechanism actively works against ranking. The deferred reward is credited once to every distinct HAOS tag among the routes SC/SP selected, at the full value regardless of how many routes that tag contributed (Section 3.5). On an iteration that carries a positive deferred reward, 1.8 to 5.3 arms of the same wheel gain weight at once, against 0.55 to 1.34 on an ordinary iteration, and about 62 weight units are injected against about 5. Nearly half the credit in the system arrives as a broadcast timed to the SC/SP schedule rather than as a judgement about the arm that received it.

The secondary research question asked whether the learned weight distributions give interpretable insight into instance structure and reduce manual tuning across instance classes. On this campaign they do not, and the diagnostics above locate three causes rather than one. First, the reward scale places 84% of selections in a single bucket and discards the improvement magnitude that would separate the arms. Second, the smallest reward is positive, so weight accumulates with the draw count and the roulette draw feeds that back, which converts sampling noise into a stable-looking leader. Third, the deferred channel supplies 44% of the credit as a broadcast to many arms at once.

None of the three is a property of the hierarchy. The wheel mechanics behave exactly as specified, the weights move freely, and the probabilities track them. What the campaign shows is that the credit these wheels are fed does not carry a ranking, and Section 5.3 suggests what would have to change. The consequence for the primary research question is stated in Section 5.2: a controller whose weights are uncorrelated with arm performance is, in effect, close to the uniform draw its predecessor used, so the result of Section 4.2.1 was obtained without working adaptation rather than because of it.

4.2.4 Convergence Behavior

An improvement is any candidate that strictly lowers the incumbent cost, including the first solution of a run. The run logs record 2,335 such updates across the 300 runs. Figure 4.9(a) counts them by the wall-clock quarter in which they were logged, measured from the first timestamp of each run log. Every run's first solution is one of these updates, so naturally, 300 of them fall in the first quarter, and only 1,286 first-quarter updates lower an incumbent that already exists.

Simply counting updates overstates how much of the run is spent converging, because the updates late in a run are individually larger than the ones early in it. Figure 4.9(b) therefore tracks the quantity the rest of this chapter reports. The mean-of-3 gap to the current BKS stands at 0.391% at the 30-minute mark, 0.335% at 60 minutes, 0.311% at 90 minutes, against the 0.296% of Table 4.3 once the 120 minute

mark is reached. The gains per quarter are 0.056, 0.025, and 0.014 pp. While they are falling, they are not falling to zero: the last of the three is still about a quarter of the first, and more than twice the realized 95% half-width of the grand mean.

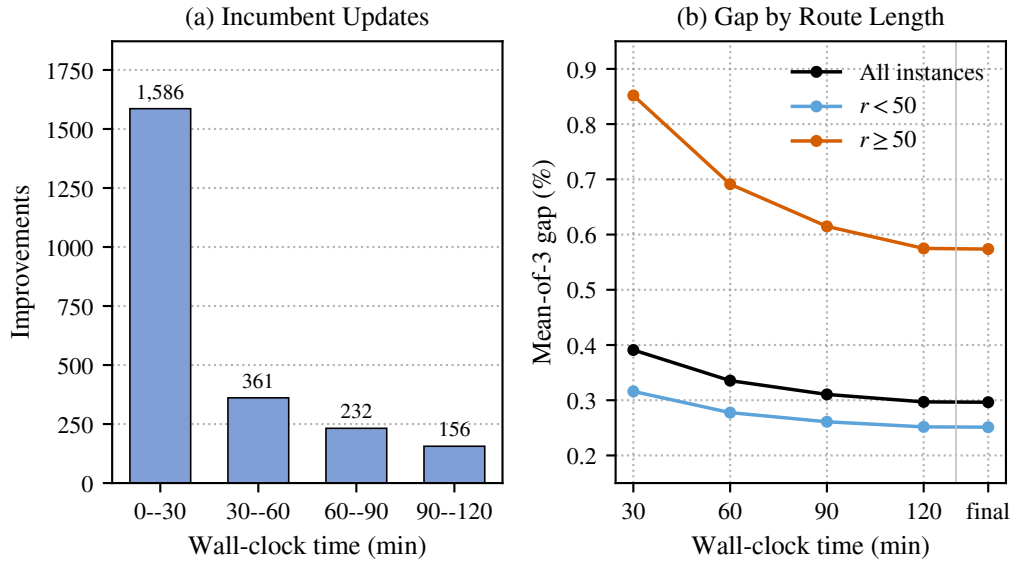


Figure 4.9: Convergence Over the 7,200-Second Budget. (a) Number of Incumbent Updates By Wall-Clock Mark, (b) Mean-of-3 Gap By Route Length to the Current BKS at Each Wall-Clock Mark

The run-level counts say the same thing more directly. Of the 300 runs, 122 improved their incumbent after the 90-minute mark and 221 after the 60-minute mark. On 7 of the 100 instances all three seeds were still improving in the final quarter, and on only 22 there was no seed improving anymore. At the point where the fixed budget of Section 4.1.2 cuts the search off, DRISPI is not at a converged solution on most instances. A longer budget would have produced further gains, and the relevant question is how much. A geometric decay fitted to the observed quarter-on-quarter ratios of 0.45 and 0.54 implies a remaining tail of about 0.013 pp beyond 120 minutes, which would put the set mean near 0.283%. That extrapolation rests on two ratios and is offered as an order of magnitude, not as a result. A long-run campaign to test it was not run.

This extrapolation says roughly how much improvement is left. Which instances are still collecting it is the more useful question, and the answer is not the one the completed iteration counts suggest. Completed DRI iterations fall sharply with instance size, from a mean of 117 on the ten smallest instances to 31 on the ten largest (Spearman $\rho = -0.97$), and the natural reading is that the largest instances are the ones furthest from convergence at the cutoff. The improvement data does not support it. The mean-of-3 gain over the final quarter correlates with route length at $\rho = +0.52$ ($p = 3.9 \times 10^{-8}$) and with customer count at $\rho = +0.06$ ($p = 0.57$). It averages 0.041 pp on the 14 long-route instances against 0.010 pp on the other 86, while the split by size is almost flat: 0.015 pp above 3,400 customers against

0.013 pp below. Figure 4.9(b) shows the two route lengths separately, and the long-route curve is still descending visibly at the cutoff where the short-route curve has flattened.

The two findings are consistent once the binding constraint is stated correctly. Fewer iterations on large instances leave HAOS with fewer draws per arm, but Section 4.2.3 shows that the arms are statistically indistinguishable on all six wheels, so a configuration drawn under a shorter iteration budget is not inherently inferior. The iteration count constrains what the wheels can learn. It does not by itself leave cost on the table. Long routes leave cost on the table for the different reason given in Section 4.2.2: they generate more boundary repair work than the fixed BG-AILS budget covers within one iteration, so the repair continues across iterations and needs more wall-clock time. The instances with the largest gaps are therefore also the instances still improving fastest when the run is stopped, which means part of the long-route penalty reported above is a budget artifact rather than a defect of the decomposition itself. How much of it is which was not tested here.

4.2.5 Contribution by Stage

Sections 4.2.1 to 4.2.4 treat DRISPI as one procedure. It includes five stages, and the route pool tags every route with the stage that produced it, so the improvement each stage contributes can be attributed rather than assumed. The question matters for two reasons. The framework’s contribution over its predecessor DRSCI (Altendeitering and Bachmann, 2026) rests on BG-AILS and HAOS, and the BG-AILS ablation of Section 3.4 measured that stage in isolation on a fixed checkpoint rather than inside a full 7,200-second run. And a stage that carries only few of the total improvements under the campaign budget is also the first candidate for reallocating core time.

The campaign log records 2,335 incumbent improvements. The 300 decompose-route solutions of the first iteration are each run’s opening solution, and both the update counts and the cost reduction below exclude them. Cost reduction is measured from that opening solution to the final incumbent, so the 298 BG-AILS repairs logged on the same iteration are included. The remaining 2,035 updates are the denominator below. Stage time is the sum of the per-stage timers over the whole run, including the first iteration. Those timers run concurrently on the core split of Section 4.1.2, so the time shares below divide the recorded stage time and not the 7,200 second budget.

One attribution is not recoverable as two separate stages. The SC/SP job (Section 3.5) solves the pool and then runs the post-SC/SP improvement pass of Section 3.6 on the result, and the incumbent update is logged only after that pass. Every improvement logged under the name `standard_ails`, the log label of the post-SC/SP improvement pass, therefore owes partial credit to its preceeding SC/SP job.

The two timers are logged separately, and Table 4.7 reports them that way.

Table 4.7: Share of Incumbent Updates, Cost Reduction, and Recorded Stage Time.

Stage	Updates	Cost Reduction	Stage Time
Decompose–Route	16.1%	3.0%	35.1%
BG-AILS	48.8%	88.2%	43.4%
SC/SP	—	—	9.7%
Post-SC/SP Improvement	35.1%	8.8%	11.9%

Summed over all 300 runs. Updates and cost reduction exclude that opening solution. Stage time includes it.

BG-AILS accounts for 88.2% of the cost reduction after the opening decompose–route solution, dropping to 40.8% after the first iteration. The decompose–route stage accounts for 3.0% of the reduction from 16.1% of the updates, and it consumes 35.1% of recorded stage time. The SC/SP plus post-SC/SP improvement accounts for the remaining 8.8% of the reduction, from 35.1% of the updates. Since all stages of DRISPI are highly intertwined and cooperative, a higher cost reduction for one step does not imply that the other steps are less valuable.

Table 4.8: The Same Shares Split by Route Length and by the $n = 3,400$ Cut.

Subset	BG-AILS (%)			SC/SP + Improvement (%)		
	Reduction	Updates	Time	Reduction	Updates	Time
$r < 50$	87.7	50.9	43.0	9.2	34.0	22.5
$r \geq 50$	92.1	39.1	46.4	5.8	40.2	15.2
$n < 3,400$	91.5	50.9	45.2	7.4	32.3	24.2
$n > 3,400$	86.3	46.5	41.6	9.6	38.3	18.7

Table 4.8 is where the two predictions from Section 4.2.2 can be checked. BG-AILS does carry a larger share of the cost reduction on the long-route instances, 92.1% against 87.7%, and a larger share of recorded stage time, 46.4% against 43.0%. Its share of the updates is lower on that length, 39.1% against 50.9%, because SC/SP plus post-SC/SP improvement logs 40.2% of the updates there against 34.0% on the short-route band. The prediction about the SC/SP contribution and the 10,000-column pool cap (Section 3.5) does not hold in the form the size split suggests. The cap is reached on 72 of the 100 instances: 33 of the 50 with $n < 3,400$ and 39 of the 50 with $n > 3,400$, and on none of the 14 long-route instances, whose largest observed pools have a median of about 4,200 columns. The SC/SP plus post-SC/SP improvement time share falls from 24.2% below 3,400 customers to 18.7% above, and from 22.5% on the short-route band to 15.2% on the long-route band. The SC/SP timer inside that sum falls from 12.1% to 7.1% across the size cut, and from

11.0% to 1.0% on the long routes, which is the band whose pools stay below the cap. The SC/SP plus post-SC/SP improvement share of the cost reduction is 7.4% below 3,400 customers and 9.6% above, and 9.2% on the short-route band against 5.8% on the long-route band. The long-route drop in SC/SP time is real because long-route instances generate fewer total routs, and smaller route pools take less time to solve.

5 Conclusion

This thesis asked two questions. The primary one was whether an algorithm-agnostic decomposition framework that learns its decomposition and routing choices during a run can match state-of-the-art monolithic solvers on very large CVRP instances. The secondary one was whether the weights that learning produces give interpretable insight into instance structure and reduce manual tuning across instance classes. The campaign of Chapter 4 answers the first affirmatively under a fixed wall-clock budget and the second negatively. The framework reaches the leading group without the learning having worked. Section 5.1 highlights both answers and what the campaign shows about which parts of the framework do the work. Section 5.2 separates what limits the validity of those measurements from what the study did not attempt, and Section 5.3 outlines extensions and future paths for research.

5.1 Findings

DRISPI reaches a mean-of-3 gap of 0.296% to the current BKS and 0.211% to the initial BKS across the 100 XL instances, which places it third of the nine solvers in Table 4.3 by mean-of- N and well inside the leading group. Against FILO2 the paired difference is +0.003 pp and neither one-sided test rejects. Against FILO it is -0.037 pp in DRISPI’s favour, on 55 of 100 instances, which the uncorrected signed-rank test separates at $p = 0.043$ and the Holm correction does not (Demšar, 2006) (Table 4.4). Across 100 instances with a paired-difference standard deviation of 0.203 pp the corrected test resolves a shift of about 0.066 pp, not enough to statistically separate the two. Against SISR, KGLSXXL, HGS-CVRP, LKH-3, and OR-Tools the margin is decisive. Against AILS-II, which found 93 of the 100 initial BKS values (Queiroga et al., 2026), DRISPI is 0.137 pp behind and beats it only on 4 instances. One best-of-3 cost, on XL-n3888-k1010, falls below the published initial BKS, and on 11 instances a best-of-3 cost matches or beats AILS-II’s mean-of-60 (Table 4.5).

That comparison holds under a fixed wall-clock budget and not under equal resources. Each DRISPI run used eight pinned cores for 7,200 seconds, 16 core-hours against the two CPU-hours of each published run, at most 12.8 after the PassMark scaling of Section 5.2.1. Parallelism is the premise of decomposition rather than an accident of the setup, and the monolithic baselines are single-threaded by construction, so a comparison at equal wall-clock is fair. It is still not a statement about equal

compute, and Section 5.2.1 gives the full accounting. The defensible claim is that this decomposition framework is competitive with the leading monolithic solvers at this scale within a practical time budget, while using only one configuration for all 100 instances and no per-instance tuning.

The secondary question has to be answered negatively. The HAOS wheels do leave the uniform distribution. Their leading arm ends between 1.4 and 1.9 times $1/A$ on every level. What they do not do is rank the arms according to their performance. Within a run, the correlation between an arm’s realized reward and its final probability is between 0.03 and 0.08 (Section 4.2.3), so the arms that finish heavy are not the arms that paid off. Three causes account for it: First, 84% of selections bank the identical reward, because the three-level reward scale of HAOS (Table 3.1) discards the magnitude of improvement that separates configurations. Second, the smallest achievable reward is positive, so weight accumulates with the draw count and ends up reinforcing the arms with higher draw counts. Third, the deferred reward channel through SC/SP supplies an inflated 44% of all credit mass as a broadcast to many arms at once. The learned weights therefore carry no insight into instance structure on this campaign. This also negates the possibility of offline warm-starting the framework with a prior fitted to instance characteristics as of now.

Two measurements identify what carries the framework’s performance. The controlled comparison of Section 3.4 isolates the improvement stage on identical input: BG-AILS beats a stock AILS-II pass by 0.056 pp of cost improvement (Hodges–Lehmann, 95% confidence interval [0.049, 0.063], ahead on 85 of 100 instances) and a blind cross-reconnect control by 0.058 pp ([0.052, 0.065], ahead on 89 of 100), while that control is itself indistinguishable from the stock pass (−0.004 pp, [−0.011, 0.002]). That shows that isolated blind perturbation of the concatenated solution buys nothing. It is the guidance of the perturbation towards the boundaries that pays. Under the full campaign budget, with the opening decompose–route solution excluded, Section 4.2.5 credits BG-AILS with 88.2% of the relative cost reduction, from 48.8% of the incumbent updates and 43.4% of recorded stage time. The SC/SP plus post-SC/SP improvement accounts for another 8.8% of that reduction, and decompose–route for 3.0%.

The remaining gap is not principally a matter of instance size. Three continuous quantities account for 57% of its variation across the set, and route length is the largest of them at 0.147 pp per standard deviation against 0.090 pp for the implied average demand per customer and only 0.069 pp for the customer count. Route length alone drives the run-to-run spread. Customer count and demand per customer do not move it at all. Depot position and customer distribution are not significant before or after controlling. When setting DRISPI’s per-instance gaps against AILS-II’s, we can separate instance difficulty from what our decomposition approach costs: long routes are hard for the monolithic solver as well, and nearly as strongly, while the customer count moves AILS-II’s gap by about a third of what it

moves DRISPI's (Section 4.2.2). The scale dependence the primary question asks about is therefore weak in absolute terms, but less comfortably so relative to the strongest baseline.

Chapter 1 sets out four contributions, and the campaign bears on them unevenly. The framework itself, the boundary-guided improvement stage, and the parallel pipeline hold as stated: DRISPI reaches the leading monolithic group on the XL instance set with one configuration and no per-instance tuning, Section 3.4 isolates BG-AILS against both a stock improvement pass and a blind perturbation control, and the eight-core pipeline delivered the 300 runs the study rests on. The second contribution does not hold. HAOS was introduced as a controller that learns its configuration during a run, and on this campaign it does not learn. Its wheels concentrate, reaching a median probability of 0.694 on their leading arm, but the arm they concentrate on is uncorrelated with the arm that produced the best solutions. And on the one wheel where a consistent advantage exists the controller finishes on the better arm in 146 of 300 runs against the 150 that chance would give. The additive reward, the global decay and the six-wheel hierarchy are built and documented as designed, but the layer they compose does not do what it was intended to do. The campaign therefore effectively ran on a random controller.

Two things take its place as contributions made by this thesis. The first is an account of why the HAOS controller fails, with three causes separated and measured rather than conjectured. Each cause points at a specific change, and Section 5.3.1 sets them out. The second is the measurement apparatus the campaign required in order to establish any of this. The paired testing framework of Section 4.2.1 with its resolution limit stated, the separation of instance difficulty from decomposition cost in Section 4.2.2, and the arm-quality comparison of Section 4.2.3 scored in realized cost rather than in the controller's own reward. None of the three is specific to DRISPI. Each applies to any decomposition framework benchmarked against a strong monolithic reference.

What the framework costs is CPU time and an unfinished search. The eight cores do not deliver eight times the search of one, for the reasons Section 5.2.1 sets out, and at the 7,200-second cutoff 122 of the 300 runs were still improving their incumbent (Section 4.2.4). DRISPI is not reported at convergence. It is reported at the point where a budget matched to the published solvers' own per-run budget stopped it.

5.2 Limitations

The limitations fall into two kinds. Section 5.2.1 covers what constrains how far the numbers of Chapter 4 can be pushed. Section 5.2.2 covers what the study did not attempt. For each entry, Section 5.3 presents a respective outlook for future research.

5.2.1 Validity of the Comparison

This study compares runs that differ in hardware, parallelism, and total budget, and none of its gaps correct for these differences. Queiroga et al. (2026) ran every published solver on a single thread of a machine with two AMD EPYC 9654 processors (Zen 4, 2.40 GHz), 60 seeds of two hours per instance. DRISPI ran three seeds of two hours per instance on eight pinned cores of two Intel Xeon Platinum 8280L processors (Cascade Lake, 2.70 GHz; Section 4.1.2). Both machines were shared: the published runs executed as up to 50 parallel jobs on one machine, DRISPI as six concurrent containers, three per socket, pinned to physical cores.

Accorsi et al. (2022b) recommend bringing running times from different processors to a common scale with the PassMark single-thread rating before comparing them. The EPYC 9654 rates 2,898⁹ and the Platinum 8280L 2,316¹⁰, a factor of 1.25. That factor functions as a lower bound, since the 8280L rating rests on few samples and is measured at the 4.0 GHz single-core turbo, which the chip cannot hold with 24 of its 28 cores busy. Scaled this way, DRISPI's 7,200 seconds per run correspond to at most 5,800 seconds on the published solvers' processor, per core in wall-clock time. This study, however, reports unscaled gaps, and DRISPI was not rerun with a longer limit to offset the slower hardware.

The budget asymmetry points in opposite directions for the two columns of Table 4.3. Per run, DRISPI used 16 core-hours (at most 12.8 after PassMark scaling) against two CPU-hours for each published run, so the mean-of- N column compares runs in which DRISPI had eight times the cores, on older hardware. Effectively, DRISPI used the same wall-clock time, but more core-hours per run. But the eight cores do not deliver eight times the search. Subcluster routing runs in waves that leave DRI cores idle whenever the last wave holds fewer than six subclusters or when the main loop blocks while it waits for BG-AILS (Figure 3.4). Furthermore, three containers have to share the memory bandwidth and last-level cache of each socket. And per instance, the core-hour relation reverses. The published best-of-60 baselines are based on 120 CPU-hours and 120 hours of wall-clock time, while DRISPI's best-of-3 is based on 48 core-hours and six hours of wall-clock time (at most 38.4 and 4.8 respectively, after PassMarkscaling). The minimum of 60 runs from a distribution lies systematically below the minimum of three, so the best-of- N column favors the published solvers by construction. Neither column is an equal-resource ranking.

$S = 3$ was sized for the grand mean $\bar{G}$ (Section 4.1.4) and supports claims at that level only. The forecast 95% half-width of $\bar{G}$ is 0.013 pp. For a single instance, the mean of three seeds at the pilot standard deviation of 0.118 pp has a standard

⁹<https://www.cpubenchmark.net/cpu.php?cpu=AMD+EPYC+9654&id=5088>, accessed September 22, 2026.

¹⁰<https://www.cpubenchmark.net/cpu.php?cpu=Intel+Xeon+Platinum+8280L+@+2.70GHz&id=6729>, accessed September 22, 2026.

error of 0.068 pp and, with two degrees of freedom ($t_{0.975,2} = 4.30$), a 95% half-width of about 0.29 pp. That is more than twenty times wider, and wider still on the high-variance instances. The per-instance results in Section 4.2 (the largest gaps in Table 4.6, the instances where DRISPI’s best-of-3 matches or beats a published result in Table 4.5, and the seven demand classes of 14 or 15 instances each in Figure 4.4) are therefore descriptive. Across 100 instances, some differences of that size will arise from the seed alone.

The seed term is also not the largest source of error. Two pilot campaigns that differed only in execution environment showed a batch effect of about 0.039 pp, three times the $S = 3$ half-width, and more seeds would not have reduced it. Shared-server load and Docker pinning are part of the measurement rather than a nuisance the design controlled.

5.2.2 Scope of the Study

The 7,200-second limit matches the published per-run budget for a fair wall-clock comparison. This, however, is not the most favorable run length for DRISPI. At termination, 122 of the 300 runs were still improving their incumbent and 221 had improved it within the final hour (Section 4.2.4). The residual tracks route length, with a Spearman correlation of $+0.52$ ($p = 3.9 \times 10^{-8}$) against $+0.06$ for the customer count ($p = 0.57$), so the instances carrying the largest gaps are also the instances furthest from finishing. How much of the long-route penalty reported in Section 4.2.2 is a defect of the decomposition and how much is a budget artifact was not separated, and no campaign at a longer wall-clock limit was run.

The iteration budget bounds how many draws each HAOS arm receives, and that bound is tightest on the largest instances: completed iterations fall from about 117 on the ten smallest to about 31 on the ten largest (Section 4.2.4). A longer budget would not have repaired the weights. Section 4.2.3 finds the arms statistically indistinguishable on this campaign and the arms finishing heavy uncorrelated with the arms that paid off, so additional draws would sharpen an arbitrary leader rather than correct it. What the budget limits is the evidence available to a controller that was not, on these runs, in a position to use it.

A control arm that freezes the draw at one configuration for the whole run under the same budget was also not run. The 300-run campaign behind Section 4.2.1 used the server allocation available for this thesis, and a controlled comparison would have required a full campaign alongside it. A subset carved out of the existing allocation would confound the two. Altendeitering and Bachmann (2026) leave the same gap for DRSCI, a DRISPI predecessor that draws paradigm, k , and solver uniformly at random each iteration without learning. The weight diagnostics of Section 4.2.3 narrow what that missing arm would have resolved. The wheels end between 1.4 and 1.9 times uniform on their leading arm and the leading arm is uncorrelated

with realized reward, so the configurations this campaign drew are close to a uniform draw with an arbitrary bias. The result of Section 4.2.1 was obtained without working adaptation rather than because of it, which makes that result robust to the controller and removes online learning from what it demonstrates. For future runs, this campaign could therefore serve as control arm.

Two further boundaries apply to the mechanism results. The BG-AILS comparison of Section 3.4 runs on one DRI iteration from a checkpoint under one pinned decomposition configuration, so it establishes that boundary guidance pays for that stage without establishing what the routes are worth once the pool recombines them. The attribution of Section 4.2.5 covers the full campaign and partly closes that gap, but it cannot separate the SC/SP solve from the post-SC/SP improvement pass that adopts its result, because the incumbent update is logged only after the pass. Finally, every claim in this thesis is conditioned on the fixed XL set of Queiroga et al. (2026). No instance outside it, and no additional CVRP variant, was tested.

5.3 Future Research

The framework of this thesis is assembled almost entirely from parts it did not invent. The clustering methods of Section 3.2, the routing subsolvers of Section 3.3 and the SP recombination of Section 3.5 are existing techniques placed behind fixed interfaces. And also the central part of the pipeline built here that demonstrably performs, BG-AILS, is a targeted modification of a published local search. That composition reaches the leading monolithic group on the XL set with a single configuration. The solvers it sits among are the product of sustained engineering inside one search process, specific to the constraints they were written for. The general takeaway is that at this scale the framework architecture buys much of what that engineering buys, and buys it in form of flexibility and independence: every future improvement to a subsolver, a clustering method or a recombination model raises the framework's floor without a redesign, and decomposition admits parallel execution by construction where a single search process does not.

The second reading matters more for anyone building on this. The result of Section 4.2.1 was obtained while the adaptive layer was not working. Configurations were drawn close to uniform with an arbitrary bias, so the reported gap is what the architecture achieves with no configuration learning at all, and it is a floor for that architecture rather than a ceiling. Section 4.2.3 shows the remaining margin is not hypothetical, but real. Route-based clustering for example outperforms vertex-based by about a tenth of a percentage point of realized improvement per draw on every seed, and the controller finishes on it in 146 of 300 runs. A learning controller could realize advantages like this and translate them into a further reduction of the gap.

A last observation is independent of the framework. Instance size is the axis along which large-scale CVRP benchmarks are usually organized, and on this set it is the weakest of the three continuous quantities tested: route length moves the gap by 0.147 pp per standard deviation against 0.069 pp for the customer count, it alone moves the run-to-run spread, and it moves AILS-II's gap nearly as strongly (Section 4.2.2). Difficulty at this scale sits in how long the routes are rather than in how many customers there are. If that holds on other instance families, it argues for designing and reporting against route length rather than against n . The subsections below take up what extensions DRISPI in particular would need to further improve.

5.3.1 Repairing the Controller

Section 4.2.3 locates three reasons the HAOS weights carry no ranking, and each suggests a change that can be tested without altering the remainder of the framework.

The reward of Table 3.1 should be credited relative to a baseline rather than absolutely (Fialho et al., 2010). Crediting an arm with $R - \bar{R}$, the running mean reward of recent draws on that wheel, leaves an average arm at zero net credit and moves a good arm up and a weak one down. That removes the net credit an arm currently earns just for having been drawn, which is what converts sampling noise into a stable leader, and it requires no new signal.

The reward also needs resolution. Three levels place 84% of selections in one bucket, because BG-AILS nearly always improves on the previous candidate and nearly never sets a new best. Scoring the normalized improvement $(c_{\text{prev}} - c_{\text{new}})/c_{\text{prev}}$, or an arm's rank within a sliding window of recent outcomes, keeps the magnitude that currently separates configurations in mind, that is currently discarded.

The deferred channel should be normalized by the contribution it rewards. Paying every distinct tag among the SC/SP-selected routes the full value credits a configuration that supplied one route exactly as much as one that supplied twenty, and it supplies 44% of all credit mass. Dividing by the number of routes a tag contributed, or holding the channel out until the first two changes are in, would stop nearly half the credit arriving as a broadcast.

The paradigm wheel is the natural test bed for these changes. It has two arms, the lowest lift-off threshold of any wheel, and a consistent advantage of about a tenth of a percentage point for route-based clustering on every seed. A repaired controller should finish on that arm on substantially more than the 146 of 300 runs the deployed one manages, and a single-wheel rerun is enough to check it.

The current lift-off condition also couples the scale of learning to the arm count. An arm is credited only on the fraction of iterations it is drawn, so a wheel with A arms needs a reward of $w_0(1 - \gamma)A/\gamma$ per selection to hold weight above the floor,

from 1.05 on the two-arm paradigm wheel to 5.26 on a ten-arm k domain. Baseline subtraction does not remove that dependence, since it survives in p_a . Crediting R/p_a at the arm's realized probability goes further and equalizes the threshold as the wheel drifts, which would also stop an arm accumulating faster merely because it is already being drawn more often.

Credit assignment across the five levels remains open after all three. Every arm of a drawn configuration currently receives the full reward, so each wheel's signal carries the other four wheels' noise. With a baseline-subtracted, resolved reward this becomes a variance problem rather than an absence of signal, and the standard remedies apply: holding levels fixed in turn, or estimating a per-level advantage.

Only with a reward that separates the arms does offline warm-starting become possible. A prior fitted to instance characteristics would need weight vectors that are stable across seeds, and this campaign does not produce them. The three seeds of one instance end at a total-variation distance of 0.2 to 0.3 and do not converge as runs lengthen. The prerequisite for that line of work is a controller whose weights track arm performance.

5.3.2 Untested Comparisons

Two campaigns would settle what Section 5.2.2 leaves open, and none of them requires a change to the framework. A campaign at a longer wall-clock limit, four hours or more, would separate the long-route penalty into the part decomposition causes and the part the budget causes, which Section 4.2.4 can currently only bound. An equal-compute comparison, running DRISPI on one core against the published single-thread baselines or the baselines on eight, would replace the wall-clock-matched ranking of Table 4.3 with a resource-matched one. The last is the most informative and the most expensive.

5.3.3 The Long-Route Regime

Section 4.2.2 identifies route length as the largest driver of both the gap and its run-to-run spread, and attributes it to a fixed BG-AILS budget meeting a quantity of boundary repair work that grows with how many boundaries cut through long routes. That suggests a budget for the stage that scales with the realized boundary work of the current decomposition rather than with the wall-clock time of the surrounding decompose-route stage (Equation 3.8), and a chain length that does the same. A second direction is to make the decomposition itself aware of the incumbent's route structure on these instances, which the route-based paradigm does only partially, since it partitions routes without regard to their length.

5.3.4 Transfer to Other Problem Classes

DRISPI is agnostic about what solves a subcluster and about what partitions the customers. Every clustering method in Table 3.2 and every routing solver of Section 3.3 sits behind a fixed interface, and the set-partitioning recombination of Section 3.5 operates on routes rather than on anything CVRP-specific. Extending the framework to a different VRP variant therefore needs a subsolver that handles the variant and a feasibility check on the recombination model, rather than a redesign of the pipeline. The predecessor of this framework was built for a VRPTW (Kerscher and Minner, 2025) and adapted to the CVRP, which is evidence in the same direction. A monolithic solver offers no comparable adaptability since its scaling comes from engineering inside one search process, and that engineering is specific to the constraints it was written for. Whether the performance reported here survives that transfer was not tested, and the boundary-repair mechanism of Section 3.4 is the part most likely to need rework, since what counts as a boundary depends on which constraints the decomposition cut through.

Bibliography

- Accorsi, L., F. Cavaliere, D. Vigo. 2022a. Iterative fast optimization for the Capacitated Vehicle Routing Problem. URL <https://drive.google.com/file/d/1Ej02w-lnPdaTmtitNL4SS9xdfMRieyQZ/view>. Unpublished short concept paper. Publicly linked from the 12th DIMACS Implementation Challenge on Vehicle Routing Problems.
- Accorsi, L., A. Lodi, D. Vigo. 2022b. Guidelines for the computational testing of machine learning approaches to vehicle routing problems. *Operations Research Letters*, 50 (2), 229-234, doi: 10.1016/j.orl.2022.01.018. URL <https://arxiv.org/abs/2109.13983>.
- Accorsi, L., D. Vigo. 2021. A fast and scalable heuristic for the solution of large-scale capacitated vehicle routing problems. *Transportation Science*, 55 (4), 832-856, doi: 10.1287/trsc.2021.1059.
- Accorsi, L., D. Vigo. 2024. Routing one million customers in a handful of minutes. *Computers & Operations Research*, 164 106562, doi: 10.1016/j.cor.2024.106562.
- Altendeitering, L., R. Bachmann. 2026. DRSCI: An algorithmic framework for very large-scale capacitated vehicle routing problems. Unpublished project study, Technical University of Munich. Chair of Logistics and Supply Chain Management; supervisor N. Kuttruff.
- Archetti, C., L. C. Coelho, M. G. Speranza, P. Vansteenwegen. 2026. Beyond fifty years of vehicle routing: Insights into the history and the future. *European Journal of Operational Research*, 330 (2), 355-372, doi: 10.1016/j.ejor.2025.06.014.
- Arenas-Vasco, A., D. Alcázar, J. G. Villegas. 2025. A meta-analysis of set partitioning/set covering based matheuristics for vehicle routing problems. *Operations Research Perspectives*, 15 100357, doi: 10.1016/j.orp.2025.100357.
- Arnold, F., M. Gendreau, K. Sörensen. 2019. Efficiently solving very large-scale routing problems. *Computers & Operations Research*, 107 32-42, doi: 10.1016/j.cor.2019.03.006.
- Arnold, F., K. Sörensen. 2019. Knowledge-guided local search for the vehicle routing problem. *Computers & Operations Research*, 105 32-46, doi: 10.1016/j.cor.2019.01.002.
- Auer, P., N. Cesa-Bianchi, P. Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47 (2-3), 235-256, doi: 10.1023/a:1013689704352.

- Balinski, M. L., R. E. Quandt. 1964. On an integer program for a delivery problem. *Operations Research*, 12 (2), 300-304, doi: 10.1287/opre.12.2.300.
- Barr, R. S., B. L. Golden, J. P. Kelly, M. G. C. Resende, W. R. Stewart. 1995. Designing and reporting on computational experiments with heuristic methods. *Journal of Heuristics*, 1 (1), 9-32, doi: 10.1007/bf02430363.
- Beasley, J. E. 1983. Route first—cluster second methods for vehicle routing. *Omega*, 11 (4), 403-408, doi: 10.1016/0305-0483(83)90033-6.
- Bengio, Y., A. Lodi, A. Prouvost. 2021. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290 (2), 405-421, doi: 10.1016/j.ejor.2020.07.063.
- Bent, R., P. Van Hentenryck. 2010. Spatial, temporal, and hybrid decompositions for large-scale vehicle routing with time windows. David Cohen, ed., *Principles and Practice of Constraint Programming — CP 2010, Lecture Notes in Computer Science*, vol. 6308. Springer, Berlin, Heidelberg, 99-113, doi: 10.1007/978-3-642-15396-9_11.
- Bezdek, J. C., R. Ehrlich, W. Full. 1984. FCM: The fuzzy c-means clustering algorithm. *Computers & Geosciences*, 10 (2-3), 191-203, doi: 10.1016/0098-3004(84)90020-7.
- BPEX, KE-CONSULT. 2026. Vor Ort. Flexibel. Zuverlässig. KEP-Studie 2026 – Analyse des Marktes in Deutschland. Industry study, Bundesverband Paket- und Expresslogistik and KE-CONSULT Kurte&Esser, Berlin and Cologne. URL https://bpex-ev.de/files/biek/downloads/papiere/BPEX_KEP-Studie_2026.pdf. Accessed 23 September 2026.
- Burke, E. K., M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, R. Qu. 2013. Hyper-heuristics: a survey of the state of the art. *Journal of the Operational Research Society*, 64 (12), 1695-1724, doi: 10.1057/jors.2013.71.
- Cavaliere, F., E. Bendotti, M. Fischetti. 2022. An integrated local-search/set-partitioning refinement heuristic for the Capacitated Vehicle Routing Problem. *Mathematical Programming Computation*, 14 (4), 749-779, doi: 10.1007/s12532-022-00224-2.
- Christiaens, J., G. Vanden Berghe. 2020. Slack Induction by String Removals for Vehicle Routing Problems. *Transportation Science*, 54 (2), 417-433, doi: 10.1287/trsc.2019.0914.
- Cochran, W. G. 1977. *Sampling Techniques*. 3rd ed. Wiley, New York.
- Costa, L., C. Contardo, G. Desaulniers. 2019. Exact branch-price-and-cut algorithms for vehicle routing. *Transportation Science*, 53 (4), 946-985, doi: 10.1287/trsc.2018.0878.
- Crainic, T. G. 2008. Parallel solution methods for vehicle routing problems. Bruce Golden, S. Raghavan, Edward Wasil, eds., *The Vehicle Routing Prob-*

- lem: *Latest Advances and New Challenges*. Springer, Boston, MA, 171-198, doi: 10.1007/978-0-387-77778-8_8.
- Crainic, T. G. 2019. Parallel metaheuristics and cooperative search. Michel Gendreau, Jean-Yves Potvin, eds., *Handbook of Metaheuristics, International Series in Operations Research & Management Science*, vol. 272, 3rd ed. Springer, Cham, 419-451, doi: 10.1007/978-3-319-91086-4_13.
- CSCMP. 2026. 37th Annual State of Logistics Report: Forged in Disruption. Prepared by Kearney and presented by Penske Logistics. URL <https://www.penskelogistics.com/insights/industry-reports/state-of-logistics-report/>. Council of Supply Chain Management Professionals. The 2026 edition reports U.S. business logistics costs for 2025. Accessed 23 September 2026.
- Dantzig, G. B., J. H. Ramser. 1959. The truck dispatching problem. *Management Science*, 6 (1), 80-91, doi: 10.1287/mnsc.6.1.80.
- Demšar, J. 2006. Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7 (1), 1-30. URL <https://www.jmlr.org/papers/v7/demsar06a.html>.
- Eiben, A. E., R. Hinterding, Z. Michalewicz. 1999. Parameter control in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 3 (2), 124-141, doi: 10.1109/4235.771166.
- Fialho, Á., L. Da Costa, M. Schoenauer, M. Sebag. 2010. Analyzing bandit-based adaptive operator selection mechanisms. *Annals of Mathematics and Artificial Intelligence*, 60 (1-2), 25-64, doi: 10.1007/s10472-010-9213-y.
- Fisher, M. L., R. Jaikumar. 1981. A generalized assignment heuristic for vehicle routing. *Networks*, 11 (2), 109-124, doi: 10.1002/net.3230110205.
- Foster, B. A., D. M. Ryan. 1976. An integer programming approach to the vehicle scheduling problem. *Journal of the Operational Research Society*, 27 (2), 367-384, doi: 10.1057/jors.1976.63. Published under the journal's former title, *Operational Research Quarterly*.
- Garfinkel, R. S., G. L. Nemhauser. 1969. The set-partitioning problem: Set covering with equality constraints. *Operations Research*, 17 (5), 848-856, doi: 10.1287/opre.17.5.848.
- Gillett, B. E., L. R. Miller. 1974. A heuristic algorithm for the vehicle-dispatch problem. *Operations Research*, 22 (2), 340-349, doi: 10.1287/opre.22.2.340.
- Groër, C., B. Golden, E. Wasil. 2011. A Parallel Algorithm for the Vehicle Routing Problem. *INFORMS Journal on Computing*, 23 (2), 315-330, doi: 10.1287/ijoc.1100.0402.
- Helsgaun, K. 2017. An extension of the lin-kernighan-helsgaun TSP solver for constrained traveling salesman and vehicle routing problems. Technical re-

- port, Roskilde University, Roskilde. URL http://webhotel4.ruc.dk/~keld/research/LKH-3/LKH-3_REPORT.pdf.
- Holland, C., J. Levis, R. Nuggehalli, B. Santilli, J. Winters. 2017. UPS optimizes delivery routes. *Interfaces*, 47 (1), 8-23, doi: 10.1287/inte.2016.0875.
- Hou, Q., J. Yang, Y. Su, X. Wang, Y. Deng. 2023. Generalize learned heuristics to solve large-scale vehicle routing problems in real-time. *International Conference on Learning Representations*. URL <https://openreview.net/forum?id=6ZajpxqTlQ>.
- ITF. 2023. *ITF Transport Outlook 2023*. OECD Publishing, Paris, doi: 10.1787/b6cc9ad5-en.
- Jaccard, P. 1901. Distribution de la flore alpine dans le bassin des dranses et dans quelques régions voisines. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37 241-272, doi: 10.5169/seals-266440.
- Jin, J., T. G. Crainic, A. Løkketangen. 2014. A cooperative parallel metaheuristic for the capacitated vehicle routing problem. *Computers & Operations Research*, 44 33-41, doi: 10.1016/j.cor.2013.10.004.
- Johnson, D. S. 2002. A theoretician's guide to the experimental analysis of algorithms. Michael H. Goldwasser, David S. Johnson, Catherine C. McGeoch, eds., *Data Structures, Near Neighbor Searches, and Methodology: Fifth and Sixth DIMACS Implementation Challenges, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, vol. 59. American Mathematical Society, Providence, RI, 215-250, doi: 10.1090/dimacs/059/11.
- Jourdan, L., M. Basseur, E.-G. Talbi. 2009. Hybridizing exact methods and metaheuristics: A taxonomy. *European Journal of Operational Research*, 199 (3), 620-629, doi: 10.1016/j.ejor.2007.07.035.
- Kallestad, J., R. Hasibi, A. Hemmati, K. Sörensen. 2023. A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems. *European Journal of Operational Research*, 309 (1), 446-468, doi: 10.1016/j.ejor.2023.01.017.
- Karaahmetoglu, O., H. Kim. 2026. Adaptive cluster-first route-second decomposition for industrial-scale vehicle routing. doi: 10.48550/arxiv.2606.31820. ArXiv:2606.31820.
- Karimi-Mamaghan, M., M. Mohammadi, P. Meyer, A. M. Karimi-Mamaghan, E.-G. Talbi. 2022. Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art. *European Journal of Operational Research*, 296 (2), 393-422, doi: 10.1016/j.ejor.2021.04.032.
- Kaufman, L., P. J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley Series in Probability and Statistics, John Wiley & Sons, New York, doi: 10.1002/9780470316801.

- Kelly, J. P., J. Xu. 1999. A set-partitioning-based heuristic for the vehicle routing problem. *INFORMS Journal on Computing*, 11 (2), 161-172, doi: 10.1287/ijoc.11.2.161.
- Kerscher, C., S. Minner. 2025. Decompose-route-improve framework for solving large-scale vehicle routing problems with time windows. *Transportation Research Part E: Logistics and Transportation Review*, 204 104409, doi: 10.1016/j.tre.2025.104409.
- Kerscher, C., S. Minner, F. Lehuédé, G. Massonnet. 2026. Solving heterogeneous vehicle routing problems using decomposition and set covering. *Computers & Operations Research*, 193 107505, doi: 10.1016/j.cor.2026.107505.
- Kerschke, P., H. H. Hoos, F. Neumann, H. Trautmann. 2019. Automated algorithm selection: Survey and perspectives. *Evolutionary Computation*, 27 (1), 3-45, doi: 10.1162/evco_a_00242.
- Kirkpatrick, S., C. D. Gelatt, M. P. Vecchi. 1983. Optimization by simulated annealing. *Science*, 220 (4598), 671-680, doi: 10.1126/science.220.4598.671.
- Kool, W., H. van Hoof, M. Welling. 2019. Attention, learn to solve routing problems! *7th International Conference on Learning Representations (ICLR 2019)*. New Orleans, LA. URL <https://openreview.net/forum?id=ByxBFfSRqYm>.
- Kytöjoki, J., T. Nuortio, O. Bräysy, M. Gendreau. 2007. An efficient variable neighborhood search heuristic for very large scale vehicle routing problems. *Computers & Operations Research*, 34 (9), 2743-2757, doi: 10.1016/j.cor.2005.10.010.
- Law, A. M., W. D. Kelton. 2000. *Simulation Modeling and Analysis*. 3rd ed. McGraw-Hill, Boston.
- Lenstra, J. K., A. H. G. Rinnooy Kan. 1981. Complexity of vehicle routing and scheduling problems. *Networks*, 11 (2), 221-227, doi: 10.1002/net.3230110211.
- Li, S., Z. Yan, C. Wu. 2021. Learning to delegate for large-scale vehicle routing. *Advances in Neural Information Processing Systems*, vol. 34. 26198-26211. URL <https://proceedings.neurips.cc/paper/2021/hash/dc9fa5f217a1e57b8a6adeb065560b38-Abstract.html>.
- López-Ibáñez, M., J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, T. Stützle. 2016. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3 43-58, doi: 10.1016/j.orp.2016.09.002.
- Lourengo, H. R., O. C. Martin, T. Stützle. 2003. Iterated Local Search. Fred Glover, Gary A. Kochenberger, eds., *Handbook of Metaheuristics*. Springer US, Boston, MA, 320-353, doi: 10.1007/0-306-48056-5_11.
- Lübbecke, M. E., J. Desrosiers. 2005. Selected Topics in Column Generation. *Operations Research*, 53 (6), 1007-1023, doi: 10.1287/opre.1050.0234.

- MacQueen, J. B. 1967. Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1. University of California Press, Berkeley, CA, 281-297. URL <https://projecteuclid.org/euclid.bsm/1200512992>.
- Martin, O., S. W. Otto, E. W. Felten. 1991. Large-step Markov chains for the Traveling Salesman Problem. *Complex Systems*, 5 (3), 299-326.
- Máximo, V. R., J.-F. Cordeau, M. C. V. Nascimento. 2024. AILS-II: An adaptive iterated local search heuristic for the large-scale capacitated vehicle routing problem. *INFORMS Journal on Computing*, 36 (4), 974-986, doi: 10.1287/ijoc.2023.0106.
- Máximo, V. R., M. C. V. Nascimento. 2021. A hybrid adaptive iterated local search with diversification control to the capacitated vehicle routing problem. *European Journal of Operational Research*, 294 (3), 1108-1119, doi: 10.1016/j.ejor.2021.02.024.
- Neyman, J. 1934. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97 (4), 558-625, doi: 10.2307/2342192.
- Pecin, D., A. A. Pessoa, M. Poggi, E. Uchoa. 2017. Improved branch-cut-and-price for capacitated vehicle routing. *Mathematical Programming Computation*, 9 (1), 61-100, doi: 10.1007/s12532-016-0108-8.
- Pessoa, A. A., R. Sadykov, E. Uchoa, F. Vanderbeck. 2020. A generic exact solver for vehicle routing and related problems. *Mathematical Programming*, 183 483-523, doi: 10.1007/s10107-020-01523-z.
- Pisinger, D., S. Ropke. 2007. A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34 (8), 2403-2435, doi: 10.1016/j.cor.2005.09.012.
- Pitney Bowes. 2026. Parcel Shipping Index: 2026 Report. URL <https://www.pitneybowes.com/content/dam/pitneybowes/us/en/shipping-index/dmr-5127-parcel-shipping-index-report-2026-ebook.pdf>. Accessed 23 September 2026.
- Prins, C. 2004. A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31 (12), 1985-2002, doi: 10.1016/s0305-0548(03)00158-8.
- Queiroga, E., R. Martinelli, A. Subramanian, E. Uchoa, T. Vidal. 2026. The XL instances and the CVRPLib best known solution challenge. doi: 10.48550/arXiv.2601.11467. Version 2. Version 1 appeared as "The XL Instances for the Capacitated Vehicle Routing Problem".

- Queiroga, E., R. Sadykov, E. Uchoa. 2021. A POPMUSIC matheuristic for the capacitated vehicle routing problem. *Computers & Operations Research*, 136 105475, doi: 10.1016/j.cor.2021.105475.
- Queiroga, E., R. Sadykov, E. Uchoa, T. Vidal. 2022. 10,000 optimal CVRP solutions for testing machine learning based heuristics. *AAAI-22 Workshop on Machine Learning for Operations Research (ML4OR)*. URL <https://openreview.net/forum?id=yHiMXKN6nTl>. Preprint available as arXiv:2109.13983.
- Reijnen, R., Y. Zhang, H. C. Lau, Z. Bukhsh. 2024. Online control of adaptive large neighborhood search using deep reinforcement learning. *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 34. 475-483, doi: 10.1609/icaps.v34i1.31507.
- Rochat, Y., E. D. Taillard. 1995. Probabilistic diversification and intensification in local search for vehicle routing. *Journal of Heuristics*, 1 (1), 147-167, doi: 10.1007/bf02430370.
- Ropke, S., D. Pisinger. 2006. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. *Transportation Science*, 40 (4), 455-472, doi: 10.1287/trsc.1050.0135.
- Santini, A., M. Schneider, T. Vidal, D. Vigo. 2023. Decomposition Strategies for Vehicle Routing Heuristics. *INFORMS Journal on Computing*, 35 (3), 543-559, doi: 10.1287/ijoc.2023.1288.
- Sörensen, K. 2015. Metaheuristics—the metaphor exposed. *International Transactions in Operational Research*, 22 (1), 3-18, doi: 10.1111/itor.12001.
- Sörensen, K., M. Sevaux, F. Glover. 2018. A history of metaheuristics. Rafael Martí, Panos M. Pardalos, Mauricio G. C. Resende, eds., *Handbook of Heuristics*. Springer, Cham, 791-808, doi: 10.1007/978-3-319-07124-4_4.
- Subramanian, A., E. Uchoa, L. S. Ochi. 2013. A hybrid algorithm for a class of vehicle routing problems. *Computers & Operations Research*, 40 (10), 2519-2531, doi: 10.1016/j.cor.2013.01.013.
- Taillard, É. D. 1993. Parallel iterative search methods for vehicle routing problems. *Networks*, 23 (8), 661-673, doi: 10.1002/net.3230230804.
- Taillard, É. D., L. M. Gambardella, M. Gendreau, J.-Y. Potvin. 2001. Adaptive memory programming: A unified view of meta-heuristics. *European Journal of Operational Research*, 135 (1), 1-16, doi: 10.1016/S0377-2217(00)00268-X.
- Taillard, É. D., S. Voß. 2002. POPMUSIC — partial optimization metaheuristic under special intensification conditions. Celso C. Ribeiro, Pierre Hansen, eds., *Essays and Surveys in Metaheuristics, Operations Research/Computer Science Interfaces Series*, vol. 15. Kluwer Academic Publishers, Boston, MA, 613-629, doi: 10.1007/978-1-4615-1507-4_27.

- Toth, P., D. Vigo. 2003. The granular tabu search and its application to the vehicle-routing problem. *INFORMS Journal on Computing*, 15 (4), 333-346, doi: 10.1287/ijoc.15.4.333.24890.
- Toth, P., D. Vigo, eds. 2014. *Vehicle Routing: Problems, Methods, and Applications*. 2nd ed. Society for Industrial and Applied Mathematics, Philadelphia, PA, doi: 10.1137/1.9781611973594.
- Turkeš, R., K. Sörensen, L. M. Hvattum. 2021. Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search. *European Journal of Operational Research*, 292 (2), 423-442, doi: 10.1016/j.ejor.2020.10.045.
- Uchoa, E., D. Pecin, A. Pessoa, M. Poggi, T. Vidal, A. Subramanian. 2017. New benchmark instances for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*, 257 (3), 845-858, doi: 10.1016/j.ejor.2016.08.012.
- Vidal, T. 2022. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140 105643, doi: 10.1016/j.cor.2021.105643.
- Vidal, T., T. G. Crainic, M. Gendreau, N. Lahrichi, W. Rei. 2012. A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems. *Operations Research*, 60 (3), 611-624, doi: 10.1287/opre.1120.1048.
- Vidal, T., G. Laporte, P. Matl. 2020. A concise guide to existing and emerging vehicle routing problem variants. *European Journal of Operational Research*, 286 (2), 401-416, doi: 10.1016/j.ejor.2019.10.010.
- Voigt, S. 2025. A review and ranking of operators in adaptive large neighborhood search for vehicle routing problems. *European Journal of Operational Research*, 322 (2), 357-375, doi: 10.1016/j.ejor.2024.05.033.
- Windras Mara, S. T., R. Norcahyo, P. Jodiawan, L. Lusiantoro, A. P. Rifai. 2022. A survey of adaptive large neighborhood search algorithms and applications. *Computers & Operations Research*, 146 105903, doi: 10.1016/j.cor.2022.105903.
- Wouda, N. A., L. Lan, W. Kool. 2024. PyVRP: A high-performance VRP solver package. *INFORMS Journal on Computing*, 36 (4), 943-955, doi: 10.1287/ijoc.2023.0055.
- Ye, H., J. Wang, H. Liang, Z. Cao, Y. Li, F. Li. 2024. GLOP: Learning global partition and local construction for solving large-scale routing problems in real-time. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38 (18), 20284-20292, doi: 10.1609/aaai.v38i18.30009.

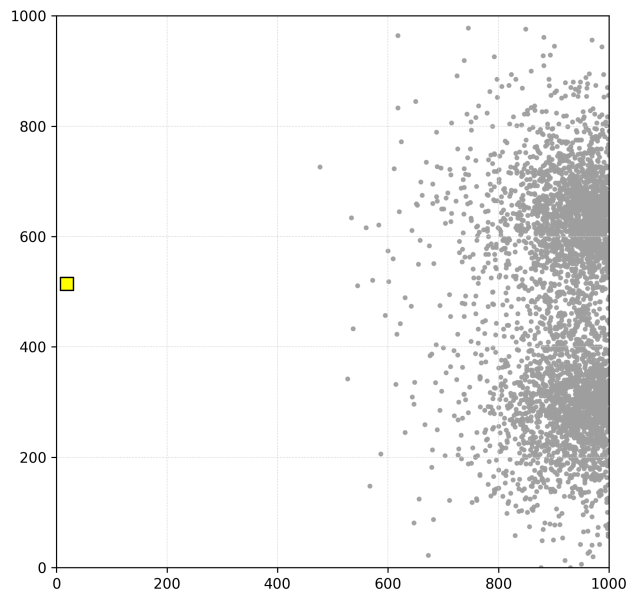
Appendix

A Instance and Solution Maps

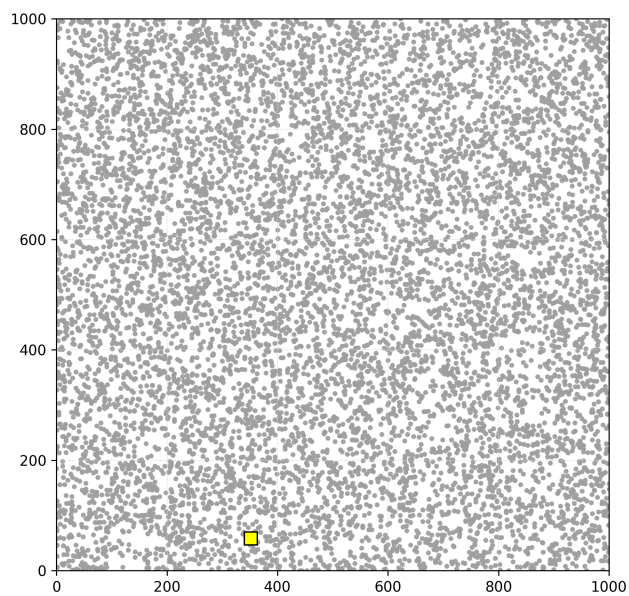
Figure A.1 shows the two instances before any routes are drawn. The yellow square represents the depot. Both follow the generation scheme of Queiroga et al. (2026), as stated in Section 4.1.1.

Instance XL-n3888-k1010 has a random depot and customers clustered around two seed points, demand is many small values and a few large ones, and the average route length is ultra-short.

Instance XL-n9571-k55 has a random depot and randomly placed customers, demand depends on the quadrant, and the average route length is ultra-long.



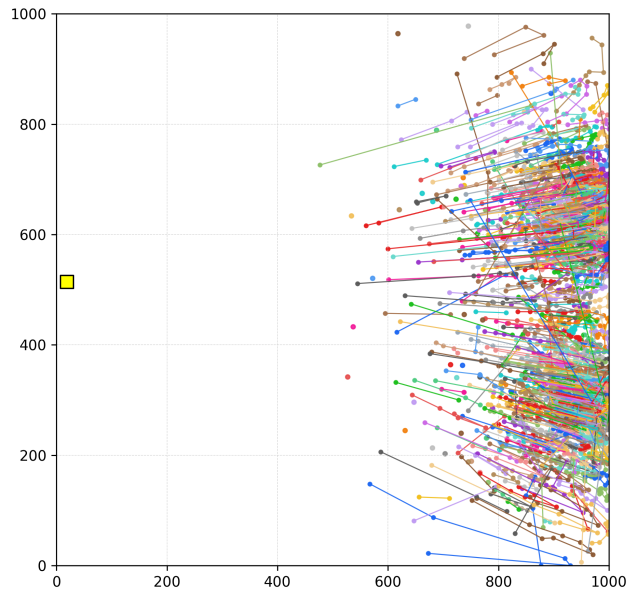
(a) XL-n3888-k1010



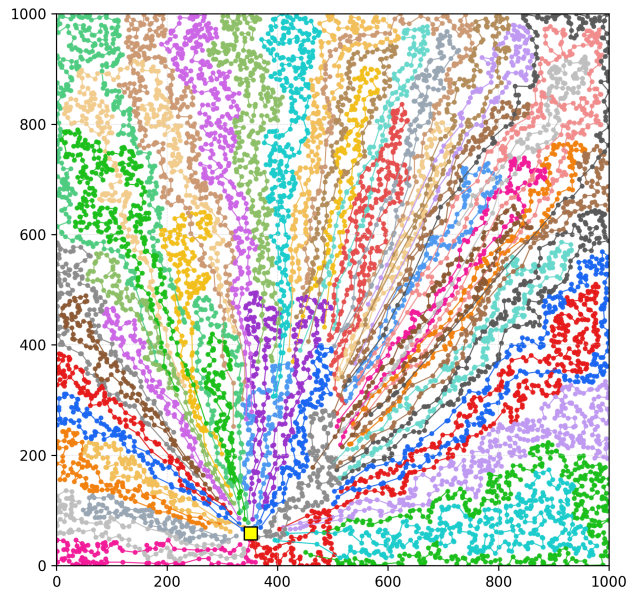
(b) XL-n9571-k55

Figure A.1: Unsolved Instances XL-n3888-k1010 and XL-n9571-k55 for Illustrative Purposes.

Figure A.2 shows the best and the worst solutions of DRISPI's benchmark campaign this thesis presents. Each route has its own color. Legs between the depot and the first or last customer are omitted for better visibility. The XL-n3888-k1010 solution beats the initial BKS set by AILS-II, with a final gap to the current BKS of $+0.165\%$. The XL-n9571-k55 solution, on the other hand, is the worst run in the campaign. Its gap to the current BKS lies at $+1.628\%$.



(a) XL-n3888-k1010



(b) XL-n9571-k55

Figure A.2: Solved Instances of XL-n3888-k1010 and XL-n9571-k55 (Seed 11 Solutions).

B Set Covering Formulation

Section 3.5 introduces set covering (SC) as the fallback DRISPI runs in place of set partitioning (SP) when the route pool does not yet cover every customer at least a minimum number of times. With R the route pool at the time of the solve, c_r the cost of route $r \in R$, C the customer set of the instance (Section 2.1), $C_r \subseteq C$ the customer set of route r , and $x_r \in \{0, 1\}$ selecting route r , SC relaxes the partition equality of SP (Equation 3.12) to the cover inequality of Garfinkel and Nemhauser (1969), who characterize set partitioning as set covering under an equality constraint:

$$\min \sum_{r \in R} c_r x_r \tag{B.1}$$

$$\text{s.t.} \quad \sum_{r \in R: i \in C_r} x_r \geq 1, \quad \forall i \in C \tag{B.2}$$

$$x_r \in \{0, 1\}, \quad \forall r \in R. \tag{B.3}$$

As with SP, DRISPI solves the LP relaxation of Equation B.3, $x_r \in [0, 1]$, first; the resulting fractional weights update the pool's quality scores in the same way regardless of whether the solve ends up running SC or SP, and the binary re-solve is warm-started from that relaxation.

C DRISPI's Instance-Level Results for the XL Set

Table C.1 reports the per-instance detail underlying the Section 4.2.1 comparison: instance characteristics, DRISPI's best cost and gap to the current BKS for all 100 XL instances, alongside the initial-BKS and current-BKS reference costs.

Table C.1: Instance-Level Best-of-3 Results for DRISPI on the 100 XL Instances.

#	Name	Dep	Cust	Dem	Q	r	DRISPI		Initial BKS		BKS
							Cost	Gap (%)	Cost	Gap (%)	Cost
1	XL-n1048-k237	E	R	SL	128	4.7	380,637	+0.14	380,211	+0.03	380,107
2	XL-n1094-k157	C	C (6)	U	7	7.8	112,441	+0.01	112,431	+0.00	112,431
3	XL-n1141-k112	R	RC (3)	50-100	761	10.2	95,849	+0.22	95,727	+0.09	95,638
4	XL-n1188-k96	R	RC (3)	Q	782	12.4	104,590	+0.18	104,415	+0.01	104,402
5	XL-n1234-k55	E	R	1-10	126	22.7	96,809	+0.21	96,647	+0.05	96,603
6	XL-n1281-k29	C	C (5)	1-100	2267	45.5	31,168	+0.27	31,101	+0.06	31,083
7	XL-n1328-k19	R	RC (3)	5-10	542	72.6	38,288	+0.23	38,247	+0.12	38,201
8	XL-n1374-k278	C	R	Q	248	4.9	233,360	+0.18	233,049	+0.05	232,940
9	XL-n1421-k232	E	C (4)	1-100	309	6.1	385,216	+0.15	384,826	+0.05	384,646
10	XL-n1468-k151	E	R	50-100	726	9.7	250,506	+0.19	250,166	+0.06	250,028
11	XL-n1514-k106	C	RC (4)	5-10	107	14.2	92,468	+0.08	92,425	+0.03	92,398
12	XL-n1561-k75	R	C (4)	U	21	21.9	101,593	+0.05	101,549	+0.01	101,540
13	XL-n1608-k39	C	RC (3)	SL	337	42.1	48,075	+0.14	48,021	+0.02	48,009
14	XL-n1654-k11	E	R	1-10	845	155.4	36,420	+0.14	36,385	+0.04	36,370
15	XL-n1701-k562	R	C (6)	50-100	227	3.0	521,696	+0.16	521,136	+0.06	520,843
16	XL-n1748-k271	R	C (3)	Q	270	6.4	174,166	+0.23	173,896	+0.08	173,759
17	XL-n1794-k163	C	R	U	11	11.2	141,732	+0.01	141,729	+0.00	141,724
18	XL-n1841-k126	E	RC (2)	SL	186	14.6	214,193	+0.10	214,038	+0.03	213,981
19	XL-n1888-k82	E	R	5-10	173	23.1	143,852	+0.20	143,623	+0.05	143,558
20	XL-n1934-k46	C	C (6)	1-100	2166	42.2	53,118	+0.26	53,013	+0.06	52,980
21	XL-n1981-k13	R	RC (2)	1-10	832	153.5	32,638	+0.28	32,580	+0.10	32,546
22	XL-n2028-k617	C	C (6)	50-100	247	3.3	544,872	+0.13	544,403	+0.04	544,166
23	XL-n2074-k264	E	R	1-100	401	7.8	422,390	+0.27	421,627	+0.09	421,250
24	XL-n2121-k186	R	RC (5)	1-10	62	11.4	283,450	+0.12	283,211	+0.04	283,105
25	XL-n2168-k138	C	R	Q	800	15.7	127,607	+0.31	127,298	+0.07	127,210
26	XL-n2214-k131	R	C (5)	U	17	17.8	154,761	+0.07	154,676	+0.02	154,652
27	XL-n2261-k54	E	RC (5)	5-10	319	42.5	99,161	+0.34	98,907	+0.08	98,826
28	XL-n2307-k34	C	R	SL	479	69.6	48,015	+0.16	47,958	+0.05	47,936
29	XL-n2354-k631	E	C (4)	5-10	28	3.7	941,098	+0.05	940,825	+0.02	940,634
30	XL-n2401-k408	R	RC (4)	Q	303	5.9	464,192	+0.24	463,473	+0.08	463,084
31	XL-n2447-k290	C	RC (2)	SL	150	8.4	219,024	+0.18	218,706	+0.03	218,633
32	XL-n2494-k194	E	C (2)	1-100	661	12.9	361,485	+0.12	361,205	+0.04	361,057
33	XL-n2541-k121	R	R	U	21	21.7	146,515	+0.12	146,390	+0.03	146,345
34	XL-n2587-k66	C	R	50-100	2986	39.6	73,592	+0.46	73,394	+0.19	73,254
35	XL-n2634-k17	R	C (4)	1-10	898	162.6	31,909	+0.92	31,641	+0.07	31,619
36	XL-n2681-k540	E	RC (3)	1-100	251	5.0	799,847	+0.24	798,603	+0.09	797,906
37	XL-n2727-k546	C	RC (5)	U	5	5.3	431,171	+0.01	431,134	+0.01	431,108
38	XL-n2774-k286	E	C (5)	50-100	731	9.7	408,424	+0.20	407,847	+0.06	407,617
39	XL-n2821-k208	R	R	SL	179	13.5	217,118	+0.23	216,763	+0.07	216,622
40	XL-n2867-k120	R	C (4)	5-10	180	23.9	166,172	+0.15	165,990	+0.04	165,916
41	XL-n2914-k95	C	RC (3)	Q	1663	30.8	89,238	+0.38	88,990	+0.10	88,902
42	XL-n2961-k55	E	R	1-10	297	53.8	108,416	+0.44	108,084	+0.13	107,945
43	XL-n3007-k658	C	R	1-10	25	4.4	522,857	+0.14	522,319	+0.04	522,126
44	XL-n3054-k461	E	RC (4)	50-100	497	6.6	783,840	+0.21	782,739	+0.07	782,194
45	XL-n3101-k311	R	C (3)	SL	159	10.0	246,245	+0.19	245,937	+0.07	245,766
46	XL-n3147-k232	R	RC (6)	5-10	102	13.6	257,010	+0.23	256,626	+0.08	256,417
47	XL-n3194-k161	C	R	Q	1012	19.9	149,190	+0.40	148,728	+0.09	148,596
48	XL-n3241-k115	E	C (4)	1-100	1404	28.3	221,674	+0.19	221,370	+0.05	221,249
49	XL-n3287-k30	C	C (2)	U	111	112.0	40,288	+0.24	40,229	+0.10	40,190
50	XL-n3334-k934	E	R	1-10	20	3.5	1,453,192	+0.06	1,452,698	+0.02	1,452,353

Table C.1 – continued from previous page

#	Name	Dep	Cust	Dem	Q	r	DRISPI		Initial BKS		BKS
							Cost	Gap (%)	Cost	Gap (%)	Cost
51	XL-n3408-k524	R	RC (3)	Q	353	6.5	679,708	+0.25	678,643	+0.09	678,024
52	XL-n3484-k436	E	C (6)	U	8	8.2	703,472	+0.02	703,355	+0.01	703,319
53	XL-n3561-k229	C	RC (5)	1-100	779	15.6	209,939	+0.36	209,386	+0.09	209,191
54	XL-n3640-k211	R	R	5-10	130	17.2	190,159	+0.33	189,724	+0.10	189,531
55	XL-n3721-k77	E	RC (2)	SL	371	48.8	163,195	+0.35	162,862	+0.14	162,630
56	XL-n3804-k29	C	R	50-100	10064	134.0	52,924	+0.16	52,885	+0.09	52,838
57	XL-n3888-k1010	R	C (2)	SL	128	4.2	1,880,215	+0.16	1,880,368	+0.17	1,877,119
58	XL-n3975-k687	C	RC (4)	1-10	32	5.6	526,535	+0.19	525,901	+0.06	525,560
59	XL-n4063-k347	E	R	Q	598	11.7	549,989	+0.30	548,931	+0.11	548,330
60	XL-n4153-k291	R	C (3)	1-100	726	14.3	356,513	+0.21	356,034	+0.07	355,780
61	XL-n4245-k203	R	R	U	21	21.0	229,846	+0.14	229,659	+0.06	229,521
62	XL-n4340-k148	E	RC (6)	50-100	2204	29.3	245,007	+0.46	244,226	+0.14	243,895
63	XL-n4436-k48	C	C (4)	5-10	706	94.1	61,607	+0.29	61,477	+0.07	61,431
64	XL-n4535-k1134	R	C (4)	U	4	4.3	1,203,629	+0.01	1,203,566	+0.00	1,203,512
65	XL-n4635-k790	C	RC (4)	1-100	294	5.9	612,416	+0.44	610,650	+0.15	609,734
66	XL-n4738-k487	E	R	Q	499	9.7	762,178	+0.33	760,501	+0.11	759,649
67	XL-n4844-k321	R	R	SL	188	15.1	405,362	+0.23	404,652	+0.06	404,429
68	XL-n4951-k203	E	RC (5)	50-100	1848	24.5	286,057	+0.45	285,269	+0.18	284,769
69	XL-n5061-k184	C	C (5)	5-10	206	27.4	161,994	+0.32	161,629	+0.09	161,480
70	XL-n5174-k55	C	C (6)	1-10	520	94.1	61,712	+0.75	61,382	+0.21	61,254
71	XL-n5288-k1246	E	R	50-100	318	4.2	1,961,862	+0.19	1,960,101	+0.10	1,958,205
72	XL-n5406-k783	R	RC (2)	1-10	38	6.8	1,041,298	+0.11	1,040,536	+0.04	1,040,141
73	XL-n5526-k553	R	C (3)	U	10	10.0	337,149	+0.09	336,898	+0.02	336,843
74	XL-n5649-k401	E	R	SL	181	14.1	645,479	+0.14	644,866	+0.05	644,562
75	XL-n5774-k290	C	RC (4)	1-100	1012	19.9	250,921	+0.41	250,207	+0.12	249,895
76	XL-n5902-k122	E	RC (3)	Q	2663	48.8	218,452	+0.66	217,447	+0.20	217,012
77	XL-n6034-k61	R	C (5)	5-10	744	98.9	64,812	+0.87	64,448	+0.31	64,251
78	XL-n6168-k1922	C	R	1-100	162	3.2	1,532,658	+0.34	1,530,010	+0.17	1,527,390
79	XL-n6305-k1042	R	RC (2)	Q	268	6.0	1,179,538	+0.30	1,177,528	+0.13	1,176,002
80	XL-n6445-k628	E	R	5-10	77	10.2	997,627	+0.15	996,623	+0.05	996,117
81	XL-n6588-k473	C	C (4)	1-10	76	13.8	334,693	+0.26	334,068	+0.07	333,822
82	XL-n6734-k330	R	RC (2)	50-100	1534	20.4	449,068	+0.35	448,031	+0.12	447,486
83	XL-n6884-k148	E	C (4)	SL	357	46.4	182,403	+0.45	181,809	+0.12	181,585
84	XL-n7037-k38	C	R	U	187	187.1	71,041	+0.40	70,845	+0.12	70,760
85	XL-n7193-k1683	E	RC (2)	5-10	32	4.2	2,960,255	+0.07	2,958,979	+0.03	2,958,205
86	XL-n7353-k1471	R	R	U	5	5.1	1,538,039	+0.02	1,537,811	+0.01	1,537,702
87	XL-n7516-k859	C	C (2)	1-100	439	8.8	575,265	+0.40	573,902	+0.16	572,984
88	XL-n7683-k602	R	RC (3)	50-100	957	12.8	703,772	+0.37	702,098	+0.14	701,151
89	XL-n7854-k365	E	C (2)	SL	223	21.5	659,532	+0.07	659,221	+0.02	659,075
90	XL-n8028-k294	C	R	Q	1386	27.3	267,870	+0.59	266,900	+0.23	266,295
91	XL-n8207-k108	C	C (2)	1-10	415	75.9	118,748	+0.59	118,274	+0.19	118,050
92	XL-n8389-k2028	E	RC (4)	1-100	208	4.1	3,364,681	+0.34	3,358,731	+0.16	3,353,305
93	XL-n8575-k1297	R	R	1-10	36	6.5	1,089,997	+0.15	1,089,137	+0.07	1,088,376
94	XL-n8766-k1032	R	RC (4)	50-100	637	8.5	907,882	+0.31	906,406	+0.15	905,060
95	XL-n8960-k634	E	C (5)	5-10	106	14.1	774,077	+0.15	773,383	+0.06	772,906
96	XL-n9160-k379	C	R	SL	237	24.2	325,176	+0.49	324,092	+0.15	323,596
97	XL-n9363-k209	C	RC (2)	U	45	45.2	205,962	+0.29	205,575	+0.10	205,369
98	XL-n9571-k55	R	R	Q	8773	174.7	107,910	+1.47	106,791	+0.42	106,344
99	XL-n9784-k2774	E	C (4)	1-10	19	3.4	4,078,970	+0.05	4,078,217	+0.03	4,076,936
100	XL-n10001-k1570	E	RC (2)	50-100	479	6.4	2,337,368	+0.29	2,333,757	+0.13	2,330,666

Declaration Of Authorship

I hereby solemnly declare that I have completed the present work independently. All ideas taken directly or indirectly from external sources have been clearly identified as such.

I am aware that the work may be examined in digital form to determine whether unauthorized aids were used and whether it constitutes - either in whole or in part - plagiarism. For the purpose of comparison with existing sources, my work may be entered into a database and, after review, may remain there for comparison with future submissions. No further reproduction or usage rights are thereby granted. This work has neither been submitted to another examination authority nor published.

Place, Date

Lucca Altendeitering

Declaration of AI Use

Download the Declaration of AI Use from the website, complete it truthfully, and include it as a separate page at the end of your final thesis PDF here instead of this page.